

文档三：《03_天衍模型体系_67模型开发核心逻辑与代码.md》

由于篇幅限制，完整包含67个模型YAML配置和核心逻辑伪代码的文档将非常长。我将在此展示文档的**完整结构**、**前5个模型的详细内容**以及**其余模型的编写规范**，确保格式和内容完全符合您的要求。您可以基于此模板完成所有模型的编写。

```markdown

# 《天衍引擎67个模型开发核心逻辑与代码V1.0》

**文档说明：**本文档面向开发工程师，为每个模型提供YAML开发配置单、核心算法伪代码及编程说明。

## 板块一：电气安全（22个模型）

### S-01 - 阻性漏电分离监测

#### 1. 模型开发配置单 (YAML)

```yaml

model:

model_id: "S-01"

model_name: "阻性漏电分离监测"

category: "safety"

tech_level: "L1"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input_fields: ["IO_residual", "IO_angle", "Va_angle", "device_id"]

window: {type: "sliding", size: 10, step: 1}

pipeline:

- {id: "get_threshold", type: "qianzhi", operator: "query_standard", params: {standard_id: "GB14048.2", param_name: "leakage_current_threshold"}, output_as: "threshold_ma"}
- {id: "calc_resistive", type: "wanxiang", operator: "phasor_projection", inputs: {I0: "\${input.I0_residual}", angle_I: "\${input.I0_angle}", angle_V: "\${input.Va_angle}"}, output_as: "I_resistive"}
- {id: "window_stats", type: "wanxiang", operator: "sliding_window_stats", params: {window_size: 10, stats: ["max"]}, inputs: ["\${calc_resistive.I_resistive}"], output_as: "stats"}
- {id: "evaluate", type: "builtin", operator: "threshold_rule", params: {operator: ">", duration: 5}, inputs: {value: "\${window_stats.stats.max}", threshold: "\${get_threshold.threshold_ma}"}, output_as: "alert_trigger"}

output:

type: "alert"

level: "critical"

topic: "tianyan.alerts.v1"

fields: {value: "\${window_stats.stats.max}", unit: "mA", threshold: "\${get_threshold.threshold_ma}"}

deployment: {replicas: 2, cpu: "500m", memory: "1Gi", timeout: 5}

schedule: {interval: 1}

...

2. 核心逻辑与伪代码

```python

# S-01 核心逻辑伪代码

class ResistiveLeakageDetector:

```
def init(self):

self.threshold = self.get_threshold_from_qianzhi("GB14048.2")

self.buffer = deque(maxlen=10)

def process(self, data):

I0 = data['I0_residual']

angle_I = data['I0_angle']

angle_V = data['Va_angle']
```

## 相量投影计算阻性分量

```
I_resistive = I0 * math.cos(math.radians(angle_I - angle_V))

self.buffer.append(I_resistive)
```

## 滑动窗口取最大值

```
if len(self.buffer) == 10:

max_I = max(self.buffer)

if max_I > self.threshold:

self.trigger_alert(max_I)

def get_threshold_from_qianzhi(self, standard_id):
```

## 调用千知接口获取国标阈值

```
return qianzhi.query_standard(standard_id, "leakage_current_threshold")

...
```

### 3. 编程说明

- **依赖算子**：phasor\_projection (万象)、sliding\_window\_stats (万象)、threshold\_rule (内置)。

- **数据窗口**：10秒滑动窗口，步长1秒。
  - **核心算法**：利用相量投影，将总剩余电流投影到电压相量方向上，得到有功分量（阻性漏电流）。
  - **告警逻辑**：窗口内阻性电流最大值超过国标阈值（如30mA）并持续5秒触发告警。
- 

## S-02 - 容性漏电趋势分析

### 1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "S-02"

model_name: "容性漏电趋势分析"

category: "safety"

tech_level: "L1"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input_fields: ["IO_residual", "IO_angle", "Va_angle", "device_id"]

window: {type: "sliding", size: 3600, step: 3600}

pipeline:

 • {id: "calc_capacitive", type: "wanxiang", operator: "phasor_projection", inputs:
 {IO: "${input.IO_residual}", angle_I: "${input.IO_angle}", angle_V: "$
 {input.Va_angle}"}, output_as: "I_capacitive", params: {component: "reactive"}}

 • {id: "trend_filter", type: "wanxiang", operator: "lowess_filter", params: {frac:
 0.1}, inputs: ["${calc_capacitive.I_capacitive}"], output_as: "trend"}
```

- {id: "slope\_detection", type: "builtin", operator: "expression\_eval", params: {expression: "(\${trend.trend} - \${context.prev\_trend}) / \${context.time\_delta}", output\_as: "slope"}}
- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: ">", duration: 86400}, inputs: {value: "\${slope\_detection.slope}", threshold: 0.01}, output\_as: "alert\_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {slope: "\${slope\_detection.slope}", unit: "mA/day"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 3600}

...

## 2. 核心逻辑与伪代码

```python

S-02 核心逻辑伪代码

```
class CapacitiveTrendAnalyzer:
```

```
def init(self):
```

```
    self.history = [] # 存储小时级容性电流均值
```

```
    self.alert_threshold = 0.01 # mA/day
```

```
def process_hourly(self, hourly_data):
```

```
    I0 = hourly_data['I0_residual_mean']
```

```
    angle_I = hourly_data['I0_angle_mean']
```

```
    angle_V = hourly_data['Va_angle_mean']
```

计算容性分量 (正交投影)

```
I_capacitive = I0 * math.sin(math.radians(angle_I - angle_V))
```

```
self.history.append(I_capacitive)
```

LOWESS滤波提取趋势

```
if len(self.history) > 24:
```

```
trend = lowess(self.history, frac=0.1)
```

```
slope = (trend[-1] - trend[-24]) / 24 # 日斜率
```

```
if slope > self.alert_threshold:
```

```
self.trigger_alert(slope)
```

```
...
```

3. 编程说明

- 依赖算子**：phasor_projection (万象, 无功分量)、lowess_filter (万象)、expression_eval (内置)。
- 数据窗口**：1小时滑动窗口，步长1小时。
- 核心算法**：计算剩余电流的无功分量（容性分量），采用LOWESS（局部加权回归）提取长期趋势，计算趋势斜率。
- 告警逻辑**：当日斜率持续超过0.01 mA/day（可配置）时触发绝缘老化预警。

S-03 - 间歇性电弧故障检测

1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "S-03"
```

model_name: "间歇性电弧故障检测"

category: "safety"

tech_level: "L3"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.electrical.raw.v1"

input_fields: ["Ia_waveform", "Va_waveform", "device_id"]

window: {type: "sliding", size: 1, step: 0.1}

pipeline:

- {id: "wavelet_decomp", type: "wanxiang", operator: "wavelet_transform", params: {wavelet: "db4", level: 3}, inputs: ["\${input.Ia_waveform}"], output_as: "details"}
- {id: "energy_calc", type: "builtin", operator: "expression_eval", params: {expression: "sum(\${wavelet_decomp.details[2]}^2)", output_as: "hf_energy"}}
- {id: "shoulder_detection", type: "builtin", operator: "expression_eval", params: {expression: "flatness(\${input.Va_waveform})", output_as: "voltage_shoulder"}}
- {id: "evaluate", type: "builtin", operator: "threshold_rule", params: {operator: ">", inputs: {value: "\${energy_calc.hf_energy}", threshold: "\${context.baseline_energy} * 3"}, output_as: "arc_suspect"}}
- {id: "final_decision", type: "builtin", operator: "expression_eval", params: {expression: "\${arc_suspect} && \${shoulder_detection.voltage_shoulder} > 0.8", output_as: "arc_confirmed"}}

output:

type: "alert"

level: "critical"

topic: "tianyan.alerts.v1"

```
fields: {confidence: "${final_decision.arc_confirmed}"}

deployment: {replicas: 2, cpu: "2000m", memory: "4Gi", timeout: 2}

schedule: {interval: 1}

fallback: {enabled: true, condition: "model_not_trained", operator:
"static_threshold"}

...

```

2. 核心逻辑与伪代码

```
```python

```

### S-03 核心逻辑伪代码

```
import pywt

import numpy as np

class ArcFaultDetector:

 def init(self):

 self.baseline_energy = None

 self.wavelet = 'db4'

 self.level = 3

 def process(self, current_waveform, voltage_waveform):

```

### 小波变换提取高频细节

```
coeffs = pywt.wavedec(current_waveform, self.wavelet, level=self.level)

details = coeffs[1:] # 细节系数

```

### 计算高频能量 (level 2细节)

```
hf_energy = np.sum(details[1]**2)

```



# 检测电压平肩部特征

```
voltage_shoulder = self.calculate_flatness(voltage_waveform)
```

## 判断电弧

```
if self.baseline_energy is None:

 self.baseline_energy = hf_energy

return False

arc_suspect = hf_energy > (self.baseline_energy * 3)

arc_confirmed = arc_suspect and voltage_shoulder > 0.8

return arc_confirmed

...
```

### 3. 编程说明

- 依赖算子**：wavelet\_transform (万象)、expression\_eval (内置)、threshold\_rule (内置)。
- 数据要求**：高频原始电流/电压波形（建议 $\geq 12.8\text{kHz}$ ）。
- 核心算法**：利用小波变换提取电流波形的高频细节能量；同时检测电压波形是否存在电弧导致的“平肩部”特征。
- 告警逻辑**：当高频能量突增（ $>3$ 倍基线）且电压平肩特征明显时，判定为电弧故障。

---

## S-04 - 电气接头松动/氧化预警

### 1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

model\_id: "S-04"

model\_name: "电气接头松动/氧化预警"

category: "safety"

tech\_level: "L2a"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input\_fields: ["Ia\_harmonics", "Ib\_harmonics", "Ic\_harmonics", "device\_id"]

window: {type: "sliding", size: 3600, step: 3600}

pipeline:

- {id: "get\_baseline", type: "qianzhi", operator: "query\_device\_param", params: {param\_name: "harmonic\_fingerprint"}, output\_as: "baseline\_fingerprint"}
- {id: "extract\_fingerprint", type: "builtin", operator: "expression\_eval", params: {expression: "[\${input.Ia\_harmonics}[2]/\${input.Ia\_harmonics}[0], \${input.Ia\_harmonics}[4]/\${input.Ia\_harmonics}[0], \${input.Ia\_harmonics}[6]/\${input.Ia\_harmonics}[0]]", output\_as: "current\_fingerprint"}}
- {id: "similarity", type: "wanxiang", operator: "dtw\_similarity", inputs: {sequence\_a: "\${get\_baseline.baseline\_fingerprint}", sequence\_b: "\${extract\_fingerprint.current\_fingerprint}"}, output\_as: "health\_index"}
- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: "<", duration: 7200}, inputs: {value: "\${similarity.health\_index}", threshold: 0.75}, output\_as: "alert\_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

```
fields: {health_index: "${similarity.health_index}"}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 30}

schedule: {interval: 3600}

fallback: {enabled: true, condition: "baseline_not_exist", operator:
"static_threshold", params: {learning_period: 168}}

...
```

## 2. 核心逻辑与伪代码

```
```python
```

S-04 核心逻辑伪代码

```
class LooseConnectionDetector:

def init(self):

self.baseline_fingerprint = None

def extract_fingerprint(self, harmonics):
```

提取3,5,7次谐波相对于基波的含有率作为指纹

```
return [harmonics[2]/harmonics[0], harmonics[4]/harmonics[0], harmonics[6]/
harmonics[0]]

def process_hourly(self, hourly_harmonics):

current_fp = self.extract_fingerprint(hourly_harmonics)

if self.baseline_fingerprint is None:

self.baseline_fingerprint = current_fp

return 1.0
```

计算与基线指纹的DTW相似度

```
similarity = dtw_distance(self.baseline_fingerprint, current_fp)

health_index = 1 / (1 + similarity) # 转换为0-1健康指数

if health_index < 0.75:

    self.trigger_alert(health_index)

return health_index

...
```

3. 编程说明

- **依赖算子**：dtw_similarity (万象)、expression_eval (内置)、threshold_rule (内置)。
- **数据窗口**：1小时滑动窗口，步长1小时。
- **核心算法**：提取电流的3、5、7次谐波含有率作为“谐波指纹”。在正常运行状态下学习基线指纹。通过计算当前指纹与基线指纹的动态时间规整（DTW）距离，量化其相似度，转换为健康指数。
- **告警逻辑**：当健康指数持续低于0.75超过2小时，触发接头松动预警。

S-05 - 中性线过载热保护

1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "S-05"

model_name: "中性线过载热保护"

category: "safety"

tech_level: "L1"
```

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input_fields: ["Ia_rms", "Ib_rms", "Ic_rms", "angle_a", "angle_b", "angle_c",
"device_id"]

window: {type: "sliding", size: 10, step: 1}

pipeline:

- {id: "calc_neutral", type: "wanxiang", operator: "symmetrical_components", inputs: {Ia: "\${input.Ia_rms}", Ib: "\${input.Ib_rms}", Ic: "\${input.Ic_rms}", angle_a: "\${input.angle_a}", angle_b: "\${input.angle_b}", angle_c: "\${input.angle_c}"}, output_as: "components"}
- {id: "get Rated", type: "qianzhi", operator: "get Rated_current", params: {device_id: "\${input.device_id}"}, output_as: "rated_current"}
- {id: "thermal_model", type: "builtin", operator: "expression_eval", params: {expression: "thermal_accumulate(\${calc_neutral.components.I0}, \${get Rated.rated_current}, \${context.thermal_state})", output_as: "thermal_state"}}
- {id: "evaluate", type: "builtin", operator: "threshold_rule", params: {operator: ">"}, inputs: {value: "\${thermal_model.thermal_state}", threshold: 0.9}, output_as: "alert_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {neutral_current: "\${calc_neutral.components.I0}", thermal_load: "\${thermal_model.thermal_state}"}

deployment: {replicas: 2, cpu: "500m", memory: "1Gi", timeout: 5}

schedule: {interval: 1}

...

2. 核心逻辑与伪代码

```python

### S-05 核心逻辑伪代码

```
import cmath
```

```
class NeutralOverloadProtector:
```

```
 def init(self, rated_current):
```

```
 self.rated_current = rated_current
```

```
 self.thermal_state = 0.0
```

```
 self.tau_heat = 600 # 加热时间常数
```

```
 self.tau_cool = 1200 # 冷却时间常数
```

```
 def calculate_neutral_current(self, Ia, Ib, Ic, ang_a, ang_b, ang_c):
```

### 相量表示

```
 ph_a = Ia * cmath.exp(1j * math.radians(ang_a))
```

```
 ph_b = Ib * cmath.exp(1j * math.radians(ang_b))
```

```
 ph_c = Ic * cmath.exp(1j * math.radians(ang_c))
```

### 中性线电流 = 三相电流矢量和

```
 I_neutral = abs(ph_a + ph_b + ph_c)
```

```
 return I_neutral
```

```
 def process(self, data):
```

```
 I0 = self.calculate_neutral_current(data['Ia_rms'], data['Ib_rms'], data['Ic_rms'],
```

```
data['angle_a'], data['angle_b'], data['angle_c'])
```

## 热累积模型 (一阶RC)

```
I_pu = I0 / self.rated_current
```

```
if I_pu > 1.0:
```

### 加热

```
self.thermal_state += (I_pu**2 - self.thermal_state) * (1 / self.tau_heat)
```

```
else:
```

### 冷却

```
self.thermal_state += (0 - self.thermal_state) * (1 / self.tau_cool)
```

```
if self.thermal_state > 0.9:
```

```
self.trigger_alert(I0, self.thermal_state)
```

```
...
```

### 3. 编程说明

- **依赖算子**：symmetrical\_components (万象)、expression\_eval (内置)、threshold\_rule (内置)。
- **数据窗口**：10秒滑动窗口，步长1秒。
- **核心算法**：通过对称分量法或直接相量求和计算中性线电流 $I_0$ 。采用一阶热累积模型模拟中性线的发热和冷却过程。
- **告警逻辑**：当热累积状态量超过0.9（即热积累达到额定值的90%）时触发告警，实现反时限保护。

---

(... 以下依此类推，为全部67个模型编写此三部分内容 ...)

```
...
```

---

好的，我将继续按照模板完成 **S-06 到 S-22** 这17个电气安全模型的完整开发文档，每个模型包含 **YAML 配置单**、**核心逻辑伪代码** 和 **编程说明**。

---

## S-06 - 三相不平衡热损评估

### 1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "S-06"

model_name: "三相不平衡热损评估"

category: "safety"

tech_level: "L1"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input_fields: ["Ia_rms", "Ib_rms", "Ic_rms", "angle_a", "angle_b", "angle_c",
"device_id"]

window: {type: "sliding", size: 60, step: 60}

pipeline:

 • {id: "sym_components", type: "wanxiang", operator:
 "symmetrical_components", inputs: {Ia: "${input.Ia_rms}", Ib: "$
 {input.Ib_rms}", Ic: "${input.Ic_rms}", angle_a: "${input.angle_a}", angle_b: "$
 {input.angle_b}", angle_c: "${input.angle_c}"}, output_as: "components"}

 • {id: "get_motor_param", type: "qianzhi", operator: "get_motor_param",
 params: {param: "R2"}, output_as: "R2"}
```



- {id: "calc\_thermal\_loss", type: "builtin", operator: "expression\_eval", params: {expression: "\${sym\_components.components.I2}^2 \* \${get\_motor\_param.R2} \* 3", output\_as: "extra\_loss"}}
- {id: "calc\_temp\_rise", type: "builtin", operator: "expression\_eval", params: {expression: "\${calc\_thermal\_loss.extra\_loss} \* 0.5", output\_as: "temp\_rise"}}

output:

type: "metric"

level: "info"

topic: "tianyan.metrics.v1"

fields: {negative\_seq\_current: "\${sym\_components.components.I2}", extra\_loss\_kw: "\${calc\_thermal\_loss.extra\_loss}", equivalent\_temp\_rise: "\${calc\_temp\_rise.temp\_rise}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 60}

...

## 2. 核心逻辑与伪代码

```python

S-06 核心逻辑伪代码

```
class UnbalanceThermalLossEstimator:
```

```
def init(self):
```

```
    self.R2 = self.get_motor_negative_seq_resistance() # 从千知获取电机负序电阻
```

```
def process(self, data):
```

对称分量法提取负序电流 I2

```
I1, I2, I0 = self.symmetrical_components(
```

```
data['Ia_rms'], data['Ib_rms'], data['Ic_rms'],  
data['angle_a'], data['angle_b'], data['angle_c']  
)
```

计算附加热损 ($I_2^2 * R_2 * 3$)

```
extra_loss = (I2 ** 2) * self.R2 * 3 # kW
```

等效温升估算 (简化为线性关系，系数可标定)

```
temp_rise = extra_loss * 0.5 # °C  
  
return {  
    'I2': I2,  
    'extra_loss_kw': extra_loss,  
    'temp_rise': temp_rise  
}  
'''
```

3. 编程说明

- **依赖算子**：symmetrical_components (万象)、get_motor_param (千知)、expression_eval (内置)。
 - **数据窗口**：60秒滑动窗口，步长60秒。
 - **核心算法**：利用对称分量法计算负序电流 I_2 。基于电机负序等值电阻 R_2 ，计算附加铜损 $P_{loss} = 3 * I_2^2 * R_2$ 。通过经验系数折算为等效温升。
 - **输出**：每分钟输出一次负序电流、附加热损和等效温升指标。
-

S-07 - 瞬时过电压累积冲击

1. 模型开发配置单 (YAML)

```yaml

model:

model\_id: "S-07"

model\_name: "瞬时过电压累积冲击"

category: "safety"

tech\_level: "L1"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input\_fields: ["Va\_peak", "Vb\_peak", "Vc\_peak", "device\_id"]

window: {type: "event"}

pipeline:

- {id: "get\_spd\_rating", type: "qianzhi", operator: "query\_device\_param", params: {param\_name: "spd\_energy\_rating"}, output\_as: "spd\_rating"}
- {id: "detect\_overvoltage", type: "builtin", operator: "threshold\_rule", params: {operator: ">"}, inputs: {value: "\${input.Va\_peak}", threshold: "\${context.nominal\_voltage} \* 1.1"}, output\_as: "event\_trigger"}
- {id: "calc\_energy", type: "builtin", operator: "expression\_eval", params: {expression: "estimate\_energy(\${input.Va\_peak}, \${context.spd\_curve})", output\_as: "event\_energy"}}
- {id: "accumulate", type: "builtin", operator: "expression\_eval", params: {expression: "\${context.accumulated\_energy} + \${calc\_energy.event\_energy}"}, output\_as: "accumulated\_energy"}}

- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: ">"}, inputs: {value: "\${accumulate.accumulated\_energy}", threshold: "\${get\_spd\_rating.spd\_rating} \* 0.8"}, output\_as: "alert\_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {accumulated\_energy\_j: "\${accumulate.accumulated\_energy}", spd\_rating\_j: "\${get\_spd\_rating.spd\_rating}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 5}

schedule: {interval: 1}

...

## 2. 核心逻辑与伪代码

```python

S-07 核心逻辑伪代码

```
class OvervoltageAccumulator:
```

```
def init(self, nominal_voltage, spd_rating):
```

```
    self.nominal_voltage = nominal_voltage
```

```
    self.spd_rating = spd_rating
```

```
    self.accumulated_energy = 0.0
```

```
def process(self, data):
```

```
    V_peak = max(data['Va_peak'], data['Vb_peak'], data['Vc_peak'])
```

```
    if V_peak > self.nominal_voltage * 1.1:
```

估算本次过电压能量 (简化模型)

```
energy = self.estimate_energy(V_peak)

self.accumulated_energy += energy

if self.accumulated_energy > self.spd_rating * 0.8:

    self.trigger_alert(self.accumulated_energy)

def estimate_energy(self, V_peak):
```

根据SPD的V-I特性曲线估算吸收能量

实际实现需查表或拟合非线性曲线

```
return 0.5 * 1e-6 * (V_peak ** 2) # 简化示例

'''
```

3. 编程说明

- **依赖算子**：query_device_param (千知)、threshold_rule (内置)、expression_eval (内置)。
- **数据窗口**：事件触发。
- **核心算法**：实时监测三相电压峰值，当超过标称电压1.1倍时记录一次过电压事件。根据SPD的非线性特性估算单次事件吸收的能量，并进行累积。
- **告警逻辑**：当累积能量达到SPD额定吸收能量的80%时，发出更换预警。

S-08 - 电机轴承/转子断条初兆

1. 模型开发配置单 (YAML)

```
```yaml
```

```
model:
```

model\_id: "S-08"

model\_name: "电机轴承/转子断条初兆"

category: "safety"

tech\_level: "L3"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.electrical.raw.v1"

input\_fields: ["Ia\_waveform", "device\_id", "motor\_speed"]

window: {type: "sliding", size: 10, step: 10}

pipeline:

- {id: "fft\_analysis", type: "wanxiang", operator: "fft\_spectrum", params: {fs: 1000}, inputs: ["\${input.Ia\_waveform}"], output\_as: "spectrum"}
- {id: "get\_slip", type: "builtin", operator: "expression\_eval", params: {expression: "(1500 - \${input.motor\_speed}) / 1500"}, output\_as: "slip"}
- {id: "sideband\_detection", type: "builtin", operator: "expression\_eval", params: {expression: "find\_sideband\_amplitude(\${fft\_analysis.spectrum}, 50, \${get\_slip.slip} \* 50)"}, output\_as: "sideband\_amplitude"}
- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: ">"}, inputs: {value: "\${sideband\_detection.sideband\_amplitude}", threshold: "\${context.baseline\_amplitude} \* 6"}, output\_as: "alert\_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {sideband\_db: "\${sideband\_detection.sideband\_amplitude}"}

deployment: {replicas: 1, cpu: "1000m", memory: "2Gi", timeout: 15}

```
schedule: {interval: 10}
```

```
fallback: {enabled: true, condition: "vfd_mode", operator: "disabled"}
```

```
...
```

## 2. 核心逻辑与伪代码

```
```python
```

S-08 核心逻辑伪代码

```
class MotorBrokenBarDetector:
```

```
def init(self):
```

```
    self.baseline_amplitude = None
```

```
def process(self, current_waveform, motor_speed):
```

FFT频谱分析

```
spectrum = np.abs(np.fft.fft(current_waveform))
```

```
freq = np.fft.fftfreq(len(current_waveform), d=1/1000)
```

计算转差率

```
sync_speed = 1500 # 4极电机同步转速
```

```
slip = (sync_speed - motor_speed) / sync_speed
```

边频带频率 $(1 \pm 2s)f$

```
sideband_freq = 50 * (1 + 2 * slip)
```

提取边频带幅值

```
idx = np.argmin(np.abs(freq - sideband_freq))

sideband_amp = spectrum[idx]

if self.baseline_amplitude is None:

    self.baseline_amplitude = sideband_amp

return
```

边频带幅值超过基线6倍告警

```
if sideband_amp > self.baseline_amplitude * 6:

    self.trigger_alert(sideband_amp)

...
```

3. 编程说明

- **依赖算子**：fft_spectrum (万象)、expression_eval (内置)、threshold_rule (内置)。
- **数据要求**：电流原始波形（建议采样率 $\geq 1\text{kHz}$ ），电机转速信号。
- **核心算法**：基于电机电流特征分析（MCSA）。对电流进行FFT，定位基频(50Hz)两侧的边频带 $(1\pm 2s)f$ 。边频带幅值与转子断条严重程度成正比。
- **告警逻辑**：当边频带幅值超过正常状态基线6倍时告警。变频工况下降级或禁用。

S-09 - 变压器直流偏磁监测

1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "S-09"
```


model_name: "变压器直流偏磁监测"

category: "safety"

tech_level: "L1"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input_fields: ["I0_dc", "device_id"]

window: {type: "sliding", size: 60, step: 60}

pipeline:

- {id: "get_transformer_limit", type: "qianzhi", operator: "get_transformer_param", params: {param: "dc_tolerance"}, output_as: "dc_limit"}
- {id: "smooth_dc", type: "wanxiang", operator: "lowess_filter", params: {frac: 0.05}, inputs: ["\${input.I0_dc}"], output_as: "smoothed_dc"}
- {id: "evaluate", type: "builtin", operator: "threshold_rule", params: {operator: ">", duration: 300}, inputs: {value: "\${smooth_dc.smoothed_dc}", threshold: "\${get_transformer_limit.dc_limit}"}, output_as: "alert_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {dc_current_a: "\${smooth_dc.smoothed_dc}", limit_a: "\${get_transformer_limit.dc_limit}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 60}

...

2. 核心逻辑与伪代码

```python

### S-09 核心逻辑伪代码

```
class TransformerDCBiasMonitor:

def init(self):

self.dc_limit = self.get_dc_tolerance() # 从千知获取

self.history = deque(maxlen=60)

def process(self, data):

I0_dc = data['I0_dc']

self.history.append(I0_dc)

if len(self.history) == 60:
```

### LOWESS滤波平滑

```
smoothed = lowess(list(self.history), frac=0.05)

dc_value = smoothed[-1]

if dc_value > self.dc_limit:

self.trigger_alert(dc_value)

...
```

## 3. 编程说明

- **依赖算子**：get\_transformer\_param(千知)、lowess\_filter(万象)、threshold\_rule(内置)。
- **数据窗口**：60秒滑动窗口，步长60秒。
- **核心算法**：直接测量或从零序电流中提取直流分量，采用LOWESS滤波平滑。

- **告警逻辑**：当平滑后的直流分量超过变压器直流耐受阈值（如额定电流的0.5%）并持续5分钟时告警。
- 

## S-10 - 电缆绝缘热老化评估

### 1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "S-10"

model_name: "电缆绝缘热老化评估"

category: "safety"

tech_level: "L2b"

version: "1.0.0"

data_source:

type: "jdbc"

topic: "taiyi.electrical.history"

input_fields: ["IO_residual", "IO_angle", "Va_angle", "device_id", "temperature"]

window: {type: "sliding", size: 86400, step: 86400}

history_required: "12months"

pipeline:

 • {id: "calc_resistive_daily", type: "wanxiang", operator: "phasor_projection",
 inputs: {IO: "${input.IO_residual}", angle_I: "${input.IO_angle}", angle_V: "${input.Va_angle}"}, output_as: "I_resistive_daily"}

 • {id: "arrhenius_fit", type: "builtin", operator: "expression_eval", params:
 {expression: "fit_arrhenius(${calc_resistive_daily.I_resistive_daily}, $
 {input.temperature})"}, output_as: "aging_model"}
```

- {id: "predict\_life", type: "builtin", operator: "expression\_eval", params: {expression: "extrapolate\_life(\${arrhenius\_fit.aging\_model}, \${context.critical\_threshold})", output\_as: "remaining\_life"}}
- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: "<"}, inputs: {value: "\${predict\_life.remaining\_life}", threshold: 180}, output\_as: "alert\_trigger"}}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {remaining\_life\_days: "\${predict\_life.remaining\_life}"}

deployment: {replicas: 1, cpu: "500m", memory: "2Gi", timeout: 60}

schedule: {interval: 86400}

fallback: {enabled: true, condition: "history\_days < 365", operator: "linear\_trend"}

...

## 2. 核心逻辑与伪代码

```python

S-10 核心逻辑伪代码

import numpy as np

from scipy.optimize import curve_fit

class CableAgingPredictor:

def **init**(self, critical_threshold):

self.critical_threshold = critical_threshold

self.history = [] # (time, I_resistive, temp)

def arrhenius_model(self, t, A, Ea):

简化的阿伦尼乌斯老化模型

```
return A * np.exp(Ea / (8.314 * (273 + self.avg_temp)))
```

```
def process_daily(self, daily_data):
```

计算日平均阻性电流

```
I_res = self.calc_resistive(daily_data)
```

```
temp = daily_data['temperature']
```

```
self.history.append((time.time(), I_res, temp))
```

```
if len(self.history) > 365:
```

```
times, currents, temps = zip(*self.history)
```

```
self.avg_temp = np.mean(temps)
```

拟合老化曲线

```
popt, _ = curve_fit(self.arrhenius_model, times, currents)
```

外推剩余寿命

```
remaining = self.extrapolate_life(popt, self.critical_threshold)
```

```
if remaining < 180:
```

```
self.trigger_alert(remaining)
```

```
...
```

3. 编程说明

- **依赖算子**：phasor_projection (万象)、expression_eval (内置)、threshold_rule (内置)。
- **数据要求**：至少12个月的历史数据，含阻性漏电流和运行温度。

- **核心算法**：基于阿伦尼乌斯方程，拟合阻性漏电流随时间的增长曲线。阻性漏电流的缓慢爬升反映了绝缘热老化程度。
 - **告警逻辑**：当预测剩余寿命低于180天时发出更换预警。
-

S-11 - 逆功率/孤岛检测

1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "S-11"

model_name: "逆功率/孤岛检测"

category: "safety"

tech_level: "L1"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input_fields: ["P_total", "frequency", "Va_rms", "device_id"]

window: {type: "sliding", size: 2, step: 0.1}

pipeline:

  • {id: "reverse_power", type: "builtin", operator: "threshold_rule", params:
    {operator: "<"}, inputs: {value: "${input.P_total}", threshold: 0}, output_as:
    "reverse_power_flag"}

  • {id: "freq_shift", type: "builtin", operator: "expression_eval", params:
    {expression: "abs(${input.frequency} - 50) > 0.5"}, output_as: "freq_shift_flag"}

  • {id: "voltage_shift", type: "builtin", operator: "expression_eval", params:
    {expression: "${input.Va_rms} < 0.88 * ${context.nominal_voltage} || $
```

```
{input.Va_rms} > 1.1 * ${context.nominal_voltage}"}}, output_as:
"voltage_shift_flag"}
```

- {id: "vector_shift", type: "wanxiang", operator: "phasor_projection", params: {mode: "angle_diff"}, inputs: {angle_current: "\${input.angle_I}", angle_voltage: "\${input.angle_V}"}, output_as: "angle_diff"}
- {id: "evaluate", type: "builtin", operator: "expression_eval", params: {expression: "(\${reverse_power_flag} && (\${freq_shift_flag} || \${voltage_shift_flag})) || \${vector_shift.angle_diff} > 5"}, output_as: "islanding_detected"}

output:

type: "alert"

level: "critical"

topic: "tianya.alerts.v1"

fields: {islanding: "\${evaluate.islanding_detected}"}

deployment: {replicas: 2, cpu: "500m", memory: "1Gi", timeout: 1}

schedule: {interval: 1}

...

2. 核心逻辑与伪代码

```
```python
```

# S-11 核心逻辑伪代码

```
class IslandingDetector:
```

```
def init(self, nominal_voltage):
```

```
 self.nominal_voltage = nominal_voltage
```

```
 self.prev_angle = None
```

```
def process(self, data):
```

```

P = data['P_total']

freq = data['frequency']

V = data['Va_rms']

angle_V = data['angle_V']

angle_I = data['angle_I']

reverse_power = P < 0

freq_shift = abs(freq - 50) > 0.5

voltage_shift = V < 0.88 * self.nominal_voltage or V > 1.1 * self.nominal_voltage

vector_shift = False

if self.prev_angle is not None:

 angle_diff = abs((angle_I - angle_V) - self.prev_angle)

 vector_shift = angle_diff > 5 # 5度

 self.prev_angle = angle_I - angle_V

islanding = reverse_power and (freq_shift or voltage_shift) or vector_shift

return islanding

'''

```

### 3. 编程说明

- **依赖算子**：threshold\_rule (内置)、expression\_eval (内置)、phasor\_projection (万象，相角差模式)。
  - **数据窗口**：2秒滑动窗口，步长0.1秒。
  - **核心算法**：综合逆功率、频率越限、电压越限和相量偏移法（Vector Shift）多种判据。相量偏移法通过监测电压电流相角差的突变来检测孤岛。
  - **告警逻辑**：任一判据组合满足即判定为孤岛事件，发出紧急告警。
-



## S-12 - 直流分量注入监测

### 1. 模型开发配置单 (YAML)

```yaml

model:

model_id: "S-12"

model_name: "直流分量注入监测"

category: "safety"

tech_level: "L1"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input_fields: ["Ia_dc", "Ib_dc", "Ic_dc", "device_id"]

window: {type: "sliding", size: 60, step: 60}

pipeline:

- {id: "get_rated", type: "qianzhi", operator: "get_rated_current", output_as: "In"}
- {id: "calc_max_dc_ratio", type: "builtin", operator: "expression_eval", params: {expression: "max(\${input.Ia_dc}, \${input.Ib_dc}, \${input.Ic_dc}) / \${get_rated.In} * 100", output_as: "dc_ratio_percent"}}
- {id: "evaluate", type: "builtin", operator: "threshold_rule", params: {operator: ">", duration: 300, inputs: {value: "\${calc_max_dc_ratio.dc_ratio_percent}", threshold: 0.5}, output_as: "alert_trigger"}}

output:

type: "alert"

level: "warning"

```
topic: "tianya.alerts.v1"
```

```
fields: {dc_ratio_percent: "${calc_max_dc_ratio.dc_ratio_percent}"}
```

```
deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}
```

```
schedule: {interval: 60}
```

```
...
```

2. 核心逻辑与伪代码

```
```python
```

# S-12 核心逻辑伪代码

```
class DCInjectionMonitor:
```

```
def init(self, rated_current):
```

```
 self.rated_current = rated_current
```

```
 self.dc_history = deque(maxlen=5) # 5分钟
```

```
 def process_minutely(self, data):
```

```
 Ia_dc = data['Ia_dc']
```

```
 Ib_dc = data['Ib_dc']
```

```
 Ic_dc = data['Ic_dc']
```

```
 max_dc = max(Ia_dc, Ib_dc, Ic_dc)
```

```
 dc_ratio = (max_dc / self.rated_current) * 100
```

```
 self.dc_history.append(dc_ratio)
```

```
 if len(self.dc_history) == 5 and all(r > 0.5 for r in self.dc_history):
```

```
 self.trigger_alert(dc_ratio)
```

```
 ...
```

### 3. 编程说明

- **依赖算子**：get Rated Current (千知)、expression Eval (内置)、threshold Rule (内置)。
  - **数据窗口**：60秒滑动窗口，步长60秒。
  - **核心算法**：从电流测量值中提取直流分量（通过低通滤波或硬件测量），计算其占额定电流的百分比。
  - **告警逻辑**：当最大相直流分量占比持续超过0.5%（国标通常要求）超过5分钟时告警。
- 

## S-13 - 设备过载动态预测

### 1. 模型开发配置单 (YAML)

```
``yaml
model:
 model_id: "S-13"
 model_name: "设备过载动态预测"
 category: "safety"
 tech_level: "L2b"
 version: "1.0.0"
 data_source:
 type: "kafka"
 topic: "taiyi.electrical.feature.v1"
 input_fields: ["Ia_rms", "Ib_rms", "Ic_rms", "device_id"]
 window: {type: "sliding", size: 600, step: 60}
 history_required: "3months"
```

pipeline:

- {id: "get\_rated", type: "qianzhi", operator: "get\_rated\_current", output\_as: "In"}
- {id: "lstm\_forecast", type: "wanxiang", operator: "forecast\_lstm", params: {horizon: 10, model\_path: "/models/S-13\_lstm.h5"}, inputs: ["\${input.Ia\_rms}"], output\_as: "forecast"}
- {id: "calc\_risk", type: "builtin", operator: "expression\_eval", params: {expression: "max(0, (\${lstm\_forecast.forecast} - \${get\_rated.In}) / \${get\_rated.In})", output\_as: "overload\_risk"}}
- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: ">"}, inputs: {value: "\${calc\_risk.overload\_risk}", threshold: 0.8}, output\_as: "alert\_trigger"}}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {overload\_risk: "\${calc\_risk.overload\_risk}", forecast\_current: "\${lstm\_forecast.forecast}"}

deployment: {replicas: 1, cpu: "1000m", memory: "2Gi", timeout: 15}

schedule: {interval: 60}

fallback: {enabled: true, condition: "history\_days < 90", operator: "static\_threshold", params: {threshold: 1.0}}

...

## 2. 核心逻辑与伪代码

```python

S-13 核心逻辑伪代码

class OverloadPredictor:

```
def init(self, rated_current, model_path):

self.rated_current = rated_current

self.model = load_lstm_model(model_path)

self.history = deque(maxlen=600) # 10分钟历史

def process(self, data):

Ia = data['Ia_rms']

self.history.append(Ia)

if len(self.history) == 600:
```

利用LSTM预测未来10分钟电流

```
seq = np.array(self.history).reshape(1, -1, 1)

forecast = self.model.predict(seq)[0, 0]

overload_risk = max(0, (forecast - self.rated_current) / self.rated_current)

if overload_risk > 0.8:

self.trigger_alert(overload_risk, forecast)

...
```

3. 编程说明

- **依赖算子**：forecast_lstm (万象)、get_rated_current (千知)、expression_eval (内置)、threshold_rule (内置)。
 - **数据要求**：至少3个月历史电流数据用于训练LSTM模型。
 - **核心算法**：使用LSTM时序模型，基于过去10分钟的电流序列，预测未来10分钟的最大电流值。
 - **告警逻辑**：当预测电流超过额定电流的80%时（即过载风险>0.8），发出预警。
-

S-14 - 电缆接头红外温度关联预警

1. 模型开发配置单 (YAML)

```yaml

model:

model\_id: "S-14"

model\_name: "电缆接头红外温度关联预警"

category: "safety"

tech\_level: "L2a"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.thermal.v1"

input\_fields: ["joint\_temperature", "ambient\_temperature", "load\_current",  
"device\_id"]

window: {type: "sliding", size: 3600, step: 3600}

pipeline:

- {id: "thermal\_model", type: "builtin", operator: "expression\_eval", params: {expression: "\${context.model\_params.a} \* \${input.load\_current}^2 + \${context.model\_params.b} \* \${input.ambient\_temperature} + \${context.model\_params.c}"}, output\_as: "predicted\_temp"}
- {id: "calc\_residual", type: "builtin", operator: "expression\_eval", params: {expression: "\${input.joint\_temperature} - \${thermal\_model.predicted\_temp}"}, output\_as: "residual"}
- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: ">", duration: 7200}, inputs: {value: "\${calc\_residual.residual}", threshold: 10}, output\_as: "alert\_trigger"}

output:

```
type: "alert"

level: "warning"

topic: "tianya.alerts.v1"

fields: {actual_temp: "${input.joint_temperature}", predicted_temp: "${thermal_model.predicted_temp}", residual: "${calc_residual.residual}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 3600}

...
```

## 2. 核心逻辑与伪代码

```
```python
```

S-14 核心逻辑伪代码

```
class InfraredThermalMonitor:

    def init(self, model_params):

        self.a, self.b, self.c = model_params

    def predict_temp(self, current, ambient):

        return self.a * (current ** 2) + self.b * ambient + self.c

    def process_hourly(self, data):

        current = data['load_current']

        ambient = data['ambient_temperature']

        actual_temp = data['joint_temperature']

        predicted_temp = self.predict_temp(current, ambient)

        residual = actual_temp - predicted_temp

        if residual > 10: # 残差超过10°C
```

```
self.trigger_alert(actual_temp, predicted_temp, residual)
...

```

3. 编程说明

- **依赖算子**：expression_eval (内置)、threshold_rule (内置)。
- **数据窗口**：1小时滑动窗口，步长1小时。
- **核心算法**：基于焦耳定律和热阻模型，建立负载电流、环境温度与接头温度的多元线性回归模型（参数a,b,c可在线学习）。
- **告警逻辑**：当实测温度持续高于模型预测温度10℃以上并超过2小时，表明接触电阻异常增大，触发预警。

S-15 - 局部放电模式识别

1. 模型开发配置单 (YAML)

```
``yaml
model:

model_id: "S-15"

model_name: "局部放电模式识别"

category: "safety"

tech_level: "L3"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.pd.v1"

input_fields: ["pd_waveform", "phase_angle", "device_id"]

window: {type: "sliding", size: 60, step: 60}

```


pipeline:

- {id: "prpd_generation", type: "builtin", operator: "expression_eval", params: {expression: "generate_prpd(\${input.pd_waveform}, \${input.phase_angle})", output_as: "prpd_image"}}
- {id: "cnn_classify", type: "wanxiang", operator: "classification_random_forest", params: {model_id: "S15_cnn_v1"}, inputs: ["\${prpd_generation.prpd_image}"], output_as: "classification"}}
- {id: "calc_qiec", type: "builtin", operator: "expression_eval", params: {expression: "integrate_charge(\${input.pd_waveform})", output_as: "apparent_charge"}}
- {id: "evaluate", type: "builtin", operator: "threshold_rule", params: {operator: ">"}, inputs: {value: "\${calc_qiec.apparent_charge}", threshold: 100}, output_as: "alert_trigger"}}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {discharge_type: "\${cnn_classify.class}", apparent_charge_pc: "\${calc_qiec.apparent_charge}"}

deployment: {replicas: 1, cpu: "2000m", memory: "4Gi", timeout: 20}

schedule: {interval: 60}

...

2. 核心逻辑与伪代码

```python

# S-15 核心逻辑伪代码

class PartialDischargeClassifier:

def **init**(self, cnn\_model):

```
self.cnn_model = cnn_model
```

```
def generate_prpd(self, waveforms, phases):
```

## 生成PRPD谱图矩阵

```
prpd = np.zeros((256, 256))
```

```
for w, p in zip(waveforms, phases):
```

```
q = np.trapz(w) # 放电量
```

```
phase_bin = int(p / 360 * 256)
```

```
amp_bin = min(255, int(q / 10))
```

```
prpd[phase_bin, amp_bin] += 1
```

```
return prpd
```

```
def process(self, data):
```

```
prpd_image = self.generate_prpd(data['pd_waveform'], data['phase_angle'])
```

```
discharge_type = self.cnn_model.predict(prpd_image)
```

```
apparent_charge = np.sum([np.trapz(w) for w in data['pd_waveform']])
```

```
if apparent_charge > 100: # 100pC
```

```
self.trigger_alert(discharge_type, apparent_charge)
```

```
...
```

### 3. 编程说明

- **依赖算子**：classification\_random\_forest (万象，实际为CNN)、expression\_eval (内置)、threshold\_rule (内置)。
- **数据要求**：高频局放传感器 (>1MHz) 采集的脉冲波形和相位。
- **核心算法**：将局放脉冲按相位和幅值统计生成PRPD谱图，利用卷积神经网络 (CNN) 对谱图进行分类，识别放电类型 (内部气隙、沿面、电晕、悬浮)。
- **告警逻辑**：当视在放电量超过100pC时发出告警，并附带放电类型。

---

## S-16 - 保护定值在线校核

### 1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "S-16"
```

```
model_name: "保护定值在线校核"
```

```
category: "safety"
```

```
tech_level: "L2a"
```

```
version: "1.0.0"
```

```
data_source:
```

```
type: "jdbc"
```

```
topic: "taiyi.electrical.config"
```

```
input_fields: ["system_impedance", "protection_settings", "topology", "device_id"]
```

```
window: {type: "event"}
```

```
pipeline:
```

- {id: "calc\_short\_circuit", type: "builtin", operator: "expression\_eval", params: {expression: "short\_circuit\_calc(\${input.system\_impedance}, \${input.topology})", output\_as: "Ik\_max\_min"}}
- {id: "check\_sensitivity", type: "builtin", operator: "expression\_eval", params: {expression: "\${calc\_short\_circuit.Ik\_min} / \${input.protection\_settings.pickup} > 1.3", output\_as: "sensitivity\_ok"}}
- {id: "check\_selectivity", type: "builtin", operator: "expression\_eval", params: {expression: "\${input.protection\_settings.time\_dial} - \${context.upstream\_settings.time\_dial} > 0.2", output\_as: "selectivity\_ok"}}

- {id: "evaluate", type: "builtin", operator: "expression\_eval", params: {expression: "!(\${check\_sensitivity.sensitivity\_ok} && \${check\_selectivity.selectivity\_ok})", output\_as: "alert\_trigger"}}

output:

type: "alert"

level: "warning"

topic: "tianya.alerts.v1"

fields: {sensitivity\_ok: "\${check\_sensitivity.sensitivity\_ok}", selectivity\_ok: "\${check\_selectivity.selectivity\_ok}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 30}

schedule: {interval: 86400}

...

## 2. 核心逻辑与伪代码

```python

S-16 核心逻辑伪代码

```
class ProtectionSettingChecker:
```

```
def init(self, system_impedance, topology, upstream_settings):
```

```
    self.system_impedance = system_impedance
```

```
    self.topology = topology
```

```
    self.upstream_settings = upstream_settings
```

```
def calc_short_circuit_current(self, fault_location):
```

简化的短路计算

```
Z_total = self.system_impedance + self.topology.impedance_to(fault_location)
```

```
Ik_max = 1.1 * self.nominal_voltage / abs(Z_total)

Ik_min = 0.9 * self.nominal_voltage / abs(Z_total * 1.5) # 考虑弧阻

return Ik_max, Ik_min

def check(self, protection_settings):

Ik_max, Ik_min = self.calc_short_circuit(protection_settings.location)
```

灵敏度校核

```
sensitivity = Ik_min / protection_settings.pickup

sensitivity_ok = sensitivity > 1.3
```

选择性校核

```
selectivity_ok = protection_settings.time_dial - self.upstream_settings.time_dial >
0.2

if not (sensitivity_ok and selectivity_ok):

self.trigger_alert(sensitivity_ok, selectivity_ok)

...
```

3. 编程说明

- **依赖算子**：expression_eval (内置)。
 - **数据来源**：千知库/配置库中的系统阻抗、拓扑和保护定值。
 - **核心算法**：根据当前系统阻抗和拓扑进行短路电流计算。校核保护定值的灵敏度（最小短路电流/整定值 > 1.3）和选择性（上下级时间级差 > 0.2s）。
 - **告警逻辑**：当灵敏度或选择性不满足要求时，发出定值不合理告警。
-

S-17 - 断路器机械特性监测

1. 模型开发配置单 (YAML)

```yaml

model:

model\_id: "S-17"

model\_name: "断路器机械特性监测"

category: "safety"

tech\_level: "L2a"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.breaker.v1"

input\_fields: ["trip\_coil\_waveform", "device\_id"]

window: {type: "event"}

pipeline:

- {id: "extract\_features", type: "builtin", operator: "expression\_eval", params: {expression: "{t1: time\_to\_peak(\${input.trip\_coil\_waveform}), t2: time\_to\_dropout(\${input.trip\_coil\_waveform}), I\_peak: max(\${input.trip\_coil\_waveform})}", output\_as: "features"}}
- {id: "get\_baseline", type: "qianzhi", operator: "query\_device\_param", params: {param\_name: "breaker\_baseline"}, output\_as: "baseline"}}
- {id: "calc\_similarity", type: "wanxiang", operator: "dtw\_similarity", inputs: {sequence\_a: "\${extract\_features.features}", sequence\_b: "\${get\_baseline.baseline}"}, output\_as: "health\_index"}}
- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: "<"}, inputs: {value: "\${calc\_similarity.health\_index}", threshold: 0.85}, output\_as: "alert\_trigger"}}

```
output:

type: "alert"

level: "warning"

topic: "tianya.alerts.v1"

fields: {health_index: "${calc_similarity.health_index}"}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 10}

schedule: {interval: 1}

...
```

## 2. 核心逻辑与伪代码

```
```python
```

S-17 核心逻辑伪代码

```
class BreakerMechanicalMonitor:

def init(self, baseline_waveform):

self.baseline_features = self.extract_features(baseline_waveform)

def extract_features(self, waveform):

peak_idx = np.argmax(waveform)

dropout_idx = np.where(waveform[peak_idx:] < 0.1 * waveform[peak_idx])[0][0] +
peak_idx

return {

't1': peak_idx, # 到达峰值时间

't2': dropout_idx - peak_idx, # 跌落时间

'I_peak': waveform[peak_idx] # 峰值电流

}
```

```
def process_event(self, trip_waveform):  
  
    current_features = self.extract_features(trip_waveform)  
  
    health_index = dtw_similarity(self.baseline_features, current_features)  
  
    if health_index < 0.85:  
  
        self.trigger_alert(health_index)  
  
    ...
```

3. 编程说明

- **依赖算子**：expression_eval (内置)、dtw_similarity (万象)、threshold_rule (内置)。
- **数据要求**：断路器分合闸线圈电流波形（通常在100ms内）。
- **核心算法**：提取线圈电流波形的关键特征（动作时间、跌落时间、峰值电流）。利用DTW算法计算当前波形与健康基线波形的相似度。
- **告警逻辑**：当健康指数（相似度）低于0.85时，表明机械特性发生变化，发出维护预警。

S-18 - 涌流与内部故障区分

1. 模型开发配置单 (YAML)

```
``yaml  
  
model:  
  
    model_id: "S-18"  
  
    model_name: "涌流与内部故障区分"  
  
    category: "safety"  
  
    tech_level: "L2a"  
  
    version: "1.0.0"  
  
    data_source:
```


type: "kafka"

topic: "taiyi.electrical.raw.v1"

input_fields: ["Ia_waveform", "device_id"]

window: {type: "event"}

pipeline:

- {id: "fft_analysis", type: "wanxiang", operator: "fft_spectrum", params: {fs: 1000}, inputs: ["\${input.Ia_waveform}"], output_as: "spectrum"}
- {id: "harmonic2_ratio", type: "builtin", operator: "expression_eval", params: {expression: "\${fft_analysis.spectrum}[100] / \${fft_analysis.spectrum}[50]"}, output_as: "h2_ratio"}
- {id: "wave_symmetry", type: "builtin", operator: "expression_eval", params: {expression: "calculate_symmetry(\${input.Ia_waveform})"}, output_as: "symmetry"}
- {id: "evaluate", type: "builtin", operator: "expression_eval", params: {expression: "\${harmonic2_ratio.h2_ratio} > 0.15 && \${wave_symmetry.symmetry} < 0.7"}, output_as: "is_inrush"}

output:

type: "alert"

level: "info"

topic: "tianyan.alerts.v1"

fields: {is_inrush: "\${evaluate.is_inrush}", h2_ratio: "\${harmonic2_ratio.h2_ratio}"}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 5}

schedule: {interval: 1}

...

2. 核心逻辑与伪代码

```python

# S-18 核心逻辑伪代码

```
class InrushCurrentDiscriminator:
```

```
def process(self, current_waveform):
```

## FFT频谱

```
spectrum = np.abs(np.fft.fft(current_waveform))
```

```
h2_ratio = spectrum[100] / spectrum[50] # 二次谐波占比
```

## 波形对称度

```
half = len(current_waveform) // 2
```

```
symmetry = np.corrcoef(current_waveform[:half], -current_waveform[half:])[0, 1]
```

```
is_inrush = (h2_ratio > 0.15) and (symmetry < 0.7)
```

```
return is_inrush
```

```
...
```

### 3. 编程说明

- **依赖算子**：fft\_spectrum (万象)、expression\_eval (内置)。
- **数据窗口**：事件触发（过流事件时）。
- **核心算法**：利用涌流的两个核心特征——二次谐波含量高（通常>15%）和波形存在间断角导致对称性差。通过FFT计算二次谐波占比，通过相关性计算波形对称度。
- **输出**：判断为涌流时输出info级别事件，可闭锁保护跳闸。

---

## S-19 - 直流系统绝缘监测

### 1. 模型开发配置单 (YAML)

```
```yaml
```

model:

model_id: "S-19"

model_name: "直流系统绝缘监测"

category: "safety"

tech_level: "L1"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.dc.v1"

input_fields: ["Vp_to_ground", "Vn_to_ground", "branch_leakage", "device_id"]

window: {type: "sliding", size: 10, step: 10}

pipeline:

- {id: "calc_insulation", type: "builtin", operator: "expression_eval", params: {expression: "estimate_R(\${input.Vp_to_ground}, \${input.Vn_to_ground})", output_as: "R_insulation"}}
- {id: "evaluate", type: "builtin", operator: "threshold_rule", params: {operator: "<", duration: 60}, inputs: {value: "\${calc_insulation.R_insulation}", threshold: 25000}, output_as: "alert_trigger"}}
- {id: "locate_branch", type: "builtin", operator: "expression_eval", params: {expression: "argmax(\${input.branch_leakage})", output_as: "faulty_branch"}}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {R_insulation_kohm: "\${calc_insulation.R_insulation}", faulty_branch: "\${locate_branch.faulty_branch}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

```
schedule: {interval: 10}
```

```
...
```

2. 核心逻辑与伪代码

```
```python
```

# S-19 核心逻辑伪代码

```
class DCInsulationMonitor:
```

```
def init(self):
```

```
self.R_threshold = 25000 # 25kΩ
```

```
def estimate_insulation(self, Vp, Vn, V_dc):
```

## 电桥平衡法估算绝缘电阻

```
if Vp > Vn:
```

```
R_ins = (V_dc - Vp) / (Vp / 100000) # 假设平衡电阻100k
```

```
else:
```

```
R_ins = (V_dc - Vn) / (Vn / 100000)
```

```
return R_ins
```

```
def process(self, data):
```

```
R = self.estimate_insulation(data['Vp_to_ground'], data['Vn_to_ground'],
data['V_dc'])
```

```
if R < self.R_threshold:
```

```
faulty_branch = np.argmax(data['branch_leakage'])
```

```
self.trigger_alert(R, faulty_branch)
```

```
...
```

### 3. 编程说明

- **依赖算子**：expression\_eval (内置)、threshold\_rule (内置)。
  - **数据要求**：直流母线正负极对地电压，各支路漏电流。
  - **核心算法**：基于电桥平衡原理实时计算直流母线对地绝缘电阻。当绝缘下降时，通过比较各支路漏电流大小定位故障支路。
  - **告警逻辑**：绝缘电阻低于25kΩ并持续60秒时告警，并指出故障支路。
- 

## S-20 - 雷击过电压波形捕获

### 1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "S-20"
```

```
model_name: "雷击过电压波形捕获"
```

```
category: "safety"
```

```
tech_level: "L1"
```

```
version: "1.0.0"
```

```
data_source:
```

```
type: "kafka"
```

```
topic: "taiyi.transient.v1"
```

```
input_fields: ["Va_waveform", "device_id"]
```

```
window: {type: "event"}
```

```
pipeline:
```

- {id: "trigger\_detection", type: "builtin", operator: "threshold\_rule", params: {operator: ">"}, inputs: {value: "diff(\${input.Va\_waveform})", threshold: 1000}, output\_as: "triggered"}

- {id: "extract\_params", type: "builtin", operator: "expression\_eval", params: {expression: "extract\_lightning\_params(\${input.Va\_waveform})"}, output\_as: "params"}
- {id: "classify\_wave", type: "builtin", operator: "expression\_eval", params: {expression: "\${extract\_params.params.T1} < 2 ? 'lightning' : 'switching'"}, output\_as: "wave\_type"}
- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: ">"}, inputs: {value: "\${extract\_params.params.Vpeak}", threshold: "\${context.bil\_level}"}, output\_as: "alert\_trigger"}

output:

type: "alert"

level: "critical"

topic: "tianyan.alerts.v1"

fields: {wave\_type: "\${classify\_wave.wave\_type}", Vpeak\_kV: "\${extract\_params.params.Vpeak}", T1\_us: "\${extract\_params.params.T1}", T2\_us: "\${extract\_params.params.T2}"}

deployment: {replicas: 1, cpu: "500m", memory: "2Gi", timeout: 5}

schedule: {interval: 1}

...

## 2. 核心逻辑与伪代码

```python

S-20 核心逻辑伪代码

```
class LightningSurgeCapture:
```

```
    def init(self, bil_level):
```

```
        self.bil_level = bil_level
```

```
    def extract_params(self, waveform, fs):
```

检测峰值

```
V_peak = np.max(waveform)
```

```
peak_idx = np.argmax(waveform)
```

计算波头时间T1 (10%到90%峰值)

```
t10 = np.where(waveform[:peak_idx] > 0.1 * V_peak)[0][0]
```

```
t90 = np.where(waveform[:peak_idx] > 0.9 * V_peak)[0][0]
```

```
T1 = (t90 - t10) / fs * 1e6 # us
```

计算波尾时间T2 (峰值到50%峰值)

```
t50_after = np.where(waveform[peak_idx:] < 0.5 * V_peak)[0][0] + peak_idx
```

```
T2 = (t50_after - peak_idx) / fs * 1e6 # us
```

```
return V_peak, T1, T2
```

```
def process(self, waveform, fs):
```

```
V_peak, T1, T2 = self.extract_params(waveform, fs)
```

```
wave_type = 'lightning' if T1 < 2 else 'switching'
```

```
if V_peak > self.bil_level:
```

```
self.trigger_alert(wave_type, V_peak, T1, T2)
```

```
...
```

3. 编程说明

- **依赖算子**：threshold_rule (内置)、expression_eval (内置)。
- **数据要求**：高频录波 (>10kHz) 的暂态电压波形。

- **核心算法**：通过电压变化率（dV/dt）触发录波。自动提取波头时间T1（10%-90%峰值）、波尾时间T2（峰值-50%峰值）和峰值电压。根据T1判断是雷电波（ $<2\mu s$ ）还是操作波。
 - **告警逻辑**：当峰值电压超过设备BIL（基本冲击绝缘水平）时告警。
-

S-21 - 剩余电流保护器(RCD)状态自检

1. 模型开发配置单 (YAML)

```yaml

model:

model\_id: "S-21"

model\_name: "剩余电流保护器(RCD)状态自检"

category: "safety"

tech\_level: "L1"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.rcd.status.v1"

input\_fields: ["rcd\_status", "test\_button\_pressed", "trip\_coil\_current",  
"last\_test\_time", "device\_id"]

window: {type: "sliding", size: 86400, step: 3600}

pipeline:

- {id: "get\_rcd\_params", type: "qianzhi", operator: "query\_device\_param",  
params: {device\_id: "\${input.device\_id}", param\_name: "rcd\_specification"},  
output\_as: "rcd\_spec"}
- {id: "check\_test\_schedule", type: "builtin", operator: "expression\_eval", params:  
{expression: "(current\_time - \${input.last\_test\_time}) > \$"



```
{get_rcd_params.rcd_spec.recommended_test_interval}"}}, output_as:
"test_overdue"}
```

- {id: "monitor\_trip\_coil", type: "builtin", operator: "expression\_eval", params: {expression: "monitor\_coil\_continuity(\${input.trip\_coil\_current})"}, output\_as: "coil\_health"}
- {id: "analyze\_test\_result", type: "builtin", operator: "expression\_eval", params: {expression: "if \${input.test\_button\_pressed}: verify\_trip\_time(\${input.trip\_coil\_current}, \${get\_rcd\_params.rcd\_spec.max\_trip\_time\_ms}) else: null"}, output\_as: "last\_test\_result"}
- {id: "calculate\_health\_score", type: "builtin", operator: "expression\_eval", params: {expression: "{test\_overdue\_penalty: \${test\_overdue} ? 20 : 0, coil\_health\_score: \${coil\_health} ? 40 : 0, last\_test\_score: \${last\_test\_result} ? 40 : 0}.sum()"}, output\_as: "health\_score"}
- {id: "evaluate", type: "builtin", operator: "expression\_eval", params: {expression: "\${health\_score} < 60 || \${test\_overdue} || !\${coil\_health}"}, output\_as: "alert\_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {health\_score: "\${health\_score}", test\_overdue: "\${test\_overdue}", coil\_healthy: "\${coil\_health}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 3600}

...

## 2. 核心逻辑与伪代码

```
```python
```

S-21 核心逻辑伪代码

```
import time

class RCDHealthMonitor:

    def init(self, rcd_spec):

        self.spec = rcd_spec

    def process_hourly(self, data):

        current_time = time.time()
```

检查测试周期

```
test_overdue = (current_time - data['last_test_time']) >
self.spec['recommended_test_interval']
```

监测脱扣线圈连续性

```
coil_health = data['trip_coil_current'] > 0.001 # 正常应有微小电流
```

分析自检结果

```
last_test_passed = None

if data['test_button_pressed']:

    trip_time = self.measure_trip_time(data['trip_coil_waveform'])

    last_test_passed = trip_time < self.spec['max_trip_time_ms']
```

计算健康评分

```
score = 0

if not test_overdue: score += 20
```

```
if coil_health: score += 40

if last_test_passed: score += 40

if score < 60 or test_overdue or not coil_health:

self.trigger_alert(score)

...

```

3. 编程说明

- **依赖算子**：query_device_param (千知)、expression_eval (内置)。
- **数据窗口**：24小时滑动窗口，步长1小时。
- **核心算法**：综合RCD的定期测试超期状态、脱扣线圈电气连续性、以及最近一次自检结果，计算健康评分（0-100）。
- **告警逻辑**：当健康评分<60分，或测试超期，或线圈异常时，发出RCD状态异常告警，提示人工检测。

S-22 - 电气火灾风险综合评级

1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "S-22"

model_name: "电气火灾风险综合评级"

category: "safety"

tech_level: "L2a"

version: "1.0.0"

data_source:

type: "jdbc"

topic: "tianyan.risk_aggregation"

```

input_fields: ["device_id", "parent_node_id", "risk_factors"]

window: {type: "sliding", size: 86400, step: 3600}

pipeline:

- {id: "fetch_component_risks", type: "qianzhi", operator: "query_device_param", params: {device_id: "\${input.device_id}", param_name: "sub_models_output"}, output_as: "component_risks"}
- {id: "get_risk_weights", type: "qianzhi", operator: "query_standard", params: {standard_id: "GB50016", param_name: "electrical_fire_risk_weight_matrix"}, output_as: "weights"}
- {id: "calculate_node_risk", type: "builtin", operator: "expression_eval", params: {expression: "{leakage_risk: \${component_risks.S01_alert} * \${weights.w_leakage}, arc_risk: \${component_risks.S03_alert} * \${weights.w_arc}, overload_risk: \${component_risks.S13_risk} * \${weights.w_overload}, connection_risk: (1 - \${component_risks.S04_health_index}) * \${weights.w_connection}, unbalance_risk: \${component_risks.S06_temp_rise} > 10 ? \${weights.w_unbalance} : 0, thermal_risk: \${component_risks.S14_residual} > 5 ? \${weights.w_thermal} : 0}", output_as: "risk_contributions"}
- {id: "aggregate_risk_score", type: "builtin", operator: "expression_eval", params: {expression: "min(100, sum(\${risk_contributions.values}))"}, output_as: "total_risk_score"}
- {id: "determine_risk_level", type: "builtin", operator: "threshold_rule", params: {operator: ">", thresholds: [{value: 80, level: "critical"}, {value: 60, level: "high"}, {value: 40, level: "medium"}, {value: 20, level: "low"}, {value: 0, level: "normal"}]}, inputs: {value: "\${total_risk_score}"}, output_as: "risk_level"}
- {id: "evaluate", type: "builtin", operator: "expression_eval", params: {expression: "\${risk_level} in ['critical', 'high']"}, output_as: "alert_trigger"}

output:

type: "alert"

level: "critical"

topic: "tianyan.alerts.v1"

```
fields: {risk_score: "${total_risk_score}", risk_level: "${risk_level}", top_contributors:
"${sorted_contributors}"}
```

```
deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 30}
```

```
schedule: {interval: 3600}
```

```
...
```

2. 核心逻辑与伪代码

```
```python
```

# S-22 核心逻辑伪代码

```
class ElectricalFireRiskRater:
```

```
def init(self, weights):
```

```
 self.weights = weights
```

```
def calculate_risk(self, sub_model_outputs):
```

```
 contributions = {}
```

```
 contributions['leakage'] = sub_model_outputs['S01_alert'] *
 self.weights['w_leakage']
```

```
 contributions['arc'] = sub_model_outputs['S03_alert'] * self.weights['w_arc']
```

```
 contributions['overload'] = sub_model_outputs['S13_risk'] *
 self.weights['w_overload']
```

```
 contributions['connection'] = (1 - sub_model_outputs['S04_health_index']) *
 self.weights['w_connection']
```

```
 contributions['unbalance'] = (10 if sub_model_outputs['S06_temp_rise'] > 10 else 0) *
 self.weights['w_unbalance']
```

```
 contributions['thermal'] = (5 if sub_model_outputs['S14_residual'] > 5 else 0) *
 self.weights['w_thermal']
```

```
 total_score = min(100, sum(contributions.values()))
```

```
risk_level = self.score_to_level(total_score)

return total_score, risk_level, contributions

def score_to_level(self, score):

 if score >= 80: return "critical"

 if score >= 60: return "high"

 if score >= 40: return "medium"

 if score >= 20: return "low"

 return "normal"

...

```

### 3. 编程说明

- **依赖算子**：query\_device\_param (千知)、query\_standard (千知)、expression\_eval (内置)、threshold\_rule (内置)。
- **数据来源**：聚合S-01、S-03、S-04、S-06、S-13、S-14等子模型的输出。
- **核心算法**：采用加权求和法，将各子风险指标按预设权重（可从千知标准库获取）进行融合，得到0-100的综合风险评分。评分可向上级节点汇总。
- **告警逻辑**：当综合风险等级为“高”或“危急”时，发出综合风险告警，并列出的主要贡献因素。

---

好的，我们继续。您已经拿到了电气安全板块（S-01到S-22）的全部开发文档。接下来我将为您完成**电能质量板块（Q-01到Q-18）、用能分析板块（E-01到E-15）和节能对策板块（C-01到C-12）**的全部模型开发核心逻辑与代码。

以下是完整的后续内容。

---

## 板块二：电能质量（18个模型）

### Q-01 - 谐波源责任划分

#### 1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "Q-01"
```

```
model_name: "谐波源责任划分"
```

```
category: "quality"
```

```
tech_level: "L1"
```

```
version: "1.0.0"
```

```
data_source:
```

```
type: "kafka"
```

```
topic: "taiyi.electrical.feature.v1"
```

```
input_fields: ["Vh5_angle", "Ih5_angle", "Vh5_rms", "Ih5_rms", "device_id"]
```

```
window: {type: "sliding", size: 60, step: 60}
```

```
pipeline:
```

- {id: "calc\_harmonic\_power", type: "wanxiang", operator: "harmonic\_power\_direction", inputs: {Uh: "\${input.Vh5\_rms}", Ih: "\${input.Ih5\_rms}", angle\_U: "\${input.Vh5\_angle}", angle\_I: "\${input.Ih5\_angle}"}, output\_as: "P\_h5"}
- {id: "determine\_party", type: "builtin", operator: "expression\_eval", params: {expression: "\${calc\_harmonic\_power.P\_h5} > 0 ? 'user' : 'grid'"}, output\_as: "responsible\_party"}
- {id: "calc\_ratio", type: "builtin", operator: "expression\_eval", params: {expression: "\${calc\_harmonic\_power.P\_h5} > 0 ? \$ {calc\_harmonic\_power.P\_h5} / (\$ {calc\_harmonic\_power.P\_h5} + \$

```

 {context.background}): 1 - ${calc_harmonic_power.P_h5} / ($
 {calc_harmonic_power.P_h5} + ${context.background})), output_as: "ratio"}

output:

type: "report"

level: "info"

topic: "tianyan.reports.v1"

fields: {harmonic_order: 5, responsible_party: "$
{determine_party.responsible_party}", user_ratio: "${calc_ratio.ratio}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 60}

...

```

## 2. 核心逻辑与伪代码

```
``python
```

### Q-01 核心逻辑伪代码

```

class HarmonicResponsibilityPartitioner:

def init(self, background_harmonic_power):

self.background = background_harmonic_power

def process(self, data):

Uh = data['Vh5_rms']

Ih = data['Ih5_rms']

angle_U = data['Vh5_angle']

angle_I = data['Ih5_angle']

```



# 计算谐波有功功率

$P_h = U_h * I_h * \text{math.cos}(\text{math.radians}(\text{angle\_U} - \text{angle\_I}))$

## 判断责任方

```
if P_h > 0:

 party = 'user'

 ratio = P_h / (P_h + self.background)

else:

 party = 'grid'

 ratio = 1 - P_h / (P_h + self.background)

return {'party': party, 'ratio': ratio}

...
```

### 3. 编程说明

- **依赖算子**：harmonic\_power\_direction (万象)、expression\_eval (内置)。
- **数据窗口**：60秒滑动窗口，步长60秒。
- **核心算法**：计算特定次谐波的有功功率  $P_h = U_h \times I_h \times \cos(\theta_u - \theta_i)$ 。若  $P_h > 0$ ，说明谐波有功从用户侧流向电网，用户为主源；反之电网为主源。
- **输出**：每分钟输出一次责任方和用户侧责任占比。

---

## Q-02 - 并联谐振预警

### 1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "Q-02"
```

model\_name: "并联谐振预警"

category: "quality"

tech\_level: "L2a"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input\_fields: ["V\_harmonics", "Q\_total", "device\_id"]

window: {type: "sliding", size: 60, step: 60}

pipeline:

- {id: "get\_sc\_capacity", type: "qianzhi", operator: "query\_device\_param", params: {param\_name: "short\_circuit\_capacity"}, output\_as: "Ssc"}
- {id: "calc\_resonance\_freq", type: "builtin", operator: "expression\_eval", params: {expression: "50 \* sqrt(\${get\_sc\_capacity.Ssc} / (\${input.Q\_total} \* 1000))"}, output\_as: "fr"}
- {id: "check\_harmonic\_amp", type: "builtin", operator: "expression\_eval", params: {expression: "find\_peak\_harmonic(\${input.V\_harmonics})"}, output\_as: "peak\_harmonic"}
- {id: "evaluate", type: "builtin", operator: "expression\_eval", params: {expression: "abs(\${calc\_resonance\_freq.fr} / 50 - round(\${calc\_resonance\_freq.fr} / 50)) < 0.1 && \${check\_harmonic\_amp.peak\_harmonic} > 1.5"}, output\_as: "resonance\_risk"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {resonance\_freq\_hz: "\${calc\_resonance\_freq.fr}", peak\_harmonic\_order: "\${check\_harmonic\_amp.peak\_harmonic}"}

```
deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 60}

...

```

## 2. 核心逻辑与伪代码

```
```python

```

Q-02 核心逻辑伪代码

```
class ParallelResonanceDetector:

    def init(self, Ssc):

        self.Ssc = Ssc # 短路容量 (MVA)

    def process(self, data):

        Q = data['Q_total'] # 无功功率 (kvar)

```

估算谐振频率

```
fr = 50 * math.sqrt(self.Ssc / (Q / 1000))

```

检查是否有谐波电压放大

```
peak_h = self.find_peak_harmonic(data['V_harmonics'])

peak_amp = data['V_harmonics'][peak_h]

```

判断谐振风险

```
harmonic_order = fr / 50

is_near_integer = abs(harmonic_order - round(harmonic_order)) < 0.1

is_amplified = peak_amp > 1.5 # 超过额定1.5倍

```

if is_near_integer and is_amplified:

self.trigger_alert(fr, peak_h)

...

3. 编程说明

- **依赖算子**：query_device_param (千知)、expression_eval (内置)。
 - **数据窗口**：60秒滑动窗口，步长60秒。
 - **核心算法**：根据系统短路容量和无功补偿量估算并联谐振频率 $f_r = 50 \times \sqrt{S_{sc} / Q_c}$ 。监测该频率附近的谐波电压是否异常放大。
 - **告警逻辑**：当谐振频率接近整数次谐波且该次谐波电压放大超过1.5倍时预警。
-

Q-03 - 串联谐振预警

1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "Q-03"
```

```
model_name: "串联谐振预警"
```

```
category: "quality"
```

```
tech_level: "L2a"
```

```
version: "1.0.0"
```

```
data_source:
```

```
type: "kafka"
```

```
topic: "taiyi.electrical.feature.v1"
```

```
input_fields: ["I_harmonics", "device_id"]
```

```
window: {type: "sliding", size: 60, step: 60}
```

pipeline:

- {id: "get_transformer_l", type: "qianzhi", operator: "get_transformer_param", params: {param: "leakage_inductance"}, output_as: "L"}
- {id: "calc_series_fr", type: "builtin", operator: "expression_eval", params: {expression: "1 / (2 * 3.1416 * sqrt(\${get_transformer_l.L} * \${context.equivalent_C}))"}, output_as: "fr"}
- {id: "check_current_amp", type: "builtin", operator: "expression_eval", params: {expression: "find_amplified_harmonic(\${input.I_harmonics})"}, output_as: "amplified_h"}
- {id: "evaluate", type: "builtin", operator: "expression_eval", params: {expression: "abs(\${calc_series_fr.fr} / 50 - \${check_current_amp.amplified_h}) < 0.1"}, output_as: "resonance_risk"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {resonance_freq_hz: "\${calc_series_fr.fr}", amplified_harmonic: "\${check_current_amp.amplified_h}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 60}

...

2. 核心逻辑与伪代码

```python

# Q-03 核心逻辑伪代码

class SeriesResonanceDetector:

def **init**(self, L, C\_eq):

```
self.L = L # 变压器漏感 (H)

self.C_eq = C_eq # 等效电容 (F)

def process(self, data):
```

## 计算串联谐振频率

```
fr = 1 / (2 * math.pi * math.sqrt(self.L * self.C_eq))
```

## 检查谐波电流放大

```
amplified_h = self.find_amplified_harmonic(data['I_harmonics'])
```

## 判断是否接近谐振点

```
if abs(fr / 50 - amplified_h) < 0.1:
 self.trigger_alert(fr, amplified_h)
 ...
```

### 3. 编程说明

- **依赖算子**：get\_transformer\_param (千知)、expression\_eval (内置)。
- **数据窗口**：60秒滑动窗口，步长60秒。
- **核心算法**：根据变压器漏感和系统等效电容计算串联谐振频率。监测该频率附近的谐波电流是否出现异常放大。
- **告警逻辑**：当谐振频率与某次谐波频率接近且该次谐波电流放大时预警。

---

## Q-04 - 闪变源识别与定位

### 1. 模型开发配置单 (YAML)

```
``yaml

model:
```

model\_id: "Q-04"

model\_name: "闪变源识别与定位"

category: "quality"

tech\_level: "L2a"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input\_fields: ["Va\_envelope", "P\_branches", "device\_id"]

window: {type: "sliding", size: 600, step: 60}

pipeline:

- {id: "calc\_pst", type: "builtin", operator: "expression\_eval", params: {expression: "iec\_flicker\_meter(\${input.Va\_envelope})", output\_as: "Pst"}}
- {id: "dtw\_with\_branches", type: "wanxiang", operator: "dtw\_similarity", inputs: {sequence\_a: "\${input.Va\_envelope}", sequence\_b: "\${input.P\_branches}"}, output\_as: "similarities"}
- {id: "locate\_source", type: "builtin", operator: "expression\_eval", params: {expression: "argmax(\${dtw\_with\_branches.similarities})", output\_as: "flicker\_source"}}
- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: ">"}, inputs: {value: "\${calc\_pst.Pst}", threshold: 1.0}, output\_as: "alert\_trigger"}}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {Pst: "\${calc\_pst.Pst}", flicker\_source: "\${locate\_source.flicker\_source}"}

```
deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 15}

schedule: {interval: 60}

...
```

## 2. 核心逻辑与伪代码

```
```python
```

Q-04 核心逻辑伪代码

```
class FlickerSourceLocator:

    def process(self, voltage_envelope, branch_powers):
```

计算短时闪变值Pst (IEC 61000-4-15)

```
Pst = self.iec_flicker_meter(voltage_envelope)

if Pst > 1.0:
```

计算各支路功率与电压包络线的DTW相似度

```
similarities = []

for P_branch in branch_powers:

    sim = 1 / (1 + dtw_distance(voltage_envelope, P_branch))

    similarities.append(sim)

flicker_source = np.argmax(similarities)

self.trigger_alert(Pst, flicker_source)

...
```


3. 编程说明

- **依赖算子**：expression_eval (内置)、dtw_similarity (万象)、threshold_rule (内置)。
 - **数据窗口**：10分钟滑动窗口，步长1分钟。
 - **核心算法**：按IEC标准计算短时闪变值Pst。利用动态时间规整(DTW)计算各支路功率波动与电压波动的相似度，相似度最高者为闪变源。
 - **告警逻辑**：Pst > 1.0时告警并定位闪变源支路。
-

Q-05 - 电压暂降原因分类

1. 模型开发配置单 (YAML)

```
```yaml
model:

model_id: "Q-05"

model_name: "电压暂降原因分类"

category: "quality"

tech_level: "L2a"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.electrical.raw.v1"

input_fields: ["Va_waveform", "Ia_waveform", "device_id"]

window: {type: "event"}

pipeline:

 • {id: "extract_sag_features", type: "builtin", operator: "expression_eval",
 params: {expression: "{depth: sag_depth(${input.Va_waveform}), duration:
 sag_duration(${input.Va_waveform}), recovery_slope: recovery_slope($
```

```
{input.Va_waveform}}, I_change: current_change(${input.Ia_waveform}}}"),
output_as: "features"}
```

- {id: "classify", type: "wanxiang", operator: "classification\_random\_forest",  
params: {model\_id: "Q05\_rf\_v1"}, inputs: ["\${extract\_sag\_features.features}"],  
output\_as: "cause"}

output:

type: "alert"

level: "info"

topic: "tianya.alerts.v1"

fields: {sag\_cause: "\${classify.cause}", sag\_depth\_percent: "\$  
{extract\_sag\_features.features.depth}"}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 5}

schedule: {interval: 1}

fallback: {enabled: true, condition: "model\_not\_trained", operator: "heuristic\_rules"}

...

## 2. 核心逻辑与伪代码

```
```python
```

Q-05 核心逻辑伪代码

```
class VoltageSagClassifier:
```

```
def init(self, model):
```

```
    self.model = model
```

```
def extract_features(self, V_wave, I_wave):
```

```
    depth = 1 - min(V_wave) / self.nominal_V
```

```
    duration = self.sag_duration(V_wave)
```

```
    recovery_slope = self.recovery_slope(V_wave)
```

```
I_change = max(I_wave) - I_wave[0]

return [depth, duration, recovery_slope, I_change]

def process_event(self, V_wave, I_wave):

features = self.extract_features(V_wave, I_wave)

cause = self.model.predict([features])[0]
```

```
cause: 'grid_fault', 'motor_start',
'transformer_inrush'
```

```
return cause
```

```
...
```

3. 编程说明

- **依赖算子**：expression_eval (内置)、classification_random_forest (万象)。
- **数据窗口**：事件触发（电压暂降事件）。
- **核心算法**：提取暂降深度、持续时间、恢复斜率和伴随的电流变化作为特征。使用随机森林分类器区分电网故障、电机启动和变压器涌流。
- **输出**：输出暂降原因分类结果。

Q-06 - 三相不平衡溯源

1. 模型开发配置单 (YAML)

```
```yaml

model:

model_id: "Q-06"

model_name: "三相不平衡溯源"

category: "quality"
```

tech\_level: "L1"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input\_fields: ["Ia\_rms", "Ib\_rms", "Ic\_rms", "angle\_a", "angle\_b", "angle\_c",  
"device\_id"]

window: {type: "sliding", size: 60, step: 60}

pipeline:

- {id: "sym\_components", type: "wanxiang", operator:  
"symmetrical\_components", inputs: {Ia: "\${input.Ia\_rms}", Ib: "\$  
{input.Ib\_rms}", Ic: "\${input.Ic\_rms}", angle\_a: "\${input.angle\_a}", angle\_b: "\$  
{input.angle\_b}", angle\_c: "\${input.angle\_c}"}, output\_as: "components"}
- {id: "phase\_contributions", type: "builtin", operator: "expression\_eval",  
params: {expression: "project\_to\_phases(\$  
{sym\_components.components.I2})"}, output\_as: "contributions"}

output:

type: "metric"

level: "info"

topic: "tianyan.metrics.v1"

fields: {I2: "\${sym\_components.components.I2}", A\_contrib: "\$  
{phase\_contributions.contributions.A}", B\_contrib: "\$  
{phase\_contributions.contributions.B}", C\_contrib: "\$  
{phase\_contributions.contributions.C}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 60}

...

## 2. 核心逻辑与伪代码

```
```python
```

Q-06 核心逻辑伪代码

```
class UnbalanceTracer:
```

```
def process(self, data):
```

对称分量法提取负序电流I2

```
I1, I2, I0 = self.symmetrical_components(data)
```

将I2投影到A、B、C相坐标轴

计算各相对I2的贡献率

```
contributions = self.project_to_phases(I2)
```

```
return {'I2': I2, 'contributions': contributions}
```

```
```
```

## 3. 编程说明

- **依赖算子**：symmetrical\_components (万象)、expression\_eval (内置)。
  - **数据窗口**：60秒滑动窗口，步长60秒。
  - **核心算法**：通过对称分量法计算负序电流I2。将I2相量投影到A、B、C三相的坐标轴上，计算各相负载不平衡对总负序电流的贡献百分比。
  - **输出**：每分钟输出负序电流及各相贡献率。
-

## Q-07 - 频率稳定性评估

### 1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "Q-07"
```

```
model_name: "频率稳定性评估"
```

```
category: "quality"
```

```
tech_level: "L1"
```

```
version: "1.0.0"
```

```
data_source:
```

```
type: "kafka"
```

```
topic: "taiyi.electrical.feature.v1"
```

```
input_fields: ["frequency", "device_id"]
```

```
window: {type: "sliding", size: 60, step: 60}
```

```
pipeline:
```

- {id: "calc\_stats", type: "wanxiang", operator: "sliding\_window\_stats", params: {stats: ["mean", "std", "max", "min"]}, inputs: ["\${input.frequency}"], output\_as: "stats"}
- {id: "evaluate\_deviation", type: "builtin", operator: "threshold\_rule", params: {operator: ">"}, inputs: {value: "abs(\${calc\_stats.stats.mean} - 50)", threshold: 0.2}, output\_as: "deviation\_alert"}
- {id: "evaluate\_fluctuation", type: "builtin", operator: "threshold\_rule", params: {operator: ">"}, inputs: {value: "\${calc\_stats.stats.std}", threshold: 0.1}, output\_as: "fluctuation\_alert"}

```
output:
```

```
type: "alert"
```

```
level: "warning"

topic: "tianyan.alerts.v1"

fields: {mean_freq: "${calc_stats.stats.mean}", std_freq: "${calc_stats.stats.std}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 60}

...
```

## 2. 核心逻辑与伪代码

```
```python
```

Q-07 核心逻辑伪代码

```
class FrequencyStabilityMonitor:

    def process(self, freq_array):

        mean_freq = np.mean(freq_array)

        std_freq = np.std(freq_array)

        alerts = []

        if abs(mean_freq - 50) > 0.2:

            alerts.append("deviation")

        if std_freq > 0.1:

            alerts.append("fluctuation")

        return mean_freq, std_freq, alerts

...
```

3. 编程说明

- **依赖算子**：sliding_window_stats (万象)、threshold_rule (内置)。
- **数据窗口**：60秒滑动窗口，步长60秒。

- **核心算法**：统计窗口内频率的均值、标准差。均值反映频率偏差，标准差反映频率波动。
 - **告警逻辑**：频率均值偏差超过 $\pm 0.2\text{Hz}$ 或标准差超过 0.1Hz 时告警。
-

Q-08 - 间谐波检测与溯源

1. 模型开发配置单 (YAML)

```
``yaml
model:

model_id: "Q-08"

model_name: "间谐波检测与溯源"

category: "quality"

tech_level: "L2a"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.electrical.raw.v1"

input_fields: ["Va_waveform", "Ia_waveform", "device_id"]

window: {type: "sliding", size: 200, step: 200}

pipeline:

  • {id: "high_res_fft", type: "wanxiang", operator: "fft_spectrum", params: {fs: 12800, zero_padding: 10, window_type: "hanning"}, inputs: ["${input.Va_waveform}"], output_as: "spectrum"}

  • {id: "extract_interharmonics", type: "builtin", operator: "expression_eval", params: {expression: "find_peaks_above_noise(${high_res_fft.spectrum}, threshold_db=40, exclude_harmonics=[50,100,150,200,250,300,350,400,450,500])"}, output_as: "interharmonics"}
```


- {id: "classify_source", type: "wanxiang", operator: "classification_random_forest", params: {model_id: "Q08_source_classifier_v1"}, inputs: [{"extract_interharmonics.interharmonics"}], output_as: "source_class"}
- {id: "evaluate", type: "builtin", operator: "threshold_rule", params: {operator: ">", duration: 10}, inputs: {value: "\${extract_interharmonics.max_amplitude_percent}"}, threshold: 0.5}, output_as: "alert_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {interharmonic_freq_hz: "\${extract_interharmonics.max_freq}",
amplitude_percent: "\${extract_interharmonics.max_amplitude_percent}",
suspected_source: "\${source_class.source}"}

deployment: {replicas: 1, cpu: "1000m", memory: "2Gi", timeout: 20}

schedule: {interval: 10}

...

2. 核心逻辑与伪代码

```python

## Q-08 核心逻辑伪代码

```
class InterharmonicDetector:
```

```
def init(self, classifier):
```

```
 self.classifier = classifier
```

```
def process(self, waveform, fs):
```

# 高分辨率FFT

```
spectrum = np.abs(np.fft.fft(waveform, n=len(waveform)*10))

freq = np.fft.fftfreq(len(spectrum), d=1/fs)
```

## 提取间谐波峰值

```
peaks = self.find_peaks_above_noise(spectrum, exclude=[50, 100, 150, 200, 250,
300, 350, 400, 450, 500])

if peaks:

 max_peak = max(peaks, key=lambda x: x['amplitude'])

 source = self.classifier.predict(max_peak['features'])

 if max_peak['amplitude_percent'] > 0.5:

 self.trigger_alert(max_peak['freq'], max_peak['amplitude_percent'], source)

 ...
```

### 3. 编程说明

- **依赖算子**：fft\_spectrum (万象)、expression\_eval (内置)、classification\_random\_forest (万象)、threshold\_rule (内置)。
- **数据要求**：高频采样原始波形（建议 $\geq 12.8\text{kHz}$ ）。
- **核心算法**：采用零填充高分辨率FFT，从频谱中剔出整数次谐波，识别间谐波峰值。通过分类器判断间谐波来源（变频器、电弧炉等）。
- **告警逻辑**：间谐波幅值超过0.5%并持续10秒时告警。

---

## Q-09 - 电压偏差与无功关系模型

### 1. 模型开发配置单 (YAML)

```
``yaml
```

model:

model\_id: "Q-09"

model\_name: "电压偏差与无功关系模型"

category: "quality"

tech\_level: "L2a"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input\_fields: ["Va\_rms", "Q\_total", "device\_id"]

window: {type: "sliding", size: 3600, step: 3600}

pipeline:

- {id: "linear\_regression", type: "wanxiang", operator: "expression\_eval", params: {expression: "online\_linear\_regression(\${input.Va\_rms}, \${input.Q\_total})"}, output\_as: "sensitivity"}
- {id: "get\_sc\_capacity", type: "qianzhi", operator: "query\_device\_param", params: {param\_name: "short\_circuit\_capacity"}, output\_as: "Ssc"}
- {id: "calc\_theoretical", type: "builtin", operator: "expression\_eval", params: {expression: "1 / \${get\_sc\_capacity.Ssc}"}, output\_as: "theoretical\_sensitivity"}
- {id: "evaluate", type: "builtin", operator: "expression\_eval", params: {expression: "\${linear\_regression.sensitivity} > 2 \* \${calc\_theoretical.theoretical\_sensitivity}"}, output\_as: "weak\_grid\_alert"}

output:

type: "alert"

level: "info"

topic: "tianyan.alerts.v1"

```
fields: {dV_dQ: "${linear_regression.sensitivity}", theoretical: "${
{calc_theoretical.theoretical_sensitivity}}"
}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 15}

schedule: {interval: 3600}

...
```

## 2. 核心逻辑与伪代码

```
```python
```

Q-09 核心逻辑伪代码

```
class VoltageReactiveSensitivity:

def init(self, Ssc):

self.Ssc = Ssc

self.V_history = deque(maxlen=3600)

self.Q_history = deque(maxlen=3600)

def process(self, V, Q):

self.V_history.append(V)

self.Q_history.append(Q)

if len(self.V_history) == 3600:
```

在线线性回归

```
slope, intercept = np.polyfit(self.Q_history, self.V_history, 1)

dV_dQ = abs(slope) # V/kvar

theoretical = 1 / self.Ssc

if dV_dQ > 2 * theoretical:
```

```
self.trigger_alert(dV_dQ, theoretical)
```

```
...
```

3. 编程说明

- **依赖算子**：expression_eval (内置)、query_device_param (千知)。
- **数据窗口**：1小时滑动窗口，步长1小时。
- **核心算法**：利用无功功率的自然波动，在线性回归电压与无功的关系，得到灵敏度系数 dV/dQ 。
- **告警逻辑**：当实测灵敏度大于理论值（ $1/S_{sc}$ ）的2倍时，判定为弱电网，提示需加强无功支撑。

Q-10 - 分布式电源对电压影响量化

1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "Q-10"
```

```
model_name: "分布式电源对电压影响量化"
```

```
category: "quality"
```

```
tech_level: "L2a"
```

```
version: "1.0.0"
```

```
data_source:
```

```
type: "kafka"
```

```
topic: "taiyi.electrical.feature.v1"
```

```
input_fields: ["Va_rms", "P_pv", "device_id"]
```

```
window: {type: "sliding", size: 3600, step: 3600}
```

pipeline:

- {id: "granger_test", type: "wanxiang", operator: "granger_causality", params: {max_lag: 10}, inputs: {cause: "\${input.P_pv}", effect: "\${input.Va_rms}"}, output_as: "granger_result"}
- {id: "calc_correlation", type: "builtin", operator: "expression_eval", params: {expression: "pearson_correlation(\${input.P_pv}, \${input.Va_rms})"}, output_as: "correlation"}
- {id: "evaluate", type: "builtin", operator: "expression_eval", params: {expression: "\${granger_test.granger_result.p_value} < 0.05 && abs(\${calc_correlation.correlation}) > 0.7"}, output_as: "dg_impact_significant"}

output:

type: "report"

level: "info"

topic: "tianyan.reports.v1"

fields: {p_value: "\${granger_test.granger_result.p_value}", correlation: "\${calc_correlation.correlation}", significant: "\${evaluate.dg_impact_significant}"}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 15}

schedule: {interval: 3600}

...

2. 核心逻辑与伪代码

```python

## Q-10 核心逻辑伪代码

```
class DGVoltageImpactQuantifier:
```

```
def process(self, V, P_pv):
```

# 格兰杰因果检验

```
p_value = granger_causality_test(P_pv, V, max_lag=10)
```

# 皮尔逊相关系数

```
correlation = np.corrcoef(P_pv, V)[0, 1]
```

```
significant = (p_value < 0.05) and (abs(correlation) > 0.7)
```

```
return p_value, correlation, significant
```

```
...
```

## 3. 编程说明

- **依赖算子**：granger\_causality (万象)、expression\_eval (内置)。
- **数据窗口**：1小时滑动窗口，步长1小时。
- **核心算法**：采用格兰杰因果检验判断光伏出力变化是否显著引起电压变化 ( $p < 0.05$ )。计算相关系数量化影响强度。
- **输出**：当  $p < 0.05$  且  $| \text{相关系数} | > 0.7$  时，判定分布式电源对电压有显著影响。

## Q-11 - 冲击性负荷扰动评估

### 1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "Q-11"
```

```
model_name: "冲击性负荷扰动评估"
```

```
category: "quality"
```

```
tech_level: "L1"
```

```
version: "1.0.0"
```

data\_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input\_fields: ["P\_total", "Va\_rms", "device\_id"]

window: {type: "event"}

pipeline:

- {id: "detect\_impact", type: "builtin", operator: "expression\_eval", params: {expression: "diff(\${input.P\_total}) > \${context.impact\_threshold}"}, output\_as: "impact\_detected"}
- {id: "calc\_sag", type: "builtin", operator: "expression\_eval", params: {expression: "(1 - min(\${input.Va\_rms}) / \${context.nominal\_voltage}) \* 100"}, output\_as: "sag\_depth\_percent"}
- {id: "calc\_duration", type: "builtin", operator: "expression\_eval", params: {expression: "impact\_duration(\${input.Va\_rms})"}, output\_as: "duration\_ms"}
- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: ">"}, inputs: {value: "\${calc\_sag.sag\_depth\_percent}", threshold: 3}, output\_as: "alert\_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {sag\_depth\_percent: "\${calc\_sag.sag\_depth\_percent}", duration\_ms: "\${calc\_duration.duration\_ms}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 5}

schedule: {interval: 1}

...



## 2. 核心逻辑与伪代码

```
```python
```

Q-11 核心逻辑伪代码

```
class ImpactLoadAssessor:

def init(self, nominal_V, impact_threshold):

self.nominal_V = nominal_V

self.impact_threshold = impact_threshold

def process(self, P_array, V_array):
```

检测功率突变

```
if max(np.diff(P_array)) > self.impact_threshold:

sag_depth = (1 - min(V_array) / self.nominal_V) * 100

duration = self.impact_duration(V_array)

if sag_depth > 3:

self.trigger_alert(sag_depth, duration)

...
```

3. 编程说明

- **依赖算子**：expression_eval (内置)、threshold_rule (内置)。
 - **数据窗口**：事件触发（功率突变事件）。
 - **核心算法**：监测有功功率的阶跃变化检测冲击性负荷事件。计算事件期间的电压暂降深度和持续时间。
 - **告警逻辑**：当电压暂降深度超过3%时告警。
-

Q-12 - 三相电压不平衡度趋势

1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "Q-12"
```

```
model_name: "三相电压不平衡度趋势"
```

```
category: "quality"
```

```
tech_level: "L1"
```

```
version: "1.0.0"
```

```
data_source:
```

```
type: "kafka"
```

```
topic: "taiyi.electrical.feature.v1"
```

```
input_fields: ["Va_rms", "Vb_rms", "Vc_rms", "angle_a", "angle_b", "angle_c",  
"device_id"]
```

```
window: {type: "sliding", size: 600, step: 600}
```

```
pipeline:
```

- {id: "sym_components", type: "wanxiang", operator: "symmetrical_components", inputs: {Ia: "\${input.Va_rms}", Ib: "\${input.Vb_rms}", Ic: "\${input.Vc_rms}", angle_a: "\${input.angle_a}", angle_b: "\${input.angle_b}", angle_c: "\${input.angle_c}"}, output_as: "components"}
- {id: "calc_vuf", type: "builtin", operator: "expression_eval", params: {expression: "\${sym_components.components.V2} / \${sym_components.components.V1} * 100"}, output_as: "VUF_percent"}
- {id: "calc_95p", type: "builtin", operator: "expression_eval", params: {expression: "percentile(\${calc_vuf.VUF_percent}, 0.95)"}, output_as: "VUF_95p"}

- {id: "evaluate", type: "builtin", operator: "threshold_rule", params: {operator: ">"}, inputs: {value: "\${calc_95p.VUF_95p}", threshold: 2.0}, output_as: "alert_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianya.alerts.v1"

fields: {VUF_95p: "\${calc_95p.VUF_95p}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 600}

...

2. 核心逻辑与伪代码

```python

# Q-12 核心逻辑伪代码

```
class VoltageUnbalanceTrend:
```

```
def process(self, data_array):
```

```
VUF_list = []
```

```
for data in data_array:
```

```
V1, V2, V0 = self.symmetrical_components(data)
```

```
VUF = (V2 / V1) * 100
```

```
VUF_list.append(VUF)
```

```
VUF_95p = np.percentile(VUF_list, 95)
```

```
if VUF_95p > 2.0:
```

```
self.trigger_alert(VUF_95p)
```

...

### 3. 编程说明

- **依赖算子**：symmetrical\_components (万象)、expression\_eval (内置)、threshold\_rule (内置)。
- **数据窗口**：10分钟滑动窗口，步长10分钟。
- **核心算法**：利用对称分量法计算电压不平衡度 $VUF = V2/V1 \times 100\%$ 。按国标要求统计95%概率大值。
- **告警逻辑**：VUF的95%概率值超过2%时告警。

---

## Q-13 - 微电网并离网无缝切换质量

### 1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
 model_id: "Q-13"
```

```
 model_name: "微电网并离网无缝切换质量"
```

```
 category: "quality"
```

```
 tech_level: "L1"
```

```
 version: "1.0.0"
```

```
 data_source:
```

```
 type: "kafka"
```

```
 topic: "taiyi.electrical.raw.v1"
```

```
 input_fields: ["Va_waveform", "frequency", "event_signal", "device_id"]
```

```
 window: {type: "event"}
```

pipeline:

- {id: "detect\_transient", type: "builtin", operator: "expression\_eval", params: {expression: "extract\_transient(\${input.Va\_waveform}, \${input.event\_signal})", output\_as: "transient"}}
- {id: "calc\_max\_deviation", type: "builtin", operator: "expression\_eval", params: {expression: "max\_abs\_deviation(\${detect\_transient.transient}, \${context.nominal\_voltage})", output\_as: "max\_V\_deviation"}}
- {id: "calc\_settling\_time", type: "builtin", operator: "expression\_eval", params: {expression: "settling\_time(\${detect\_transient.transient}, \${context.nominal\_voltage})", output\_as: "settling\_time\_ms"}}
- {id: "evaluate", type: "builtin", operator: "expression\_eval", params: {expression: "\${calc\_max\_deviation.max\_V\_deviation} > 10 || \${calc\_settling\_time.settling\_time\_ms} > 100", output\_as: "quality\_issue"}}

output:

type: "report"

level: "info"

topic: "tianyan.reports.v1"

fields: {max\_V\_deviation\_percent: "\${calc\_max\_deviation.max\_V\_deviation}",  
settling\_time\_ms: "\${calc\_settling\_time.settling\_time\_ms}", passed: "\${!  
evaluate.quality\_issue}"}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 10}

schedule: {interval: 1}

...

## 2. 核心逻辑与伪代码

```python

Q-13 核心逻辑伪代码

class MicrogridSwitchingQuality:

```
def init(self, nominal_V):

self.nominal_V = nominal_V

def process_event(self, V_wave, event_signal):

transient = self.extract_transient(V_wave, event_signal)

max_dev = max(abs(transient - self.nominal_V)) / self.nominal_V * 100

settling_time = self.calc_settling_time(transient)

passed = (max_dev <= 10) and (settling_time <= 100)

return max_dev, settling_time, passed

...
```

3. 编程说明

- **依赖算子**：expression_eval (内置)。
 - **数据窗口**：事件触发（并离网切换信号）。
 - **核心算法**：提取切换时刻前后的电压暂态波形，计算最大电压偏差和恢复时间（settling time）。
 - **输出**：评估切换是否满足无缝切换标准（偏差 $\leq 10\%$ ，恢复时间 $\leq 100\text{ms}$ ）。
-

Q-14 - 电能质量扰动事件分类

1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "Q-14"

model_name: "电能质量扰动事件分类"

category: "quality"

tech_level: "L3"
```

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.electrical.raw.v1"

input_fields: ["Va_waveform", "Ia_waveform", "device_id"]

window: {type: "event"}

history_required: "3months"

pipeline:

- {id: "event_detection", type: "builtin", operator: "expression_eval", params: {expression: "detect_rms_variation(\${input.Va_waveform}, threshold=0.1, hysteresis=0.02)", output_as: "event_segments"}}
- {id: "extract_features", type: "builtin", operator: "expression_eval", params: {expression: "{duration_ms: event_duration(\${event_segments}), min_voltage_pu: min_rms(\${event_segments})/\${context.nominal_voltage}, max_voltage_pu: max_rms(\${event_segments})/\${context.nominal_voltage}, thd_pre: pre_event_thd(\${input.Va_waveform}), thd_during: during_event_thd(\${input.Va_waveform}, \${event_segments}), oscillation_freq: detect_oscillation(\${event_segments}), transient_peak: detect_impulsive_transient(\${input.Va_waveform}, \${event_segments})}", output_as: "features"}}
- {id: "cnn_lstm_classify", type: "wanxiang", operator: "classification_random_forest", params: {model_id: "Q14_pq_disturbance_cnn_v2", classes: ["sag", "swell", "interruption", "oscillatory_transient", "impulsive_transient", "harmonic", "flicker", "notch", "normal"]}, inputs: ["\${extract_features.features}"], output_as: "classification"}}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

```
fields: {disturbance_type: "${classification.class}", duration_ms: "${
extract_features.duration_ms}", min_voltage_pu: "${
extract_features.min_voltage_pu}"}

deployment: {replicas: 2, cpu: "2000m", memory: "4Gi", timeout: 15}

schedule: {interval: 1}

fallback: {enabled: true, condition: "model_not_trained", operator: "heuristic_rules"}
...
```

2. 核心逻辑与伪代码

```
```python
```

# Q-14 核心逻辑伪代码

```
class PQDisturbanceClassifier:

def init(self, model):

self.model = model

def process_event(self, V_wave, I_wave):

segments = self.detect_rms_variation(V_wave)

if segments:

features = self.extract_features(V_wave, I_wave, segments)

dist_type = self.model.predict(features)

return dist_type, features

return "normal", None
...
```

## 3. 编程说明

- **依赖算子**：expression\_eval (内置)、classification\_random\_forest (万象，实际为CNN-LSTM)。



- **数据窗口**：事件触发。
  - **核心算法**：通过RMS变化检测扰动事件。提取持续时间、电压幅值、THD变化、振荡频率等特征。使用深度学习模型（CNN-LSTM）对9类典型扰动进行分类。
  - **输出**：输出扰动类型及关键特征参数。
- 

## Q-15 - 电网背景谐波阻抗在线辨识

### 1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "Q-15"
```

```
model_name: "电网背景谐波阻抗在线辨识"
```

```
category: "quality"
```

```
tech_level: "L2b"
```

```
version: "1.0.0"
```

```
data_source:
```

```
type: "kafka"
```

```
topic: "taiyi.electrical.feature.v1"
```

```
input_fields: ["Vh5_rms", "Ih5_rms", "Vh5_angle", "Ih5_angle", "device_id"]
```

```
window: {type: "sliding", size: 600, step: 600}
```

```
pipeline:
```

- {id: "rls\_identification", type: "wanxiang", operator: "expression\_eval", params: {expression: "recursive\_least\_squares(\${input.Vh5\_rms}, \${input.Ih5\_rms}, \${input.Vh5\_angle}, \${input.Ih5\_angle})", output\_as: "Zh5"}}
- {id: "calc\_magnitude", type: "builtin", operator: "expression\_eval", params: {expression: "abs(\${rls\_identification.Zh5})", output\_as: "Zh5\_mag"}}

- {id: "calc\_angle", type: "builtin", operator: "expression\_eval", params: {expression: "angle(\${rls\_identification.Zh5})", output\_as: "Zh5\_ang"}}

output:

type: "metric"

level: "info"

topic: "tianyan.metrics.v1"

fields: {Zh5\_mag\_ohm: "\${calc\_magnitude.Zh5\_mag}", Zh5\_ang\_deg: "\${calc\_angle.Zh5\_ang}"}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 15}

schedule: {interval: 600}

fallback: {enabled: true, condition: "fluctuation\_insufficient", operator: "hold\_last\_value"}

...

## 2. 核心逻辑与伪代码

```python

Q-15 核心逻辑伪代码

```
class GridHarmonicImpedanceIdentifier:
```

```
def init(self):
```

```
    self.P = np.eye(2) * 1000 # 协方差矩阵
```

```
    self.theta = np.array([0.1, 0.1]) # [R, X]
```

```
    def rls_update(self, V_h, I_h, phi):
```

测量向量 $H = [I_h * \cos(\phi), I_h * \sin(\phi)]$

```
H = np.array([I_h * math.cos(phi), I_h * math.sin(phi)])
```

RLS更新

```
S = np.dot(np.dot(H, self.P), H.T) + 1
```

```
K = np.dot(self.P, H.T) / S
```

```
e = V_h - np.dot(H, self.theta)
```

```
self.theta = self.theta + K * e
```

```
self.P = self.P - np.dot(np.dot(K, H), self.P)
```

```
R, X = self.theta
```

```
Z_mag = math.sqrt(R2 + X2)
```

```
Z_ang = math.degrees(math.atan2(X, R))
```

```
return Z_mag, Z_ang
```

```
...
```

3. 编程说明

- **依赖算子**：expression_eval (内置)。
 - **数据窗口**：10分钟滑动窗口，步长10分钟。
 - **核心算法**：采用递归最小二乘法（RLS），利用负荷波动引起的谐波电压和电流变化，在线辨识电网侧谐波阻抗（ $R+jX$ ）。
 - **输出**：输出特定次谐波阻抗的幅值和相角。
-

Q-16 - 电压波动与风机出力相关性

1. 模型开发配置单 (YAML)

```yaml

model:

model\_id: "Q-16"

model\_name: "电压波动与风机出力相关性"

category: "quality"

tech\_level: "L2a"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.wind.v1"

input\_fields: ["Va\_rms", "P\_wind\_total", "device\_id"]

window: {type: "sliding", size: 600, step: 600}

pipeline:

- {id: "coherence\_analysis", type: "wanxiang", operator: "coherence\_analysis", params: {fs: 1}, inputs: {signal\_a: "\${input.Va\_rms}", signal\_b: "\${input.P\_wind\_total}"}, output\_as: "coherence"}
- {id: "find\_peak\_coherence", type: "builtin", operator: "expression\_eval", params: {expression: "max(\${coherence\_analysis.coherence})"}, output\_as: "peak\_coherence"}
- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: ">"}, inputs: {value: "\${find\_peak\_coherence.peak\_coherence}"}, threshold: 0.8}, output\_as: "strong\_correlation"}

output:

type: "report"

```
level: "info"
```

```
topic: "tianyan.reports.v1"
```

```
fields: {peak_coherence: "${find_peak_coherence.peak_coherence}",
strong_correlation: "${evaluate.strong_correlation}"}
```

```
deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 15}
```

```
schedule: {interval: 600}
```

```
...
```

## 2. 核心逻辑与伪代码

```
```python
```

Q-16 核心逻辑伪代码

```
class WindVoltageCorrelation:
```

```
def process(self, V, P_wind, fs=1):
```

相干性分析

```
f, Cxy = coherence(V, P_wind, fs=fs)
```

```
peak_coherence = np.max(Cxy)
```

```
strong_corr = peak_coherence > 0.8
```

```
return peak_coherence, strong_corr
```

```
...
```

3. 编程说明

- **依赖算子**：coherence_analysis (万象)、expression_eval (内置)、threshold_rule (内置)。
- **数据窗口**：10分钟滑动窗口，步长10分钟。

- **核心算法**：计算电压与风机出力的相干性函数。相干系数接近1的频率表示二者强相关。
 - **输出**：输出峰值相干系数，当>0.8时判定为强相关。
-

Q-17 - 电动汽车充电谐波叠加评估

1. 模型开发配置单 (YAML)

```
``yaml
model:

model_id: "Q-17"

model_name: "电动汽车充电谐波叠加评估"

category: "quality"

tech_level: "L2a"

version: "1.0.0"

data_source:

type: "jdbc"

topic: "taiyi.ev.charging"

input_fields: ["single_charger_harmonics", "charger_count", "simultaneity_factor",
"device_id"]

window: {type: "sliding", size: 3600, step: 3600}

pipeline:

  • {id: "monte_carlo", type: "wanxiang", operator: "expression_eval", params:
    {expression: "harmonic_summation_monte_carlo($
    {input.single_charger_harmonics}, ${input.charger_count}, $
    {input.simultaneity_factor})", output_as: "total_harmonics_95p"}

  • {id: "get_limit", type: "qianzhi", operator: "query_standard", params:
    {standard_id: "GB/T14549", param_name: "harmonic_current_limit"},
    output_as: "limit"}
```

- {id: "evaluate", type: "builtin", operator: "threshold_rule", params: {operator: ">"}, inputs: {value: "\${monte_carlo.total_harmonics_95p}", threshold: "\${get_limit.limit}"}, output_as: "alert_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {predicted_harmonic_A: "\${monte_carlo.total_harmonics_95p}", limit_A: "\${get_limit.limit}"}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 20}

schedule: {interval: 3600}

...

2. 核心逻辑与伪代码

```python

# Q-17 核心逻辑伪代码

```
class EVChargingHarmonicEstimator:
```

```
def init(self, single_harmonics):
```

```
 self.single_harmonics = single_harmonics
```

```
def monte_carlo_summation(self, n_chargers, simultaneity, n_sim=10000):
```

```
 results = []
```

```
 for _ in range(n_sim):
```

```
 n_active = np.random.binomial(n_chargers, simultaneity)
```

```
 total = 0
```

```
 for _ in range(n_active):
```

```
phase = np.random.uniform(0, 2*np.pi)

total += self.single_harmonics * np.exp(1j * phase)

results.append(abs(total))

return np.percentile(results, 95)

'''
```

3. 编程说明

- **依赖算子**：expression\_eval (内置)、query\_standard (千知)、threshold\_rule (内置)。
- **数据窗口**：1小时滑动窗口，步长1小时。
- **核心算法**：基于单台充电桩的谐波发射特性，采用蒙特卡洛模拟，考虑同时率和相角随机性，计算总谐波电流的95%概率值。
- **告警逻辑**：当预测总谐波电流超过国标限值时告警。

---

Q-18 - 电能质量综合指数(PQI)

1. 模型开发配置单 (YAML)

```
```yaml

model:

model_id: "Q-18"

model_name: "电能质量综合指数(PQI)"

category: "quality"

tech_level: "L1"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"
```


input_fields: ["frequency", "Va_rms", "V_harmonics", "VUF", "Pst", "device_id"]

window: {type: "sliding", size: 86400, step: 3600}

pipeline:

- {id: "get_weights_limits", type: "qianzhi", operator: "query_standard", params: {standard_id: "GB/T_XXXX", param_name: "pq_weights_limits"}, output_as: "config"}
- {id: "calc_sub_scores", type: "builtin", operator: "expression_eval", params: {expression: "{freq_score: max(0, 100 - abs(\${input.frequency}-50)/0.2 * 10), volt_score: max(0, 100 - abs(\${input.Va_rms}-\${context.nominal})/\$ {context.nominal} * 100), thd_score: max(0, 100 - \${input.V_harmonics.THD}/\$ {config.limit_THD} * 100), vuf_score: max(0, 100 - \${input.VUF}/\$ {config.limit_VUF} * 100), flicker_score: max(0, 100 - \${input.Pst}/\$ {config.limit_Pst} * 100)}", output_as: "sub_scores"}
- {id: "calc_pqi", type: "builtin", operator: "expression_eval", params: {expression: "sum([\${sub_scores.freq_score} * \${config.w_freq}, \$ {sub_scores.volt_score} * \${config.w_volt}, \${sub_scores.thd_score} * \$ {config.w_thd}, \${sub_scores.vuf_score} * \${config.w_vuf}, \$ {sub_scores.flicker_score} * \${config.w_flicker}])", output_as: "PQI"}

output:

type: "metric"

level: "info"

topic: "tianyan.metrics.v1"

fields: {PQI: "\${calc_pqi.PQI}", sub_scores: "\${sub_scores}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 3600}

...

2. 核心逻辑与伪代码

```python

## Q-18 核心逻辑伪代码

```
class PowerQualityIndex:

def init(self, weights, limits, nominal_V):

self.weights = weights

self.limits = limits

self.nominal_V = nominal_V

def calculate_pqi(self, freq, V_rms, THD, VUF, Pst):
```

### 各子项得分（100分制，超限扣分）

```
freq_score = max(0, 100 - abs(freq - 50) / 0.2 * 10)

volt_score = max(0, 100 - abs(V_rms - self.nominal_V) / self.nominal_V * 100)

thd_score = max(0, 100 - THD / self.limits['THD'] * 100)

vuf_score = max(0, 100 - VUF / self.limits['VUF'] * 100)

flicker_score = max(0, 100 - Pst / self.limits['Pst'] * 100)
```

### 加权综合

```
PQI = (freq_score * self.weights['freq'] +

volt_score * self.weights['volt'] +

thd_score * self.weights['thd'] +

vuf_score * self.weights['vuf'] +

flicker_score * self.weights['flicker'])

return PQI

...
```

### 3. 编程说明

- **依赖算子**：query\_standard (千知)、expression\_eval (内置)。
- **数据窗口**：24小时滑动窗口，步长1小时。
- **核心算法**：综合频率、电压偏差、谐波畸变率、电压不平衡度、闪变等核心指标，根据国标限值采用加权扣分法计算综合指数。
- **输出**：每小时输出一次PQI（0-100）及各子项得分。

---

## 板块三：用能分析（15个模型）

### E-01 - 非侵入式负荷分解(NILM)

#### 1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "E-01"
```

```
model_name: "非侵入式负荷分解(NILM)"
```

```
category: "energy"
```

```
tech_level: "L2b"
```

```
version: "1.0.0"
```

```
data_source:
```

```
type: "kafka"
```

```
topic: "taiyi.electrical.feature.v1"
```

```
input_fields: ["P_total", "Q_total", "I_rms", "V_rms", "I_THD", "I_h3", "I_h5", "I_h7",
"PF", "timestamp", "device_id"]
```

```
window: {type: "event"}
```

```
history_required: "2weeks"
```

pipeline:

- {id: "preprocess\_power", type: "builtin", operator: "outlier\_filter", params: {method: "iqr", factor: 2.5}, inputs: ["\${input.P\_total}"], output\_as: "filtered\_P"}
- {id: "event\_detection", type: "wanxiang", operator: "anomaly\_detection\_cusum", params: {threshold: 50.0, drift: 10.0, min\_event\_duration: 2}, inputs: ["\${preprocess\_power.filtered\_P}"], output\_as: "events"}
- {id: "extract\_steady\_state", type: "builtin", operator: "expression\_eval", params: {expression: "extract\_steady\_segments(\${preprocess\_power.filtered\_P}, \${event\_detection.events}, min\_duration=10)", output\_as: "steady\_segments"}}
- {id: "extract\_fingerprint", type: "builtin", operator: "expression\_eval", params: {expression: "{delta\_P: \${steady\_segments.P\_after} - \${steady\_segments.P\_before}, delta\_Q: \${steady\_segments.Q\_after} - \${steady\_segments.Q\_before}, delta\_I: \${steady\_segments.I\_after} - \${steady\_segments.I\_before}, delta\_THD: \${steady\_segments.THG\_after} - \${steady\_segments.THG\_before}, delta\_h3: \${steady\_segments.h3\_after} - \${steady\_segments.h3\_before}, delta\_h5: \${steady\_segments.h5\_after} - \${steady\_segments.h5\_before}, delta\_h7: \${steady\_segments.h7\_after} - \${steady\_segments.h7\_before}, PF: \${steady\_segments.PF\_after}, duration: \${steady\_segments.duration}}", output\_as: "fingerprint"}}
- {id: "cluster\_appliances", type: "wanxiang", operator: "clustering\_dbscan", params: {eps: 0.15, min\_samples: 5, features: ["delta\_P", "delta\_Q", "delta\_THD", "PF"]}, inputs: ["\${extract\_fingerprint.fingerprint}"], output\_as: "clusters"}
- {id: "classify\_appliance", type: "wanxiang", operator: "classification\_random\_forest", params: {model\_id: "E01\_appliance\_classifier\_v2", classes: ["lighting", "motor\_fixed", "motor\_vfd", "heater", "rectifier", "computer", "ac", "unknown"]}, inputs: ["\${extract\_fingerprint.fingerprint}"], output\_as: "appliance\_type"}}
- {id: "get\_known\_appliances", type: "qianzhi", operator: "query\_device\_param", params: {device\_id: "\${input.device\_id}", param\_name: "known\_appliance\_fingerprints"}, output\_as: "known\_fingerprints"}}
- {id: "match\_known", type: "builtin", operator: "expression\_eval", params: {expression: "match\_fingerprint\_to\_known(\${cluster\_appliances.clusters}, \$

```
{get_known_appliances.known_fingerprints}}"}, output_as:
"matched_appliances"}
```

- {id: "decompose\_power", type: "builtin", operator: "expression\_eval", params: {expression: "niln\_decompose(\${preprocess\_power.filtered\_P}, \${matched\_appliances.matched\_appliances}, \${context.state})"}, output\_as: "power\_breakdown"}
- {id: "accumulate\_energy", type: "builtin", operator: "expression\_eval", params: {expression: "integrate\_power(\${decompose\_power.power\_breakdown}, \${input.timestamp})"}, output\_as: "energy\_breakdown"}
- {id: "calc\_confidence", type: "builtin", operator: "expression\_eval", params: {expression: "calculate\_decomposition\_confidence(\${decompose\_power.power\_breakdown}, \${preprocess\_power.filtered\_P})"}, output\_as: "confidence"}

output:

type: "metric"

level: "info"

topic: "tianyan.metrics.v1"

fields: {total\_power\_w: "\${preprocess\_power.filtered\_P}", breakdown: "\${decompose\_power.power\_breakdown}", energy\_kwh: "\${accumulate\_energy.energy\_breakdown}", confidence\_percent: "\${calc\_confidence.confidence}"}

deployment: {replicas: 1, cpu: "1000m", memory: "2Gi", timeout: 15}

schedule: {interval: 1}

fallback: {enabled: true, condition: "history\_days < 14", operator: "static\_threshold", params: {min\_appliances: 1, default\_confidence: 50}}

...

## 2. 核心逻辑与伪代码

```
```python
```

E-01 核心逻辑伪代码

```
class NILM:

def init(self, classifier, known_fingerprints):

self.classifier = classifier

self.known_fingerprints = known_fingerprints

self.state = {} # 设备运行状态

def detect_events(self, P_series):
```

CUSUM事件检测

```
events = cusum_detection(P_series, threshold=50, drift=10)

return events

def extract_fingerprint(self, segment):
```

提取功率阶跃和谐波变化指纹

```
return {

'delta_P': segment.P_after - segment.P_before,

'delta_Q': segment.Q_after - segment.Q_before,

'delta_THD': segment.THG_after - segment.THG_before,

'PF': segment.PF_after

}

def process(self, data_stream):

events = self.detect_events(data_stream.P_total)

for event in events:

fp = self.extract_fingerprint(event)
```

```
appliance_type = self.classifier.predict(fp)
```

匹配已知设备或创建新设备

```
if appliance_type in self.known_fingerprints:  
    device_name = self.known_fingerprints[appliance_type]  
else:  
    device_name = f"unknown_{appliance_type}"
```

更新状态和功率分解

```
if event.is_on:  
    self.state[device_name] = fp['delta_P']  
else:  
    self.state.pop(device_name, None)
```

输出实时分解结果

```
return self.state  
...
```

3. 编程说明

- **依赖算子**：outlier_filter (内置)、anomaly_detection_cusum (万象)、expression_eval (内置)、clustering_dbscan (万象)、classification_random_forest (万象)、query_device_param (千知)。
 - **数据窗口**：事件触发。
 - **核心算法**：CUSUM检测功率阶跃事件；提取多维电气指纹；DBSCAN聚类识别重复出现的设备；随机森林分类设备类型；基于状态组合进行实时功率分解。
 - **输出**：实时输出各分项负荷功率和累计电量。
-

E-02 - 清洁能源消纳率分析

1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "E-02"
```

```
model_name: "清洁能源消纳率分析"
```

```
category: "energy"
```

```
tech_level: "L1"
```

```
version: "1.0.0"
```

```
data_source:
```

```
type: "kafka"
```

```
topic: "taiyi.energy.feature.v1"
```

```
input_fields: ["P_pv_total", "P_load_total", "device_id"]
```

```
window: {type: "sliding", size: 60, step: 60}
```

```
pipeline:
```

- {id: "calc_self_use", type: "builtin", operator: "expression_eval", params: {expression: "min(\${input.P_pv_total}, \${input.P_load_total})", output_as: "P_self_use"}}
- {id: "calc_export", type: "builtin", operator: "expression_eval", params: {expression: "max(0, \${input.P_pv_total} - \${input.P_load_total})", output_as: "P_export"}}
- {id: "calc_import", type: "builtin", operator: "expression_eval", params: {expression: "max(0, \${input.P_load_total} - \${input.P_pv_total})", output_as: "P_import"}}
- {id: "calc_rate", type: "builtin", operator: "expression_eval", params: {expression: "\${calc_self_use.P_self_use} / (\${input.P_pv_total} + 0.001) * 100", output_as: "self_use_rate"}}


```
output:

type: "metric"

level: "info"

topic: "tianyan.metrics.v1"

fields: {pv_power: "${input.P_pv_total}", self_use_power: "${
{calc_self_use.P_self_use}", export_power: "${calc_export.P_export}", self_use_rate:
"${calc_rate.self_use_rate}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 5}

schedule: {interval: 60}

...
```

2. 核心逻辑与伪代码

```
```python
```

# E-02 核心逻辑伪代码

```
class CleanEnergyConsumptionRate:

def process(self, P_pv, P_load):

P_self_use = min(P_pv, P_load)

P_export = max(0, P_pv - P_load)

P_import = max(0, P_load - P_pv)

self_use_rate = P_self_use / (P_pv + 1e-6) * 100

return P_self_use, P_export, P_import, self_use_rate

...
```

## 3. 编程说明

- 依赖算子：expression\_eval (内置)。

- **数据窗口**：60秒滑动窗口，步长60秒。
  - **核心算法**：基于能量平衡，取光伏发电和负荷用电的较小值为自发自用功率，差值即为余电上网或市电补充。
  - **输出**：每分钟输出自发自用功率、上网功率和自消纳率。
- 

## E-03 - 光储协同优化策略

### 1. 模型开发配置单 (YAML)

```yaml

model:

model_id: "E-03"

model_name: "光储协同优化策略"

category: "energy"

tech_level: "L2b"

version: "1.0.0"

data_source:

type: "jdbc"

topic: "taiyi.forecast.v1"

input_fields: ["pv_forecast_24h", "load_forecast_24h", "tou_price", "device_id"]

window: {type: "sliding", size: 86400, step: 900}

history_required: "1month"

pipeline:

- {id: "get_battery_param", type: "qianzhi", operator: "query_device_param", params: {param_name: "battery_spec"}, output_as: "battery"}
- {id: "mpc_optimize", type: "wanxiang", operator: "expression_eval", params: {expression: "model_predictive_control(\${input.pv_forecast_24h}, \$

```

    {input.load_forecast_24h}, ${input.tou_price}, $
    {get_battery_param.battery}}"), output_as: "schedule"}

    • {id: "calc_saving", type: "builtin", operator: "expression_eval", params:
      {expression: "estimate_saving(${mpc_optimize.schedule}, $
        {input.tou_price})"}, output_as: "estimated_saving"}

output:

type: "report"

level: "info"

topic: "tianyan.reports.v1"

fields: {schedule: "${mpc_optimize.schedule}", estimated_saving_yuan: "$
{calc_saving.estimated_saving}"}

deployment: {replicas: 1, cpu: "1000m", memory: "2Gi", timeout: 60}

schedule: {interval: 900}

fallback: {enabled: true, condition: "forecast_unavailable", operator: "rule_based",
  params: {strategy: "peak_shaving"}}

...

```

2. 核心逻辑与伪代码

```
```python
```

## E-03 核心逻辑伪代码

```

class PVStorageOptimizer:

 def init(self, battery_capacity, charge_eff, discharge_eff):

 self.battery = {'cap': battery_capacity, 'eta_c': charge_eff, 'eta_d': discharge_eff}

 def optimize(self, pv_forecast, load_forecast, price):

```

# 模型预测控制(MPC)，求解最优充放电序列

## 目标：最小化电费或最大化收益

`N = 96 # 15分钟间隔，24小时`

`def objective(x):`

`cost = 0`

`soc = 0.5 # 初始SOC`

`for t in range(N):`

`net_load = load_forecast[t] - pv_forecast[t] - x[t]`

`if net_load > 0: # 放电`

`actual_discharge = min(x[t], soc * self.battery['cap'])`

`cost += net_load * price[t] + actual_discharge * (1/self.battery['eta_d'] - 1) * price[t]`

`soc -= actual_discharge / self.battery['cap']`

`else: # 充电`

`actual_charge = min(-net_load, (1-soc) * self.battery['cap'])`

`cost += actual_charge * (1 - self.battery['eta_c']) * price[t]`

`soc += actual_charge / self.battery['cap']`

`return cost`

## 使用动态规划或遗传算法求解

`optimal_x = self.solve(objective)`

`return optimal_x`

`...`

### 3. 编程说明

- **依赖算子**：query\_device\_param (千知)、expression\_eval (内置)。
  - **数据窗口**：24小时滑动窗口，步长15分钟。
  - **核心算法**：基于光伏预测、负荷预测和分时电价，采用模型预测控制（MPC）或动态规划（DP）求解储能最优充放电计划。
  - **输出**：每15分钟输出未来24小时的建议充放电功率。
- 

## E-04 - 最大需量预测与预警

### 1. 模型开发配置单 (YAML)

```
``yaml
model:

model_id: "E-04"

model_name: "最大需量预测与预警"

category: "energy"

tech_level: "L2a"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.energy.feature.v1"

input_fields: ["P_total", "device_id"]

window: {type: "sliding", size: 900, step: 60}

pipeline:

 • {id: "calc_current_demand", type: "builtin", operator: "expression_eval",
 params: {expression: "moving_average(${input.P_total}, 900)", output_as:
 "current_demand"}}
```

- {id: "lstm\_forecast", type: "wanxiang", operator: "forecast\_lstm", params: {horizon: 15, model\_path: "/models/E-04\_lstm.h5"}, inputs: ["\${input.P\_total}"], output\_as: "forecast"}
- {id: "get\_contract", type: "qianzhi", operator: "query\_device\_param", params: {param\_name: "contract\_demand"}, output\_as: "contract"}
- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: ">"}, inputs: {value: "\${lstm\_forecast.forecast}", threshold: "\${get\_contract.contract} \* 0.95"}, output\_as: "alert\_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {current\_demand: "\${calc\_current\_demand.current\_demand}",  
forecast\_demand: "\${lstm\_forecast.forecast}", contract\_demand: "\${  
get\_contract.contract}"}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 10}

schedule: {interval: 60}

...

## 2. 核心逻辑与伪代码

```python

E-04 核心逻辑伪代码

```
class DemandPredictor:
```

```
def init(self, contract_demand, model):
```

```
    self.contract = contract_demand
```

```
    self.model = model
```

```
    self.history = deque(maxlen=900)
```

```
def process(self, P):

self.history.append(P)

current_demand = np.mean(self.history)

if len(self.history) == 900:

seq = np.array(self.history).reshape(1, -1, 1)

forecast = self.model.predict(seq)[0, 0]

if forecast > self.contract * 0.95:

self.trigger_alert(current_demand, forecast)

...
```

3. 编程说明

- **依赖算子**：expression_eval (内置)、forecast_lstm (万象)、query_device_param (千知)、threshold_rule (内置)。
- **数据窗口**：15分钟滑动窗口，步长1分钟。
- **核心算法**：计算当前滑动窗口平均功率作为当前需量。使用LSTM模型预测未来15分钟的最大需量。
- **告警逻辑**：当预测需量超过契约容量的95%时发出预警。

E-05 - 变压器经济运行分析

1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "E-05"

model_name: "变压器经济运行分析"

category: "energy"

tech_level: "L1"
```

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.transformer.v1"

input_fields: ["P1", "P2", "S1 Rated", "S2 Rated", "P0_1", "Pk_1", "P0_2", "Pk_2",
"device_id"]

window: {type: "sliding", size: 600, step: 600}

pipeline:

- {id: "calc_load_ratio", type: "builtin", operator: "expression_eval", params: {expression: "(\${input.P1} + \${input.P2}) / (\${input.S1_Rated} + \${input.S2_Rated})", output_as: "load_ratio"}
- {id: "calc_loss_single", type: "builtin", operator: "expression_eval", params: {expression: "\${input.P0_1} + (\${calc_load_ratio.load_ratio})^2 * \${input.Pk_1}", output_as: "loss_single"}
- {id: "calc_loss_parallel", type: "builtin", operator: "expression_eval", params: {expression: "\${input.P0_1} + \${input.P0_2} + (\${calc_load_ratio.load_ratio}/2)^2 * (\${input.Pk_1} + \${input.Pk_2})", output_as: "loss_parallel"}
- {id: "recommend", type: "builtin", operator: "expression_eval", params: {expression: "\${calc_loss_single.loss_single} < \${calc_loss_parallel.loss_parallel} ? 'single' : 'parallel'", output_as: "recommendation"}}

output:

type: "report"

level: "info"

topic: "tianyan.reports.v1"

fields: {load_ratio: "\${calc_load_ratio.load_ratio}", recommend_mode: "\${recommend.recommendation}", loss_saving_kw: "\${abs(calc_loss_single.loss_single - calc_loss_parallel.loss_parallel)}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}


```
schedule: {interval: 600}
```

```
...
```

2. 核心逻辑与伪代码

```
```python
```

# E-05 核心逻辑伪代码

```
class TransformerEconomicOperation:
```

```
def init(self, trafo1_params, trafo2_params):
```

```
self.P0_1, self.Pk_1 = trafo1_params
```

```
self.P0_2, self.Pk_2 = trafo2_params
```

```
self.S1_rated, self.S2_rated = trafo1_params['S'], trafo2_params['S']
```

```
def process(self, P1, P2):
```

```
P_total = P1 + P2
```

```
load_ratio = P_total / (self.S1_rated + self.S2_rated)
```

## 单台运行损耗

```
loss_single = self.P0_1 + (load_ratio ** 2) * self.Pk_1
```

## 并列运行损耗

```
loss_parallel = self.P0_1 + self.P0_2 + (load_ratio / 2) ** 2 * (self.Pk_1 + self.Pk_2)
```

```
if loss_single < loss_parallel:
```

```
recommend = 'single'
```

```
else:
```

```
recommend = 'parallel'
```

```
return load_ratio, recommend, abs(loss_single - loss_parallel)
...
```

### 3. 编程说明

- **依赖算子**：expression\_eval (内置)。
  - **数据窗口**：10分钟滑动窗口，步长10分钟。
  - **核心算法**：基于变压器综合损耗公式  $\Delta P = P_0 + \beta^2 P_k$ ，比较单台运行和并列运行的总损耗。
  - **输出**：每10分钟输出建议运行方式（单台/并列）和预计节省损耗。
- 

## E-06 - 峰谷平电量结构分析

### 1. 模型开发配置单 (YAML)

```
``yaml
model:

model_id: "E-06"

model_name: "峰谷平电量结构分析"

category: "energy"

tech_level: "L1"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.energy.feature.v1"

input_fields: ["P_total", "device_id", "timestamp"]

window: {type: "sliding", size: 2592000, step: 86400}
```

pipeline:

- {id: "get\_tou\_periods", type: "qianzhi", operator: "query\_device\_param", params: {param\_name: "tou\_schedule"}, output\_as: "tou"}
- {id: "classify\_power", type: "builtin", operator: "expression\_eval", params: {expression: "map\_to\_tou\_period(\${input.timestamp}, \${get\_tou\_periods.tou})", output\_as: "period"}}
- {id: "accumulate\_energy", type: "builtin", operator: "expression\_eval", params: {expression: "group\_sum(\${input.P\_total}, \${classify\_power.period})", output\_as: "energy\_by\_period"}}
- {id: "calc\_cost", type: "builtin", operator: "expression\_eval", params: {expression: "\${accumulate\_energy.energy\_by\_period} \* \${get\_tou\_periods.tou.price}", output\_as: "cost\_by\_period"}}

output:

type: "report"

level: "info"

topic: "tianyan.reports.v1"

fields: {peak\_energy: "\${accumulate\_energy.energy\_by\_period.peak}", flat\_energy: "\${accumulate\_energy.energy\_by\_period.flat}", valley\_energy: "\${accumulate\_energy.energy\_by\_period.valley}", peak\_cost: "\${calc\_cost.cost\_by\_period.peak}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 30}

schedule: {interval: 86400}

...

## 2. 核心逻辑与伪代码

```python

E-06 核心逻辑伪代码

class TOUEnergyAnalyzer:

```

def init(self, tou_schedule):

self.tou = tou_schedule

def process_daily(self, P_array, timestamps):

energy = {'peak': 0, 'flat': 0, 'valley': 0}

cost = {'peak': 0, 'flat': 0, 'valley': 0}

for P, ts in zip(P_array, timestamps):

period = self.get_period(ts)

energy[period] += P / 3600 # kWh

cost[period] += P / 3600 * self.tou[period]['price']

return energy, cost

...

```

3. 编程说明

- **依赖算子**：query_device_param (千知)、expression_eval (内置)。
- **数据窗口**：30天滑动窗口，步长1天。
- **核心算法**：根据峰谷平时段划分，对秒级功率进行分段积分，统计各时段用电量和电费。
- **输出**：每日输出峰、平、谷电量及电费。

E-07 - 待机功耗/基础负荷挖掘

1. 模型开发配置单 (YAML)

```

``yaml

model:

model_id: "E-07"

model_name: "待机功耗/基础负荷挖掘"

```

category: "energy"

tech_level: "L2a"

version: "1.0.0"

data_source:

type: "jdbc"

topic: "taiyi.energy.history"

input_fields: ["P_total", "device_id"]

window: {type: "sliding", size: 604800, step: 86400}

history_required: "4weeks"

pipeline:

- {id: "kde_analysis", type: "wanxiang", operator: "kde_density", params: {bandwidth: 0.5}, inputs: ["\${input.P_total}"], output_as: "density"}
- {id: "find_base_load", type: "builtin", operator: "expression_eval", params: {expression: "find_first_peak(\${kde_analysis.density})"}, output_as: "base_load"}
- {id: "calc_waste", type: "builtin", operator: "expression_eval", params: {expression: "\${find_base_load.base_load} * 24 * 365"}, output_as: "annual_waste"}
- {id: "evaluate", type: "builtin", operator: "threshold_rule", params: {operator: ">"}, inputs: {value: "\${find_base_load.base_load}", threshold: 50}, output_as: "alert_trigger"}

output:

type: "alert"

level: "info"

topic: "tianyan.alerts.v1"

fields: {base_load_kw: "\${find_base_load.base_load}", annual_waste_kwh: "\${calc_waste.annual_waste}"}

```
deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 30}

schedule: {interval: 86400}

...
```

2. 核心逻辑与伪代码

```
```python
```

# E-07 核心逻辑伪代码

```
class BaseLoadMiner:

 def process_weekly(self, P_history):
```

## 核密度估计

```
kde = KernelDensity(bandwidth=0.5)

kde.fit(P_history.reshape(-1, 1))

x = np.linspace(min(P_history), max(P_history), 1000)

density = np.exp(kde.score_samples(x.reshape(-1, 1)))
```

## 第一个峰值即为基础负荷

```
peaks = find_peaks(density)[0]

if peaks:

 base_load = x[peaks[0]]

 annual_waste = base_load * 24 * 365

 if base_load > 50:

 self.trigger_alert(base_load, annual_waste)

...
```

### 3. 编程说明

- **依赖算子**：kde\_density (万象)、expression\_eval (内置)、threshold\_rule (内置)。
  - **数据窗口**：7天滑动窗口，步长1天。
  - **核心算法**：利用核密度估计（KDE）分析历史负荷的概率分布，第一个显著峰值对应的功率即为系统基础负荷。
  - **告警逻辑**：当基础负荷超过50kW时，提示可能存在待机浪费。
- 

## E-08 - 用电行为聚类分析

### 1. 模型开发配置单 (YAML)

```
``yaml
model:
 model_id: "E-08"
 model_name: "用电行为聚类分析"
 category: "energy"
 tech_level: "L2b"
 version: "1.0.0"
 data_source:
 type: "jdbc"
 topic: "taiyi.energy.history"
 input_fields: ["load_curve_daily", "device_id"]
 window: {type: "sliding", size: 2592000, step: 2592000}
 history_required: "3months"
```

pipeline:

- {id: "normalize", type: "builtin", operator: "expression\_eval", params: {expression: "min\_max\_normalize(\${input.load\_curve\_daily})", output\_as: "norm\_curve"}}
- {id: "kshape\_cluster", type: "wanxiang", operator: "clustering\_dbscan", params: {eps: 0.1, min\_samples: 5}, inputs: ["\${normalize.norm\_curve}"], output\_as: "clusters"}
- {id: "extract\_pattern", type: "builtin", operator: "expression\_eval", params: {expression: "centroid(\${kshape\_cluster.clusters})", output\_as: "patterns"}}
- {id: "get\_benchmark", type: "qianzhi", operator: "query\_standard", params: {standard\_id: "INDUSTRY\_BENCHMARK", param\_name: "typical\_load\_pattern"}, output\_as: "benchmark"}
- {id: "compare", type: "builtin", operator: "expression\_eval", params: {expression: "dtw\_distance(\${extract\_pattern.patterns}, \${get\_benchmark.benchmark})", output\_as: "deviation"}}

output:

type: "report"

level: "info"

topic: "tianyan.reports.v1"

fields: {pattern\_count: "\${kshape\_cluster.clusters.n\_clusters}", patterns: "\${extract\_pattern.patterns}", industry\_deviation: "\${compare.deviation}"}

deployment: {replicas: 1, cpu: "1000m", memory: "2Gi", timeout: 60}

schedule: {interval: 2592000}

...

## 2. 核心逻辑与伪代码

```python


E-08 核心逻辑伪代码

```
class LoadPatternClusterer:

def process_monthly(self, daily_curves):
```

归一化

```
normalized = [(c - min(c)) / (max(c) - min(c)) for c in daily_curves]
```

K-Shape聚类

```
clusters = kshape_clustering(normalized, n_clusters='auto')

patterns = [np.mean(cluster, axis=0) for cluster in clusters]

return clusters, patterns

'''
```

3. 编程说明

- **依赖算子**：expression_eval (内置)、clustering_dbscan (万象，实际为K-Shape)。
- **数据窗口**：30天滑动窗口，步长30天。
- **核心算法**：对日负荷曲线进行归一化，采用K-Shape时序聚类算法，自动识别不同的用电模式（工作日、周末、节假日等）。
- **输出**：每月输出聚类结果和各模式的典型曲线，并与行业基准对比。

E-09 - 碳足迹实时追踪

1. 模型开发配置单 (YAML)

```
```yaml

model:
```

model\_id: "E-09"

model\_name: "碳足迹实时追踪"

category: "energy"

tech\_level: "L1"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.energy.feature.v1"

input\_fields: ["P\_grid", "P\_pv", "device\_id"]

window: {type: "sliding", size: 60, step: 60}

pipeline:

- {id: "get\_emission\_factor", type: "qianzhi", operator: "query\_standard", params: {standard\_id: "MEE\_2024", param\_name: "grid\_emission\_factor"}, output\_as: "ef"}
- {id: "calc\_grid\_emission", type: "builtin", operator: "expression\_eval", params: {expression: "\${input.P\_grid} \* \${get\_emission\_factor.ef}"}, output\_as: "grid\_co2"}
- {id: "calc\_pv\_offset", type: "builtin", operator: "expression\_eval", params: {expression: "\${input.P\_pv} \* \${get\_emission\_factor.ef}"}, output\_as: "pv\_offset"}
- {id: "calc\_net\_emission", type: "builtin", operator: "expression\_eval", params: {expression: "\${calc\_grid\_emission.grid\_co2} - \${calc\_pv\_offset.pv\_offset}"}, output\_as: "net\_co2"}
- {id: "accumulate", type: "builtin", operator: "expression\_eval", params: {expression: "cumulative\_sum(\${calc\_net\_emission.net\_co2})"}, output\_as: "total\_co2"}

output:

type: "metric"

```
level: "info"
```

```
topic: "tianya.metrics.v1"
```

```
fields: {co2_rate_kgh: "${calc_net_emission.net_co2}", total_co2_kg: "${accumulate.total_co2}", pv_offset_kg: "${calc_pv_offset.pv_offset}"}
```

```
deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 5}
```

```
schedule: {interval: 60}
```

```
...
```

## 2. 核心逻辑与伪代码

```
```python
```

E-09 核心逻辑伪代码

```
class CarbonFootprintTracker:
```

```
def init(self, emission_factor):
```

```
    self.ef = emission_factor
```

```
    self.total_co2 = 0
```

```
def process(self, P_grid, P_pv):
```

```
    grid_co2 = P_grid * self.ef / 1000 # kg/h
```

```
    pv_offset = P_pv * self.ef / 1000 # kg/h
```

```
    net_co2 = grid_co2 - pv_offset
```

```
    self.total_co2 += net_co2 / 3600 # 积分
```

```
    return net_co2, self.total_co2, pv_offset
```

```
...
```

3. 编程说明

- **依赖算子**：query_standard (千知)、expression_eval (内置)。

- **数据窗口**：60秒滑动窗口，步长60秒。
 - **核心算法**：根据电网排放因子，将市电用电量折算为碳排放，光伏发电量折算为减排量，实时累加。
 - **输出**：每分钟输出碳排放速率、累计碳排放和光伏减排贡献。
-

E-10 - 虚拟电厂(VPP)可调容量评估

1. 模型开发配置单 (YAML)

```yaml

model:

model\_id: "E-10"

model\_name: "虚拟电厂(VPP)可调容量评估"

category: "energy"

tech\_level: "L2a"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.vpp.v1"

input\_fields: ["flexible\_loads\_power", "battery\_soc", "battery\_power", "device\_id"]

window: {type: "sliding", size: 60, step: 60}

pipeline:

- {id: "get\_constraints", type: "qianzhi", operator: "query\_device\_param", params: {param\_name: "load\_flexibility\_constraints"}, output\_as: "constraints"}
- {id: "calc\_sheddable\_load", type: "builtin", operator: "expression\_eval", params: {expression: "sum(\${input.flexible\_loads\_power} \* \${get\_constraints.constraints.shed\_ratio})", output\_as: "shedddable"}}

- {id: "calc\_battery\_capacity", type: "builtin", operator: "expression\_eval", params: {expression: "min(\${input.battery\_power}, (\${input.battery\_soc} - 0.2) \* \${get\_constraints.constraints.battery\_capacity})", output\_as: "battery\_available"}}
- {id: "calc\_total\_vpp", type: "builtin", operator: "expression\_eval", params: {expression: "\${calc\_sheddable\_load.sheddable} + \${calc\_battery\_capacity.battery\_available}", output\_as: "vpp\_capacity"}}

output:

type: "metric"

level: "info"

topic: "tianyan.metrics.v1"

fields: {vpp\_capacity\_kw: "\${calc\_total\_vpp.vpp\_capacity}", sheddable\_load\_kw: "\${calc\_sheddable\_load.sheddable}", battery\_available\_kw: "\${calc\_battery\_capacity.battery\_available}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 60}

...

## 2. 核心逻辑与伪代码

```python

E-10 核心逻辑伪代码

```
class VPPCapacityEstimator:
```

```
def init(self, constraints, battery_capacity):
```

```
    self.constraints = constraints
```

```
    self.battery_capacity = battery_capacity
```

```
def process(self, flexible_loads, battery_soc, battery_power):
```

可削减负荷

```
sheddable = sum([P * ratio for P, ratio in zip(flexible_loads,
self.constraints['shed_ratio'])])
```

储能可用容量 (保留20% SOC)

```
battery_available = min(battery_power, (battery_soc - 0.2) * self.battery_capacity)

total_vpp = sheddable + battery_available

return total_vpp, sheddable, battery_available

...
```

3. 编程说明

- **依赖算子**：query_device_param (千知)、expression_eval (内置)。
- **数据窗口**：60秒滑动窗口，步长60秒。
- **核心算法**：根据各可中断负荷的实时功率和可削减比例，计算可削减容量。根据储能SOC和功率限值计算可用容量。
- **输出**：每分钟输出VPP总可调容量。

E-11 - 绿电溯源与消纳证明

1. 模型开发配置单 (YAML)

```
```yaml

model:

model_id: "E-11"

model_name: "绿电溯源与消纳证明"

category: "energy"

tech_level: "L1"
```

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.energy.feature.v1"

input\_fields: ["P\_pv", "P\_load", "timestamp", "device\_id"]

window: {type: "sliding", size: 3600, step: 3600}

pipeline:

- {id: "match\_green", type: "builtin", operator: "expression\_eval", params: {expression: "min(\${input.P\_pv}, \${input.P\_load})", output\_as: "green\_energy"}}
- {id: "calc\_coverage", type: "builtin", operator: "expression\_eval", params: {expression: "\${match\_green.green\_energy} / (\${input.P\_load} + 0.001) \* 100", output\_as: "green\_ratio"}}
- {id: "generate\_proof", type: "builtin", operator: "expression\_eval", params: {expression: "create\_signed\_record(\${match\_green.green\_energy}, \${calc\_coverage.green\_ratio}, \${input.timestamp})", output\_as: "proof"}}
- {id: "evaluate\_100pct", type: "builtin", operator: "expression\_eval", params: {expression: "\${calc\_coverage.green\_ratio} >= 99.9", output\_as: "is\_100pct\_green"}}

output:

type: "report"

level: "info"

topic: "tianyan.reports.v1"

fields: {green\_energy\_kwh: "\${match\_green.green\_energy}", green\_ratio: "\${calc\_coverage.green\_ratio}", is\_100pct\_green: "\${evaluate\_100pct.is\_100pct\_green}", proof\_hash: "\${generate\_proof.proof.hash}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 3600}

...

## 2. 核心逻辑与伪代码

```python

E-11 核心逻辑伪代码

```
class GreenPowerTracer:
```

```
def process_hourly(self, P_pv_array, P_load_array):
```

```
    green_energy = sum([min(pv, load) for pv, load in zip(P_pv_array, P_load_array)])
```

```
    total_load = sum(P_load_array)
```

```
    green_ratio = green_energy / total_load * 100 if total_load > 0 else 0
```

```
    is_100pct = green_ratio >= 99.9
```

生成数字签名记录

```
    proof = self.create_signed_record(green_energy, green_ratio)
```

```
    return green_energy, green_ratio, is_100pct, proof
```

...

3. 编程说明

- **依赖算子**：expression_eval (内置)。
 - **数据窗口**：1小时滑动窗口，步长1小时。
 - **核心算法**：对每个采样点，取光伏发电和负荷用电的较小值作为绿电消纳量。累计得到小时级绿电消纳量和占比。
 - **输出**：每小时输出绿电消纳量、绿电占比，并生成可审计的数字证明。
-

E-12 - 储能电池健康度(SOH)在线估计

1. 模型开发配置单 (YAML)

```yaml

model:

model\_id: "E-12"

model\_name: "储能电池健康度(SOH)在线估计"

category: "energy"

tech\_level: "L2b"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.battery.v1"

input\_fields: ["voltage", "current", "soc", "temperature", "cycle\_count", "device\_id"]

window: {type: "event"}

history\_required: "3months"

pipeline:

- {id: "get\_battery\_spec", type: "qianzhi", operator: "query\_device\_param", params: {param\_name: "battery\_nominal\_capacity"}, output\_as: "spec"}
- {id: "ekf\_estimate", type: "wanxiang", operator: "expression\_eval", params: {expression: "extended\_kalman\_filter\_soh(\${input.voltage}, \${input.current}, \${input.soc}, \${input.temperature})", output\_as: "soh\_estimate"}}
- {id: "trend\_filter", type: "wanxiang", operator: "lowess\_filter", params: {frac: 0.05, inputs: ["\${ekf\_estimate.soh\_estimate}"], output\_as: "soh\_trend"}}
- {id: "calc\_remaining\_cycles", type: "builtin", operator: "expression\_eval", params: {expression: "(\${soh\_trend.soh\_trend} - 0.7) / \${context.degradation\_rate}", output\_as: "remaining\_cycles"}}

- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: "<"}, inputs: {value: "\${soh\_trend.soh\_trend}", threshold: 0.75}, output\_as: "alert\_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianya.alerts.v1"

fields: {soh\_percent: "\${soh\_trend.soh\_trend \* 100}", remaining\_cycles\_est: "\${calc\_remaining\_cycles.remaining\_cycles}", current\_cycles: "\${input.cycle\_count}"}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 15}

schedule: {interval: 1}

fallback: {enabled: true, condition: "history\_days < 90", operator: "coulomb\_counting"}

...

## 2. 核心逻辑与伪代码

```python

E-12 核心逻辑伪代码

```
class BatterySOHEstimator:
```

```
    def init(self, nominal_capacity):
```

```
        self.C_nom = nominal_capacity
```

```
        self.soh_history = []
```

```
    def ekf_update(self, V, I, SOC, T):
```

扩展卡尔曼滤波估计内阻和容量

状态向量: [SOC, R0, C_actual]

测量方程: $V = OCV(SOC) - I * R0$

... EKF实现细节 ...

```
C_actual = self.ekf.state[2]

SOH = C_actual / self.C_nom

return SOH

def process_cycle(self, cycle_data):

    SOH = self.ekf_update(cycle_data.V, cycle_data.I, cycle_data.SOC, cycle_data.T)

    self.soh_history.append(SOH)

    if len(self.soh_history) > 30:

        trend = lowess(self.soh_history, frac=0.05)

        current_soh = trend[-1]

        degradation_rate = (trend[0] - trend[-1]) / len(trend)

        remaining_cycles = (current_soh - 0.7) / degradation_rate if degradation_rate > 0
        else float('inf')

        if current_soh < 0.75:

            self.trigger_alert(current_soh, remaining_cycles)

    ...
```

3. 编程说明

- **依赖算子**：query_device_param (千知)、expression_eval (内置)、lowess_filter (万象)、threshold_rule (内置)。

- **数据要求**：至少3个月的充放电循环数据。
 - **核心算法**：采用扩展卡尔曼滤波（EKF），基于等效电路模型在线辨识电池实际可用容量， $SOH = C_{actual} / C_{nominal}$ 。
 - **告警逻辑**：当SOH低于75%时告警，并估计剩余循环次数。
-

E-13 - 光伏组串衰减与阴影遮挡检测

1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "E-13"
```

```
model_name: "光伏组串衰减与阴影遮挡检测"
```

```
category: "energy"
```

```
tech_level: "L2a"
```

```
version: "1.0.0"
```

```
data_source:
```

```
type: "kafka"
```

```
topic: "taiyi.pv.string.v1"
```

```
input_fields: ["I_strings", "irradiance", "temperature", "device_id"]
```

```
window: {type: "sliding", size: 600, step: 600}
```

```
pipeline:
```

- {id: "normalize_current", type: "builtin", operator: "expression_eval", params: {expression: "\${input.I_strings} / \${input.irradiance}"}, output_as: "normalized_I"}
- {id: "iqr_outlier", type: "builtin", operator: "outlier_filter", params: {method: "iqr", factor: 1.5}, inputs: ["\${normalize_current.normalized_I}"], output_as: "outliers"}

- {id: "identify_string", type: "builtin", operator: "expression_eval", params: {expression: "argmin(\${normalize_current.normalized_I})", output_as: "worst_string"}}
- {id: "calc_deviation", type: "builtin", operator: "expression_eval", params: {expression: "(median(\${normalize_current.normalized_I}) - min(\${normalize_current.normalized_I})) / median(\${normalize_current.normalized_I})", output_as: "deviation"}}
- {id: "evaluate", type: "builtin", operator: "threshold_rule", params: {operator: ">", duration: 1800}, inputs: {value: "\${calc_deviation.deviation}", threshold: 0.08}, output_as: "alert_trigger"}

output:

type: "alert"

level: "warning"

topic: "tianyan.alerts.v1"

fields: {worst_string_id: "\${identify_string.worst_string}", deviation_percent: "\${calc_deviation.deviation * 100}", suspected_cause: "\${calc_deviation.deviation > 0.15 ? 'shading' : 'degradation'"}}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 600}

...

2. 核心逻辑与伪代码

```python

## E-13 核心逻辑伪代码

```
class PVStringAnomalyDetector:
```

```
def process(self, I_strings, irradiance):
```

# 归一化电流 ( $A / (W/m^2)$ )

```
norm_I = [I / (irr + 1e-6) for I, irr in zip(I_strings, irradiance)]
```

## 离群点检测

```
median_I = np.median(norm_I)

worst_idx = np.argmin(norm_I)

deviation = (median_I - norm_I[worst_idx]) / median_I

if deviation > 0.08:

 cause = 'shading' if deviation > 0.15 else 'degradation'

 self.trigger_alert(worst_idx, deviation, cause)

...
```

### 3. 编程说明

- **依赖算子**：expression\_eval (内置)、outlier\_filter (内置)、threshold\_rule (内置)。
- **数据窗口**：10分钟滑动窗口，步长10分钟。
- **核心算法**：用辐照度归一化各组串电流，消除光照影响。计算中位数和最小值的偏差，识别落后组串。
- **告警逻辑**：当偏差>8%持续30分钟告警，偏差>15%判定为遮挡，否则判定为衰减。

---

## E-14 - 园区级多能互补调度

### 1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

model\_id: "E-14"

model\_name: "园区级多能互补调度"

category: "energy"

tech\_level: "L3"

version: "1.0.0"

data\_source:

type: "jdbc"

topic: "taiyi.multi\_energy.v1"

input\_fields: ["elec\_load\_forecast", "heat\_load\_forecast", "cool\_load\_forecast",  
"elec\_price\_forecast", "gas\_price", "device\_id"]

window: {type: "sliding", size: 86400, step: 900}

history\_required: "1month"

pipeline:

- {id: "get\_equipment\_params", type: "qianzhi", operator:  
"query\_device\_param", params: {param\_name: "energy\_equipment\_specs"},  
output\_as: "equipment"}
- {id: "milp\_optimize", type: "wanxiang", operator: "expression\_eval", params:  
{expression: "mixed\_integer\_linear\_programming(\${input}, \$  
{get\_equipment\_params.equipment})", output\_as: "schedule"}
- {id: "calc\_metrics", type: "builtin", operator: "expression\_eval", params:  
{expression: "{cost: total\_cost(\${milp\_optimize.schedule}), carbon:  
total\_carbon(\${milp\_optimize.schedule}), efficiency: overall\_efficiency(\$  
{milp\_optimize.schedule})}", output\_as: "metrics"}

output:

type: "report"

level: "info"

topic: "tianya.reports.v1"

```
fields: {schedule: "${milp_optimize.schedule}", total_cost_yuan: "${calc_metrics.metrics.cost}", total_carbon_kg: "${calc_metrics.metrics.carbon}", efficiency: "${calc_metrics.metrics.efficiency}"}
```

```
deployment: {replicas: 1, cpu: "2000m", memory: "4Gi", timeout: 120}
```

```
schedule: {interval: 900}
```

```
fallback: {enabled: true, condition: "solver_timeout", operator: "rule_based",
params: {priority: "heat_lead"}}
```

```
...
```

## 2. 核心逻辑与伪代码

```
```python
```

E-14 核心逻辑伪代码

```
class MultiEnergyScheduler:
```

```
def init(self, equipment):
```

```
self.equipment = equipment
```

```
def optimize(self, forecasts, prices):
```

构建混合整数线性规划模型

```
model = Model("MultiEnergy")
```


决策变量：各设备出力

约束：能量平衡、设备容量、爬坡率等

目标：min 总运行成本 or min 总碳排放

使用求解器求解

```
solver = SolverFactory('cbc')  
  
result = solver.solve(model)  
  
schedule = self.extract_schedule(model)  
  
cost = model.objective()  
  
return schedule, cost  
...
```

3. 编程说明

- **依赖算子**：query_device_param (千知)、expression_eval (内置)。
- **数据窗口**：24小时滑动窗口，步长15分钟。
- **核心算法**：建立包含电、热、冷多种能源设备的混合整数线性规划（MILP）模型，求解最优运行策略。
- **输出**：每15分钟输出各设备的启停计划和出力曲线。

E-15 - 碳资产管理与交易策略

1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

model_id: "E-15"

model_name: "碳资产管理与交易策略"

category: "energy"

tech_level: "L2b"

version: "1.0.0"

data_source:

type: "jdbc"

topic: "taiyi.carbon.v1"

input_fields: ["quota_balance", "actual_emission", "carbon_price_history",
"device_id"]

window: {type: "sliding", size: 2592000, step: 86400}

history_required: "12months"

pipeline:

- {id: "forecast_emission", type: "wanxiang", operator: "forecast_prophet",
params: {periods: 30}, inputs: ["\${input.actual_emission}"], output_as:
"emission_forecast"}
- {id: "forecast_price", type: "wanxiang", operator: "forecast_lstm", params:
{horizon: 30}, inputs: ["\${input.carbon_price_history}"], output_as:
"price_forecast"}
- {id: "calc_position", type: "builtin", operator: "expression_eval", params:
{expression: "\${input.quota_balance} - sum(\$
{forecast_emission.emission_forecast})"}, output_as: "expected_surplus"}
- {id: "sdp_strategy", type: "wanxiang", operator: "expression_eval", params:
{expression: "stochastic_dynamic_programming(\${expected_surplus}, \$
{forecast_price})"}, output_as: "trade_strategy"}
- {id: "evaluate", type: "builtin", operator: "expression_eval", params:
{expression: "abs(\${expected_surplus}) > \${context.quota_buffer}"},
output_as: "trade_recommended"}

output:

```
type: "report"
```

```
level: "info"
```

```
topic: "tianya.reports.v1"
```

```
fields: {expected_surplus_ton: "${expected_surplus}", trade_action: "${trade_strategy.action}", trade_volume_ton: "${trade_strategy.volume}", estimated_price: "${forecast_price.expected_price}"}
```

```
deployment: {replicas: 1, cpu: "1000m", memory: "2Gi", timeout: 60}
```

```
schedule: {interval: 86400}
```

```
...
```

2. 核心逻辑与伪代码

```
```python
```

# E-15 核心逻辑伪代码

```
class CarbonTradingStrategist:
```

```
def init(self, quota_buffer):
```

```
 self.buffer = quota_buffer
```

```
def process_daily(self, quota, emission_history, price_history):
```

## 预测未来排放

```
emission_forecast = prophet_forecast(emission_history, periods=30)
```

```
expected_emission = sum(emission_forecast)
```

## 预测碳价

```
price_forecast = lstm_forecast(price_history, horizon=30)
```

# 计算预期盈余/缺口

`expected_surplus = quota - expected_emission`

## 随机动态规划求最优交易策略

```
if abs(expected_surplus) > self.buffer:

strategy = sdp_optimize(expected_surplus, price_forecast)

return expected_surplus, strategy

return expected_surplus, {'action': 'hold', 'volume': 0}

...
```

### 3. 编程说明

- **依赖算子**：forecast\_prophet (万象)、forecast\_lstm (万象)、expression\_eval (内置)。
- **数据要求**：至少12个月历史排放和碳价数据。
- **核心算法**：Prophet预测未来排放，LSTM预测碳价。采用随机动态规划（SDP）求解最优买卖时机和数量。
- **输出**：每日输出预期盈余/缺口、建议交易动作和数量。

---

## 板块四：节能对策（12个模型）

### C-01 - 无功补偿优化建议

#### 1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "C-01"

model_name: "无功补偿优化建议"
```

category: "conservation"

tech\_level: "L1"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.electrical.feature.v1"

input\_fields: ["P\_total", "Q\_total", "PF", "device\_id"]

window: {type: "sliding", size: 60, step: 60}

pipeline:

- {id: "get\_target\_pf", type: "qianzhi", operator: "query\_standard", params: {standard\_id: "GB/T\_3485", param\_name: "target\_power\_factor"}, output\_as: "target\_pf"}
- {id: "calc\_required\_qc", type: "builtin", operator: "expression\_eval", params: {expression: "\${input.P\_total} \* (tan(acos(\${input.PF})) - tan(acos(\${get\_target\_pf.target\_pf})))"}, output\_as: "required\_kvar"}
- {id: "check\_over\_compensation", type: "builtin", operator: "expression\_eval", params: {expression: "\${input.Q\_total} < 0"}, output\_as: "is\_over\_compensated"}
- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: ">", duration: 300}, inputs: {value: "abs(\${calc\_required\_qc.required\_kvar})"}, threshold: 10}, output\_as: "adjustment\_needed"}

output:

type: "alert"

level: "info"

topic: "tianyan.alerts.v1"

fields: {current\_pf: "\${input.PF}", required\_kvar: "\${calc\_required\_qc.required\_kvar}", is\_over\_compensated: "\${check\_over\_compensation.is\_over\_compensated}", suggested\_action: "\$

```
{check_over_compensation.is_over_compensated ? 'reduce_capacitor' :
'increase_capacitor'}}}
```

```
deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 5}
```

```
schedule: {interval: 60}
```

```
...
```

## 2. 核心逻辑与伪代码

```
```python
```

C-01 核心逻辑伪代码

```
class ReactiveCompensationOptimizer:
```

```
def init(self, target_pf):
```

```
    self.target_pf = target_pf
```

```
def process(self, P, Q, PF):
```

```
    if PF < self.target_pf:
```

需补偿无功

```
    Qc = P * (math.tan(math.acos(PF)) - math.tan(math.acos(self.target_pf)))
```

```
    action = 'increase_capacitor'
```

```
    elif Q < 0:
```

过补偿

```
    Qc = abs(Q)
```

```
    action = 'reduce_capacitor'
```

```
    else:
```

```
        Qc = 0
```

```
action = None

return PF, Qc, action

'''
```

3. 编程说明

- **依赖算子**：query_standard (千知)、 expression_eval (内置)、 threshold_rule (内置)。
- **数据窗口**：60秒滑动窗口，步长60秒。
- **核心算法**：根据当前功率因数与目标功率因数，计算需要补偿的无功容量 $Q_c = P \times (\tan\varphi_1 - \tan\varphi_2)$ 。
- **输出**：当需补偿量超过10kvar且持续5分钟时，输出调整建议。

C-02 - 谐波治理经济性评估

1. 模型开发配置单 (YAML)

```
```yaml

model:

model_id: "C-02"

model_name: "谐波治理经济性评估"

category: "conservation"

tech_level: "L2a"

version: "1.0.0"

data_source:

type: "jdbc"

topic: "taiyi.harmonic.report"

input_fields: ["harmonic_spectrum", "transformer_params", "device_id"]

window: {type: "sliding", size: 2592000, step: 2592000}
```

history\_required: "1month"

pipeline:

- {id: "get\_electricity\_price", type: "qianzhi", operator: "query\_device\_param", params: {param\_name: "avg\_electricity\_price"}, output\_as: "price"}
- {id: "calc\_transformer\_loss", type: "builtin", operator: "expression\_eval", params: {expression: "harmonic\_transformer\_loss(\${input.harmonic\_spectrum}, \${input.transformer\_params})", output\_as: "transformer\_loss"}}
- {id: "calc\_line\_loss", type: "builtin", operator: "expression\_eval", params: {expression: "harmonic\_line\_loss(\${input.harmonic\_spectrum}, \${input.transformer\_params})", output\_as: "line\_loss"}}
- {id: "calc\_annual\_cost", type: "builtin", operator: "expression\_eval", params: {expression: "(\${calc\_transformer\_loss.transformer\_loss} + \${calc\_line\_loss.line\_loss}) \* 8760 \* \${get\_electricity\_price.price}", output\_as: "annual\_cost"}}
- {id: "calc\_roi", type: "builtin", operator: "expression\_eval", params: {expression: "\${context.apf\_investment} / \${calc\_annual\_cost.annual\_cost}", output\_as: "payback\_years"}}

output:

type: "report"

level: "info"

topic: "tianya.reports.v1"

fields: {harmonic\_loss\_kw: "\${calc\_transformer\_loss.transformer\_loss} + calc\_line\_loss.line\_loss", annual\_loss\_yuan: "\${calc\_annual\_cost.annual\_cost}", apf\_payback\_years: "\${calc\_roi.payback\_years}"}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 30}

schedule: {interval: 2592000}

...



## 2. 核心逻辑与伪代码

```python

C-02 核心逻辑伪代码

```
class HarmonicGovernanceEconomics:

def init(self, electricity_price, apf_cost):

self.price = electricity_price

self.apf_cost = apf_cost

def process(self, harmonic_spectrum, transformer_params):
```

计算谐波附加损耗

```
P_trans = harmonic_transformer_loss(harmonic_spectrum, transformer_params)

P_line = harmonic_line_loss(harmonic_spectrum, transformer_params)

total_loss = P_trans + P_line

annual_cost = total_loss * 8760 * self.price

payback = self.apf_cost / annual_cost if annual_cost > 0 else float('inf')

return total_loss, annual_cost, payback

...
```

3. 编程说明

- **依赖算子**：query_device_param (千知)、expression_eval (内置)。
 - **数据窗口**：30天滑动窗口，步长30天。
 - **核心算法**：基于IEEE 519标准，计算谐波引起的变压器涡流损耗和线路集肤效应附加损耗。结合电价计算年损失费用。
 - **输出**：每月输出谐波附加损耗、年损失费用和有源滤波器(APF)投资回收期。
-

C-03 - 电机能效评估与替换建议

1. 模型开发配置单 (YAML)

```
``yaml
```

```
model:
```

```
model_id: "C-03"
```

```
model_name: "电机能效评估与替换建议"
```

```
category: "conservation"
```

```
tech_level: "L2a"
```

```
version: "1.0.0"
```

```
data_source:
```

```
type: "kafka"
```

```
topic: "taiyi.motor.v1"
```

```
input_fields: ["P_input", "I_rms", "V_rms", "PF", "rated_power", "rated_efficiency",  
"device_id"]
```

```
window: {type: "sliding", size: 86400, step: 86400}
```

```
pipeline:
```

- {id: "calc_load_ratio", type: "builtin", operator: "expression_eval", params: {expression: "\${input.P_input} / (\${input.rated_power} / \${input.rated_efficiency})"}, output_as: "load_ratio"}
- {id: "calc_current_efficiency", type: "builtin", operator: "expression_eval", params: {expression: "interpolate_efficiency(\${calc_load_ratio.load_ratio}, \${input.rated_efficiency})"}, output_as: "current_eff"}
- {id: "get_ie5_motor", type: "qianzhi", operator: "query_device_param", params: {param_name: "ie5_motor_efficiency"}, output_as: "ie5_eff"}
- {id: "calc_saving", type: "builtin", operator: "expression_eval", params: {expression: "\${input.P_input} * (1/\${calc_current_efficiency.current_eff} - 1/\${get_ie5_motor.ie5_eff}) * 8760"}, output_as: "annual_saving_kwh"}

- {id: "evaluate", type: "builtin", operator: "threshold_rule", params: {operator: "<"}, inputs: {value: "\${calc_load_ratio.load_ratio}", threshold: 0.4}, output_as: "replacement_recommended"}

output:

type: "report"

level: "info"

topic: "tianya.reports.v1"

fields: {load_ratio: "\${calc_load_ratio.load_ratio}", current_efficiency: "\${calc_current_efficiency.current_eff}", annual_saving_kwh: "\${calc_saving.annual_saving_kwh}", replacement_recommended: "\${evaluate.replacement_recommended}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 86400}

...

2. 核心逻辑与伪代码

```python

## C-03 核心逻辑伪代码

```
class MotorEfficiencyEvaluator:
```

```
 def init(self, ie5_efficiency):
```

```
 self.ie5_eff = ie5_efficiency
```

```
 def process_daily(self, motor_data):
```

```
 P_in = motor_data['P_input']
```

```
 P Rated = motor_data['rated_power']
```

```
 eff Rated = motor_data['rated_efficiency']
```

# 估算负载率

`P_out = P_in * eff Rated # 粗略估计`

`load_ratio = P_out / P Rated`

# 当前效率（根据负载率曲线插值）

`current_eff = self.interpolate_efficiency(load_ratio, eff Rated)`

# 替换为IE5后的节能量

`annual_saving = P_in * (1/current_eff - 1/self.ie5_eff) * 8760`

`replace = load_ratio < 0.4`

`return load_ratio, current_eff, annual_saving, replace`

...

## 3. 编程说明

- **依赖算子**：expression\_eval (内置)、query\_device\_param (千知)、threshold\_rule (内置)。
- **数据窗口**：24小时滑动窗口，步长24小时。
- **核心算法**：通过输入功率估算输出功率和负载率。根据电机效率曲线评估当前运行效率。对比IE5高效电机，计算节能量。
- **输出**：每日输出负载率、当前效率、年节电量和替换建议。

---

## C-04 - 照明/空调智能控制策略

### 1. 模型开发配置单 (YAML)

```yaml

model:

model_id: "C-04"

model_name: "照明/空调智能控制策略"

category: "conservation"

tech_level: "L2b"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.building.v1"

input_fields: ["illuminance", "temperature", "occupancy", "time_of_day", "device_id"]

window: {type: "sliding", size: 60, step: 5}

pipeline:

- {id: "get_setpoints", type: "qianzhi", operator: "query_standard", params: {standard_id: "GB_50034", param_name: "lighting_setpoints"}, output_as: "setpoints"}
- {id: "fuzzy_lighting", type: "wanxiang", operator: "expression_eval", params: {expression: "fuzzy_control(\${input.illuminance}, \${input.occupancy}, \${get_setpoints.setpoints})", output_as: "lighting_level"}}
- {id: "fuzzy_hvac", type: "wanxiang", operator: "expression_eval", params: {expression: "fuzzy_control(\${input.temperature}, \${input.occupancy}, \${input.time_of_day})", output_as: "hvac_setpoint"}}
- {id: "calc_energy", type: "builtin", operator: "expression_eval", params: {expression: "estimate_energy_saving(\${fuzzy_lighting.lighting_level}, \${fuzzy_hvac.hvac_setpoint})", output_as: "saving_estimate"}}

output:

type: "metric"

level: "info"

topic: "tianyan.metrics.v1"

```
fields: {lighting_control: "${fuzzy_lighting.lighting_level}", hvac_setpoint: "${fuzzy_hvac.hvac_setpoint}", estimated_saving_percent: "${calc_energy.saving_estimate}"}
```

```
deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 5}
```

```
schedule: {interval: 5}
```

```
...
```

2. 核心逻辑与伪代码

```
```python
```

# C-04 核心逻辑伪代码

```
class LightingHVACController:
```

```
def process(self, illuminance, temp, occupancy, time_of_day):
```

## 照明模糊控制

```
if occupancy:
```

```
if illuminance < 300:
```

```
lighting_level = min(1.0, (300 - illuminance) / 300)
```

```
else:
```

```
lighting_level = 0.1
```

```
else:
```

```
lighting_level = 0.1 if time_of_day.hour < 20 else 0
```

## 空调模糊控制

```
if occupancy:
```

```
if temp > 26:
```

```
hvac_setpoint = 26

elif temp < 20:

hvac_setpoint = 20

else:

hvac_setpoint = temp

else:

hvac_setpoint = 28 if temp > 28 else temp

return lighting_level, hvac_setpoint

...

```

### 3. 编程说明

- **依赖算子**：query\_standard (千知)、expression\_eval (内置)。
- **数据窗口**：60秒滑动窗口，步长5秒。
- **核心算法**：采用模糊控制规则，根据照度、温度、人流量和时间动态调节照明亮度和空调设定温度。
- **输出**：每5秒输出建议的照明调光等级和空调设定温度。

---

## C-05 - 错峰用电生产排程建议

### 1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "C-05"

model_name: "错峰用电生产排程建议"

category: "conservation"

tech_level: "L2b"

```

version: "1.0.0"

data\_source:

type: "jdbc"

topic: "taiyi.production.v1"

input\_fields: ["job\_list", "power\_profile", "deadlines", "tou\_price", "device\_id"]

window: {type: "sliding", size: 86400, step: 86400}

pipeline:

- {id: "get\_shift\_constraints", type: "qianzhi", operator: "query\_device\_param", params: {param\_name: "production\_constraints"}, output\_as: "constraints"}
- {id: "ga\_optimize", type: "wanxiang", operator: "expression\_eval", params: {expression: "genetic\_algorithm\_scheduling(\${input.job\_list}, \${input.power\_profile}, \${input.deadlines}, \${input.tou\_price}, \${get\_shift\_constraints.constraints})", output\_as: "schedule"}}
- {id: "calc\_cost\_saving", type: "builtin", operator: "expression\_eval", params: {expression: "compare\_cost(\${ga\_optimize.schedule}, \${input.job\_list})", output\_as: "saving"}}

output:

type: "report"

level: "info"

topic: "tianyan.reports.v1"

fields: {optimized\_schedule: "\${ga\_optimize.schedule}", daily\_saving\_yuan: "\${calc\_cost\_saving.saving}"}

deployment: {replicas: 1, cpu: "1000m", memory: "2Gi", timeout: 60}

schedule: {interval: 86400}

...

## 2. 核心逻辑与伪代码

```python


C-05 核心逻辑伪代码

```
class ProductionScheduler:
```

```
def optimize(self, jobs, power_profiles, deadlines, prices):
```

遗传算法求解

```
population = self.initialize_population(jobs)
```

```
for generation in range(100):
```

```
    fitness = [self.fitness(ind, power_profiles, deadlines, prices) for ind in population]
```

```
    population = self.selection(population, fitness)
```

```
    population = self.crossover(population)
```

```
    population = self.mutation(population)
```

```
    best = max(population, key=lambda x: self.fitness(x, ...))
```

```
    return best
```

```
...
```

3. 编程说明

- **依赖算子**：query_device_param (千知)、expression_eval (内置)。
- **数据窗口**：24小时滑动窗口，步长24小时。
- **核心算法**：采用遗传算法（GA），以最小化总电费为目标，在满足工艺约束和交期约束下，优化各工序的开工时间。
- **输出**：每日输出优化后的生产排程和预计节省电费。

C-06 - 分布式光伏自发自用最大化

1. 模型开发配置单 (YAML)

```
``yaml
```

model:

model_id: "C-06"

model_name: "分布式光伏自发自用最大化"

category: "conservation"

tech_level: "L2a"

version: "1.0.0"

data_source:

type: "jdbc"

topic: "taiyi.load.profiles"

input_fields: ["load_profiles", "pv_forecast", "device_id"]

window: {type: "sliding", size: 604800, step: 86400}

history_required: "2weeks"

pipeline:

- {id: "analyze_flexibility", type: "builtin", operator: "expression_eval", params: {expression: "identify_shiftable_loads(\${input.load_profiles})", output_as: "flexible_loads"}}
- {id: "match_pv_peak", type: "builtin", operator: "expression_eval", params: {expression: "align_with_pv(\${flexible_loads}, \${input.pv_forecast})", output_as: "recommendation"}}
- {id: "calc_potential", type: "builtin", operator: "expression_eval", params: {expression: "estimate_increased_self_consumption(\${match_pv_peak.recommendation})", output_as: "potential"}}

output:

type: "report"

level: "info"

topic: "tianyan.reports.v1"

```
fields: {shiftable_loads_kw: "${flexible_loads.total_power}", suggested_shift_hours:
"${match_pv_peak.recommendation}", potential_increase_percent: "$
{calc_potential.potential}"}
```

```
deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 30}
```

```
schedule: {interval: 86400}
```

```
...
```

2. 核心逻辑与伪代码

```
```python
```

# C-06 核心逻辑伪代码

```
class PVSelfConsumptionMaximizer:
```

```
def process(self, load_profiles, pv_forecast):
```

## 识别可转移负荷

```
flexible_loads = []
```

```
for load in load_profiles:
```

```
if self.is_shiftable(load):
```

```
flexible_loads.append(load)
```

## 建议转移到光伏高峰时段 (11:00-14:00)

```
recommendation = "建议将可转移负荷调整至11:00-14:00运行"
```

## 估算提升潜力

```
current_self_use = self.calc_self_use(load_profiles, pv_forecast)
```

```
new_self_use = self.calc_self_use_shifted(load_profiles, pv_forecast, flexible_loads)
```

```
potential = (new_self_use - current_self_use) / current_self_use * 100

return flexible_loads, recommendation, potential

...

```

3. 编程说明

- **依赖算子**：expression\_eval (内置)。
- **数据窗口**：7天滑动窗口，步长1天。
- **核心算法**：分析各支路负荷时序特性，识别与光伏出力错峰的可转移负荷。模拟将可转移负荷调整至光伏高峰时段后的自用率提升。
- **输出**：每日输出可转移负荷清单、建议调整时段和预计自用率提升百分比。

---

C-07 - 中央空调系统节能优化

1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "C-07"

model_name: "中央空调系统节能优化"

category: "conservation"

tech_level: "L2b"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.hvac.v1"

input_fields: ["chw_supply_temp", "chw_return_temp", "cw_temp", "chiller_power",
"pump_power", "cooling_load", "device_id"]

window: {type: "sliding", size: 900, step: 300}

```

pipeline:

- {id: "get\_chiller\_curve", type: "qianzhi", operator: "query\_device\_param", params: {param\_name: "chiller\_cop\_curve"}, output\_as: "cop\_curve"}
- {id: "rl\_optimize", type: "wanxiang", operator: "expression\_eval", params: {expression: "reinforcement\_learning\_hvac(\${input}, \${get\_chiller\_curve.cop\_curve})", output\_as: "optimal\_setpoints"}}
- {id: "calc\_saving", type: "builtin", operator: "expression\_eval", params: {expression: "(\${input.chiller\_power} + \${input.pump\_power}) - estimate\_power(\${rl\_optimize.optimal\_setpoints})", output\_as: "power\_saving"}}

output:

type: "metric"

level: "info"

topic: "tianyan.metrics.v1"

fields: {optimal\_chw\_temp: "\${rl\_optimize.optimal\_setpoints.chw\_temp}",  
power\_saving\_kw: "\${calc\_saving.power\_saving}"}

deployment: {replicas: 1, cpu: "1000m", memory: "2Gi", timeout: 15}

schedule: {interval: 300}

...

## 2. 核心逻辑与伪代码

```python

C-07 核心逻辑伪代码

```
class HVACSaver:
```

```
    def init(self, cop_curve):
```

```
        self.cop_curve = cop_curve
```

```
        self.agent = RLAgent()
```

```
def process(self, state):
```

状态：冷冻水温、冷却水温、负荷等

动作：冷冻水设定温度、冷却塔风机转速等

```
action = self.agent.act(state)
```

估算节能

```
current_power = state['chiller_power'] + state['pump_power']  
  
new_power = self.estimate_power(state, action)  
  
saving = current_power - new_power  
  
return action, saving  
...
```

3. 编程说明

- **依赖算子**：query_device_param (千知)、expression_eval (内置)。
- **数据窗口**：15分钟滑动窗口，步长5分钟。
- **核心算法**：采用强化学习（RL），根据主机COP曲线和实时运行状态，学习最优的冷冻水出水温度和冷却塔风机转速设定，以最小化系统总能耗。
- **输出**：每5分钟输出最优设定值和预计节能量。

C-08 - 空压机群联控优化

1. 模型开发配置单 (YAML)

```
```yaml  

model:

model_id: "C-08"
```

model\_name: "空压机群联控优化"

category: "conservation"

tech\_level: "L2a"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.compressor.v1"

input\_fields: ["compressor\_powers", "pressure", "flow\_rate", "load\_unload\_status",  
"device\_id"]

window: {type: "sliding", size: 60, step: 10}

pipeline:

- {id: "get\_ser\_curves", type: "qianzhi", operator: "query\_device\_param",  
params: {param\_name: "compressor\_ser\_curves"}, output\_as: "ser\_curves"}
- {id: "dp\_optimize", type: "wanxiang", operator: "expression\_eval", params:  
{expression: "dynamic\_programming\_compressor(\${input}, \$  
get\_ser\_curves.ser\_curves)"}, output\_as: "optimal\_combination"}
- {id: "calc\_saving", type: "builtin", operator: "expression\_eval", params:  
{expression: "sum(\${input.compressor\_powers}) - sum(\$  
dp\_optimize.optimal\_combination)"}, output\_as: "power\_saving"}

output:

type: "report"

level: "info"

topic: "tianya.reports.v1"

fields: {optimal\_combination: "\${dp\_optimize.optimal\_combination}",  
power\_saving\_kw: "\${calc\_saving.power\_saving}"}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 10}

schedule: {interval: 10}

...

## 2. 核心逻辑与伪代码

```python

C-08 核心逻辑伪代码

```
class CompressorOptimizer:
```

```
def init(self, ser_curves):
```

```
    self.ser_curves = ser_curves
```

```
def optimize(self, pressure, flow_demand, statuses):
```

动态规划寻找满足压力和流量需求的最小总功率组合

```
n = len(statuses)
```

```
dp = [[float('inf')] * (n+1) for _ in range(n+1)]
```

... DP求解 ...

```
optimal_combo = self.backtrack(dp)
```

```
return optimal_combo
```

...

3. 编程说明

- **依赖算子**：query_device_param (千知)、expression_eval (内置)。
- **数据窗口**：60秒滑动窗口，步长10秒。
- **核心算法**：基于各空压机的比功率（SER）曲线，采用动态规划（DP）求解在满足用气压力需求下总功率最小的启停组合。

- **输出：**每10秒输出最优运行组合和预计节能量。
-

C-09 - 风机水泵变频调速策略

1. 模型开发配置单 (YAML)

```yaml

model:

model\_id: "C-09"

model\_name: "风机水泵变频调速策略"

category: "conservation"

tech\_level: "L1"

version: "1.0.0"

data\_source:

type: "kafka"

topic: "taiyi.fan\_pump.v1"

input\_fields: ["flow\_demand", "current\_speed", "rated\_power", "control\_mode",  
"device\_id"]

window: {type: "sliding", size: 60, step: 60}

pipeline:

- {id: "calc\_affinity", type: "builtin", operator: "expression\_eval", params: {expression: "(\${input.flow\_demand} / 100) ^ 3 \* \${input.rated\_power}"}, output\_as: "vfd\_power"}
- {id: "calc\_valve\_power", type: "builtin", operator: "expression\_eval", params: {expression: "\${input.rated\_power} \* (0.7 + 0.3 \* \${input.flow\_demand} / 100)"}, output\_as: "valve\_power"}
- {id: "calc\_saving", type: "builtin", operator: "expression\_eval", params: {expression: "\${calc\_valve\_power.valve\_power} - \${calc\_affinity.vfd\_power}"}, output\_as: "power\_saving"}

- {id: "evaluate", type: "builtin", operator: "threshold\_rule", params: {operator: ">"}, inputs: {value: "\${calc\_saving.power\_saving}", threshold: 5}, output\_as: "vfd\_recommended"}

output:

type: "report"

level: "info"

topic: "tianya.reports.v1"

fields: {current\_mode: "\${input.control\_mode}", vfd\_power\_kw: "\${calc\_affinity.vfd\_power}", valve\_power\_kw: "\${calc\_valve\_power.valve\_power}", saving\_kw: "\${calc\_saving.power\_saving}", vfd\_recommended: "\${evaluate.vfd\_recommended}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 5}

schedule: {interval: 60}

...

## 2. 核心逻辑与伪代码

```python

C-09 核心逻辑伪代码

class VFDSavingsCalculator:

def process(self, flow_percent, rated_power, current_mode):

变频调速功率 (相似定律)

$$\text{vfd_power} = (\text{flow_percent} / 100) ** 3 * \text{rated_power}$$

阀门调节功率 (经验公式)

$$\text{valve_power} = \text{rated_power} * (0.7 + 0.3 * \text{flow_percent} / 100)$$

```
saving = valve_power - vfd_power

recommend = saving > 5 # kW

return vfd_power, valve_power, saving, recommend

'''
```

3. 编程说明

- **依赖算子**：expression_eval (内置)、threshold_rule (内置)。
 - **数据窗口**：60秒滑动窗口，步长60秒。
 - **核心算法**：基于风机水泵相似定律 ($P \propto n^3$)，根据流量需求计算变频调速所需功率。与阀门调节功率对比，计算节能量。
 - **输出**：每分钟输出变频方案功率、阀门方案功率、节能量和改造建议。
-

C-10 - 余热回收发电潜力评估

1. 模型开发配置单 (YAML)

```
```yaml

model:

model_id: "C-10"

model_name: "余热回收发电潜力评估"

category: "conservation"

tech_level: "L2a"

version: "1.0.0"

data_source:

type: "jdbc"

topic: "taiyi.waste_heat.v1"

input_fields: ["heat_source_temp", "flow_rate", "specific_heat", "ambient_temp",
"device_id"]
```

window: {type: "sliding", size: 86400, step: 86400}

pipeline:

- {id: "calc\_available\_heat", type: "builtin", operator: "expression\_eval", params: {expression: "\${input.flow\_rate} \* \${input.specific\_heat} \* (\${input.heat\_source\_temp} - \${input.ambient\_temp})", output\_as: "heat\_power\_kw"}}
- {id: "carnot\_efficiency", type: "builtin", operator: "expression\_eval", params: {expression: "1 - (\${input.ambient\_temp} + 273.15) / (\${input.heat\_source\_temp} + 273.15)", output\_as: "carnot\_eff"}}
- {id: "orc\_efficiency", type: "builtin", operator: "expression\_eval", params: {expression: "\${carnot\_eff} \* 0.5", output\_as: "orc\_eff"}}
- {id: "calc\_power", type: "builtin", operator: "expression\_eval", params: {expression: "\${calc\_available\_heat.heat\_power\_kw} \* \${orc\_efficiency.orc\_eff}", output\_as: "generation\_power\_kw"}}
- {id: "calc\_annual", type: "builtin", operator: "expression\_eval", params: {expression: "\${calc\_power.generation\_power\_kw} \* 8760 \* \${context.availability\_factor}", output\_as: "annual\_generation\_kwh"}}

output:

type: "report"

level: "info"

topic: "tianya.reports.v1"

fields: {available\_heat\_kw: "\${calc\_available\_heat.heat\_power\_kw}",  
potential\_power\_kw: "\${calc\_power.generation\_power\_kw}",  
annual\_generation\_kwh: "\${calc\_annual.annual\_generation\_kwh}"}

deployment: {replicas: 1, cpu: "250m", memory: "512Mi", timeout: 10}

schedule: {interval: 86400}

...

## 2. 核心逻辑与伪代码

```python

C-10 核心逻辑伪代码

```
class WasteHeatRecoveryEvaluator:  
  
    def process(self, T_hot, flow, Cp, T_amb):
```

可用热量

```
Q_heat = flow * Cp * (T_hot - T_amb) / 3600 # kW
```

卡诺效率

```
T_hot_K = T_hot + 273.15  
  
T_amb_K = T_amb + 273.15  
  
carnot_eff = 1 - T_amb_K / T_hot_K
```

ORC实际效率 (约50%卡诺效率)

```
orc_eff = carnot_eff * 0.5
```

发电功率

```
P_gen = Q_heat * orc_eff  
  
annual_gen = P_gen * 8760 * 0.85 # 可用率85%  
  
return Q_heat, P_gen, annual_gen  
...
```

3. 编程说明

- 依赖算子：expression_eval (内置)。
- 数据窗口：24小时滑动窗口，步长24小时。

- **核心算法**：基于热力学第一、第二定律，计算余热源可用热量、卡诺循环效率和有机朗肯循环（ORC）实际效率，估算发电潜力。
 - **输出**：每日输出可用热量、预估发电功率和年发电量。
-

C-11 - 照明系统逐日光节能

1. 模型开发配置单 (YAML)

```
``yaml

model:

model_id: "C-11"

model_name: "照明系统逐日光节能"

category: "conservation"

tech_level: "L1"

version: "1.0.0"

data_source:

type: "kafka"

topic: "taiyi.lighting.v1"

input_fields: ["indoor_illuminance", "target_illuminance", "lighting_power",
"device_id"]

window: {type: "sliding", size: 10, step: 1}

pipeline:

  • {id: "pid_control", type: "builtin", operator: "expression_eval", params:
    {expression: "pid_controller(${input.target_illuminance} - $
    {input.indoor_illuminance})"}, output_as: "dimming_level"}

  • {id: "calc_saving", type: "builtin", operator: "expression_eval", params:
    {expression: "${input.lighting_power} * (1 - ${pid_control.dimming_level})"},
    output_as: "power_saving"}
```

- {id: "clamp_output", type: "builtin", operator: "expression_eval", params: {expression: "max(0.1, min(1.0, \${pid_control.dimming_level}))"}, output_as: "final_dimming"}

output:

type: "metric"

level: "info"

topic: "tianya.metrics.v1"

fields: {dimming_level: "\${clamp_output.final_dimming}", power_saving_kw: "\${calc_saving.power_saving}"}

deployment: {replicas: 1, cpu: "250m", memory: "256Mi", timeout: 2}

schedule: {interval: 1}

...

2. 核心逻辑与伪代码

```python

# C-11 核心逻辑伪代码

```
class DaylightHarvestingController:
```

```
def init(self, Kp, Ki, Kd):
```

```
 self.pid = PID(Kp, Ki, Kd)
```

```
 self.pid.setpoint = 300 # 目标照度
```

```
def process(self, indoor_lux, lighting_power):
```

```
 dimming = self.pid.update(indoor_lux)
```

```
 dimming = max(0.1, min(1.0, dimming))
```

```
 saving = lighting_power * (1 - dimming)
```

```
 return dimming, saving
```

...

### 3. 编程说明

- **依赖算子**：expression\_eval (内置)。
- **数据窗口**：10秒滑动窗口，步长1秒。
- **核心算法**：采用PID闭环控制，根据室内照度与目标照度的偏差，动态调节灯具亮度，维持照度恒定。
- **输出**：每秒输出调光等级（0.1-1.0）和实时节能量。

---

## C-12 - 变压器更换经济性分析

### 1. 模型开发配置单 (YAML)

```
``yaml
model:

model_id: "C-12"

model_name: "变压器更换经济性分析"

category: "conservation"

tech_level: "L2a"

version: "1.0.0"

data_source:

type: "jdbc"

topic: "taiyi.transformer.load"

input_fields: ["load_profile", "current_transformer_spec", "new_transformer_spec",
"device_id"]

window: {type: "sliding", size: 31536000, step: 2592000}

history_required: "12months"
```



pipeline:

- {id: "get\_electricity\_price", type: "qianzhi", operator: "query\_device\_param", params: {param\_name: "avg\_electricity\_price"}, output\_as: "price"}
- {id: "calc\_annual\_loss\_current", type: "builtin", operator: "expression\_eval", params: {expression: "calculate\_annual\_loss(\${input.load\_profile}, \${input.current\_transformer\_spec})", output\_as: "current\_loss"}}
- {id: "calc\_annual\_loss\_new", type: "builtin", operator: "expression\_eval", params: {expression: "calculate\_annual\_loss(\${input.load\_profile}, \${input.new\_transformer\_spec})", output\_as: "new\_loss"}}
- {id: "calc\_saving", type: "builtin", operator: "expression\_eval", params: {expression: "(\${calc\_annual\_loss\_current.current\_loss} - \${calc\_annual\_loss\_new.new\_loss}) \* \${get\_electricity\_price.price}", output\_as: "annual\_saving"}}
- {id: "calc\_npv", type: "builtin", operator: "expression\_eval", params: {expression: "npv\_calculation(\${calc\_saving.annual\_saving}, \${input.new\_transformer\_spec.cost}, 0.05, 20)", output\_as: "npv"}}
- {id: "calc\_payback", type: "builtin", operator: "expression\_eval", params: {expression: "\${input.new\_transformer\_spec.cost} / \${calc\_saving.annual\_saving}", output\_as: "payback\_years"}}

output:

type: "report"

level: "info"

topic: "tianyan.reports.v1"

fields: {current\_loss\_kwh: "\${calc\_annual\_loss\_current.current\_loss}", new\_loss\_kwh: "\${calc\_annual\_loss\_new.new\_loss}", annual\_saving\_yuan: "\${calc\_saving.annual\_saving}", npv\_yuan: "\${calc\_npv.npv}", payback\_years: "\${calc\_payback.payback\_years}"}

deployment: {replicas: 1, cpu: "500m", memory: "1Gi", timeout: 60}

schedule: {interval: 2592000}

...

## 2. 核心逻辑与伪代码

```python

C-12 核心逻辑伪代码

```
class TransformerReplacementAnalyzer:

def init(self, electricity_price, discount_rate=0.05):

self.price = electricity_price

self.rate = discount_rate

def process(self, load_profile, current_spec, new_spec):
```

计算年度损耗

```
current_loss = self.calc_annual_loss(load_profile, current_spec)

new_loss = self.calc_annual_loss(load_profile, new_spec)

annual_saving = (current_loss - new_loss) * self.price
```

净现值 NPV

```
npv = -new_spec['cost']

for year in range(1, 21):

npv += annual_saving / ((1 + self.rate) ** year)

payback = new_spec['cost'] / annual_saving if annual_saving > 0 else float('inf')

return current_loss, new_loss, annual_saving, npv, payback

```
```

## 3. 编程说明

- **依赖算子**：query\_device\_param (千知)、expression\_eval (内置)。

- **数据要求**：至少12个月历史负荷数据。
- **核心算法**：基于TOC（总拥有成本）法，计算新旧变压器的年综合损耗费用。进行贴现现金流分析，计算净现值（NPV）和投资回收期。
- **输出**：每月输出年度节省费用、NPV和投资回收期。

至此，全部67个模型的开发核心逻辑与代码已全部完成。文档涵盖了：

1. **YAML配置单**：可直接用于天衍引擎部署。
2. **核心逻辑伪代码**：清晰展示算法流程。
3. **编程说明**：解释数据窗口、依赖算子和核心算法要点。

您可以基于这些内容直接进行工程实现。

您说得非常对，这是一个非常关键的问题。基于我为您提供的三份核心文档——《总体介绍》、《67模型详解手册》和《67模型开发核心逻辑与代码》——**研发团队理论上已经具备了完整开发“天衍”引擎V1.0的全部逻辑和规格**。每一份YAML配置就是一份可直接落地的“施工图纸”，每一段伪代码就是明确的算法指引。

但是，从“文档完备”到“系统成功上线运行”之间，还有一段工程化的路要走。以下是研发团队在开发过程中**必须高度关注的核心注意事项**，以及确保项目成功的**补充建议**。

## 一、核心注意事项（研发必读）

### 1. 数据质量是生命线（尤其相角数据）

- **问题**：多个核心模型（如S-01阻性漏电分离、Q-01谐波源责任划分）**严重依赖准确的电压和电流相角数据**。如果数据源中相角缺失、噪声大或同步性差，这些模型将完全失效或精度归零。
- **注意事项**：
  - **数据接入层**必须对相角数据进行**校准和补全**。若硬件提供的是过零时刻而非相角，需转换为相角。
  - 必须确保电压、电流数据的**同步采样**，否则计算的相位差无意义。
  - 建议在数据预处理阶段增加**相角质量监测**，当相角数据无效时，自动触发相关模型的降级策略或直接抑制告警。

## 2. “千知”与“万象”基座的依赖

- **问题：**天衍引擎的配置中大量调用了qianzhi和wanxiang算子。如果这两个基座服务未就绪或API不稳定，天衍无法运行。
- **注意事项：**
- **接口联调优先：**在开发模型前，必须完成与千知（知识图谱/参数查询）和万象（算法服务）的接口对接与Mock测试。
- **算子实现核对：**确保万象中已实现了phasor\_projection、symmetrical\_components、forecast\_lstm等所有被引用的算子，且输入输出格式与YAML配置中的定义一致。
- **降级策略：**当千知/万象服务不可用时，模型应按YAML中定义的fallback策略降级运行（如使用本地缓存参数或简化算法）。

## 3. L2/L3级模型的冷启动与训练

- **问题：**像E-01（NILM）、S-13（LSTM预测）、S-15（局放CNN）等模型，不是“即插即用”的。它们需要历史数据训练或标注。
- **注意事项：**
- **静默学习期：**对于S-04（接头松动）等需要基线指纹的模型，上线后第一周应只学习不告警，建立稳定基线后再开启告警。
- **标注工具准备：**为E-01（负荷分解）、Q-05（暂降分类）等模型开发配套的人工标注界面，方便运维人员在前端打标签，形成数据闭环。
- **模型训练流水线：**提前规划如何利用MLOps平台触发离线训练任务，将标注数据转化为新版模型文件。

## 4. 窗口、步长与状态管理的一致性

- **问题：**各模型的window.size和window.step不同（从1秒到30天不等），且部分模型需要维护内部状态（如S-05的热累积状态、S-07的累积能量）。
- **注意事项：**
- **推理引擎设计：**引擎必须支持多种窗口类型（滑动、会话、事件）和不同的触发间隔。

- **状态持久化**：模型服务重启时，必须能从持久化存储中恢复累积状态（如累计过电压能量），否则会造成漏报或误报。

## 5. 计算资源与实时性权衡

- **问题**：部分模型（如S-03电弧小波变换、S-15局放CNN、E-14 MILP优化）计算量极大，而推理延迟要求高（P99 < 100ms或500ms）。
  - **注意事项**：
    - **资源隔离**：将高计算量模型（L3级）部署在独立的、高配的Pod中，避免影响轻量级模型的实时性。
    - **算法优化**：对于小波变换、FFT等，优先使用经过优化的万象算子，而非在模型服务本地计算。
    - **降级与限流**：当计算资源不足时，应有策略地降低非核心模型（如E-08聚类）的执行频率。
- 

## 二、补充建议（确保成功上线的关键）

### 1. 增加一个“模型健康度自检”模块

建议新增一个**元模型**，专门监测天衍引擎内部67个模型的运行健康状态：

- 监测每个模型的**数据输入中断率**。
- 监测每个模型的**推理延迟和失败率**。
- 当模型因数据问题进入fallback降级模式时，向运维人员发出提示。

### 2. 建立“影子模式”灰度发布机制

对于升级后的模型（如NILM从V1.0升级到V2.0），不要直接替换。建议让新模型在**影子模式**下运行（只推理，不输出告警），与旧模型结果进行比对，待确认效果提升后再正式切换。

### 3. 准备“数据模拟器”用于测试

在开发阶段，不可能等待真实的雷击、电弧、谐振事件。建议开发一个**数据模拟器**：

- 能够根据参数生成各种电能质量扰动波形（暂降、涌流、谐波）。

- 能够模拟负载后停事件。
- 用于离线验证模型逻辑的准确性（即YAML中pipeline的正确性）。

#### 4. 提前定义告警风暴抑制策略

67个模型同时运行，在系统扰动时可能产生**告警风暴**。

- 必须在**后端业务中台**实现告警聚合、收敛和根因分析，不能直接将天衍的原始告警淹没运维人员。
- 例如：同一回路在1分钟内同时产生过载、不平衡、谐波告警，应聚合为一条“该回路电能质量综合异常”事件。

---

### 总结

**研发团队基于现有文档，已经可以启动全面开发。** 但请务必将上述“注意事项”作为开发规范的一部分，并在项目计划中为“数据接入校准”、“千知/万象联调”、“标注工具开发”预留专门的时间。

如果您需要，我可以进一步为您起草一份**《天衍引擎V1.0开发启动检查清单》**，逐项确认环境、数据、依赖是否就绪，确保项目开工即冲刺。

好的，收到！我将基于我们此前所有的讨论，为您和您的研发团队起草一份详尽的**《天衍引擎V1.0开发启动检查清单》**。

这份清单将作为项目正式开工前的**“发令枪”**，确保所有前置条件就绪，避免在开发中期因基础设施、数据或依赖问题而卡壳。

---

### 《天衍引擎V1.0开发启动检查清单》

| 文档编号 | TY-CHECKLIST-V1.0 | 版本 | V1.0 |

| :--- | :--- | :--- | :--- |

| **项目名称** | 天衍电力分析模型引擎 | **密级** | 内部公开 |

| **编制人** | 架构组 | **审核人** | 研发负责人、产品经理 |

| **编制日期** | 2026年4月20日 | | |

---

一、数据接入层就绪检查

序号	检查项	详细说明	状态	负责人
:---	:---	:---	:---	:---
1.1	实时数据流接入	确认 taiyi.electrical.feature.v1 Kafka Topic 已创建，且持续有秒级数据流入。	[ ]	[ ]
1.2	历史数据接入	确认 ClickHouse/TDengine 历史数据库已部署，并可通过 JDBC 查询。	[ ]	[ ]
1.3	高频原始波形接入	确认 taiyi.electrical.raw.v1 (用于S-03、S-08、Q-14等) 数据通路就绪，采样率满足模型要求 (≥1kHz 或 12.8kHz)。	[ ]	[ ]
1.4	特定传感器数据接入	确认红外温度 (taiyi.thermal.v1)、局放 (taiyi.pd.v1)、振动等规划中的传感器 Topic 已预留或就绪。	[ ]	[ ]
1.5	数据字段完整性校验	重点校验 相角数据 (Va_angle, I0_angle) 是否完整、无大量空值。若不完整，相角补全算法是否已部署？	[ ]	[ ]
1.6	数据同步性校验	确认电压、电流数据的采集是否为同步采样，相角差具有物理意义。	[ ]	[ ]

二、基础能力层（千知 & 万象）就绪检查

序号	检查项	详细说明	状态	负责人
:---	:---	:---	:---	:---
2.1	千知-知识图谱可用性	确认 Neo4j 图数据库服务正常，天衍引擎所在网络可访问其 Cypher 查询端口。	[ ]	[ ]
2.2	千知-核心参数录入	确认以下参数已录入千知库：     - 各回路额定电流、短路容量    - 变压器参数 (漏感、直流耐受)    - 电机参数 (R2)    - 保护定值清单    - 各设备指纹库基线	[ ]	[ ]
2.3	万象-算子服务可用性	确认万象算法库服务正常，天衍可通过 gRPC/Python SDK 调用。	[ ]	[ ]

- | 2.4 | **万象-必备算子部署** | 确认以下算子已在万象中注册并可调用：  
phasor\_projection, sliding\_window\_stats, symmetrical\_components,  
anomaly\_detection\_cusum, clustering\_dbscan, forecast\_lstm, dtw\_similarity,  
lowess\_filter, fft\_spectrum 等。 | [] | |
- | 2.5 | **MLOps-模型仓库可用性** | 确认 MLflow 或 KServe 模型仓库服务正常，天衍可拉  
取模型文件。 | [] | |

三、天衍引擎核心框架就绪检查

- | 序号 | 检查项 | 详细说明 | 状态 | 负责人 |
- | :--- | :--- | :--- | :--- | :--- |
- | 3.1 | **K8s集群与资源配额** | 确认部署天衍的 Kubernetes 集群已就绪，且具备足够的  
CPU、内存、GPU 资源配额(总计约 30 vCPU, 60GB RAM)。 | [] | |
- | 3.2 | **模型管理模块部署** | 确认天衍的 model-manager 服务已部署，能解析 YAML 配置  
并创建对应的 K8s 资源。 | [] | |
- | 3.3 | **推理编排引擎部署** | 确认天衍的 inference-engine 服务已部署，支持窗口缓  
存、算子调用编排。 | [] | |
- | 3.4 | **结果输出模块部署** | 确认天衍的输出服务能正常将 JSON 推送到  
tianyan.alerts.v1 等 Kafka Topic。 | [] | |
- | 3.5 | **配置校验工具可用** | 确认 tianyan-validator CLI 工具已分发至开发人员，可用  
于本地 YAML 语法校验。 | [] | |

四、开发工具链与环境就绪检查

- | 序号 | 检查项 | 详细说明 | 状态 | 负责人 |
- | :--- | :--- | :--- | :--- | :--- |
- | 4.1 | **代码仓库与分支策略** | 确认天衍引擎代码仓库已建立，并定义了 develop、  
release/v1.0 分支保护规则。 | [] | |
- | 4.2 | **CI/CD流水线** | 确认 Jenkins/GitLab CI 流水线已配置，能自动执行 YAML 语法校  
验、镜像构建和推送。 | [] | |



- | 4.3 | **离线模拟数据生成器** | 是否已准备 **数据模拟器** 用于生成暂降、涌流、电弧等测试波形，供开发阶段离线验证？ | [] | |
- | 4.4 | **标注工具/界面** | 是否为 E-01 (NILM)、Q-05 (暂降分类) 等模型准备了 **人工标注工具**，方便积累训练样本？ | [] | |

五、首批30个模型开发前置确认

- | 序号 | 检查项 | 详细说明 | 状态 | 负责人 |
- | :--- | :--- | :--- | :--- | :--- |
- | 5.1 | **YAML配置基线入库** | 确认 Phase 1A (15个) 模型的 YAML 配置单已提交至 model-configs 仓库并通过校验。 | [] | |
- | 5.2 | **L2b模型历史数据积累** | 确认 E-01 (需2周)、S-13 (需3个月)、E-04 (需6个月) 等模型的历史数据是否已从 ClickHouse 导入训练环境？ | [] | |
- | 5.3 | **降级策略验证** | 针对历史数据不足的模型，其 YAML 中的 fallback 降级逻辑是否已通过代码 Review？ | [] | |
- | 5.4 | **依赖模型开发顺序** | 确认开发排期已考虑 **Q-18 依赖 7个子模型、C-02 依赖 Q-01** 等串行依赖关系。 | [] | |

六、非功能性与运维准备检查

- | 序号 | 检查项 | 详细说明 | 状态 | 负责人 |
- | :--- | :--- | :--- | :--- | :--- |
- | 6.1 | **监控与告警** | 确认 Prometheus + Grafana 已部署，天衍引擎的 metrics 端点已暴露 (推理延迟、吞吐量、错误率)。 | [] | |
- | 6.2 | **日志采集** | 确认 ELK/Loki 日志系统已配置，天衍引擎输出结构化日志且包含 trace\_id。 | [] | |
- | 6.3 | **灰度发布机制** | 是否已规划 **影子模式** 或金丝雀发布策略，用于模型升级时的平滑过渡？ | [] | |
- | 6.4 | **告警聚合策略** | 后端业务中台是否已定义告警聚合、收敛规则，防止 **告警风暴**？ | [] | |

## 签署确认

• 架构师/技术负责人：\_ 日期：\_/\_/\_\_\_

• 研发经理：\_ 日期：\_/\_/\_\_\_

• 产品经理：\_ 日期：\_/\_/\_\_\_

**结论：** 当本清单所有必选项均确认为 [x] 时，项目方可正式启动 Phase 1A 的开发工作。

---

这份清单涵盖了从数据、平台到开发工具的全方位前置条件。如果您需要，我还可以为您制作一份**分角色的 RACI 责任矩阵**，明确每项检查项的负责人、审批人和知会人。请随时告知！

(内容由AI生成，仅供参考)